APPLICATION DEVELOPMENT BLUEPRINT · WRITTEN FOR AN AI BUILD AGENT

MySpace One

One offline-first Android app — Calculator · Notes · Scanner · Calendar · Files · Music · Recorder · Diagnostics · On-device AI

From an empty repository to a fully tested, signed, installable APK: requirements, experience design, architecture, on-device AI integration, agent methodology, testing, release engineering and maintenance — all stages and cycles, with machine-checkable acceptance criteria.

Document	MySpace One — combined-app development blueprint
Version / status	1.1 · approved for build execution
Date	31 October 2026
Primary reader	Autonomous AI build agent (secondary: human reviewers)
Target platform	Android 8.0+ · phones and tablets · no internet, no sign-in
Deliverable	Signed release APK + full evidence bundle + test reports

Contents

Executive summary 20

About this blueprint 21

Who reads this document 21

What this document commits to 21

How to read this document 21

Document conventions 22

Scope: in and out 22

Success criteria for the whole program 23

PART I — FOUNDATIONS, SCOPE AND PROMISE 24

Chapter 1 — Mission, vision and non-negotiables 25

1.1 Mission statement 25

1.2 The five promises 25

1.3 Non-negotiable constraints 25

1.4 Permanently out of scope 26

1.5 Defaults where this document is silent 26

Chapter 2 — Why combined, why offline: product rationale 27

2.1 The case for one app 27

2.2 The case for on-device, always-on offline 27

2.3 Risks of the combined approach and their engineered answers 28

2.4 Reference points the owner recognizes 28

2.5 Why-combined per module (value table) 28

2.6 Alternatives considered (and why combined won) 29

Chapter 3 — Users, personas and environments 30

3.1 Personas 30

3.2 Device tiers and environments 30

3.3 Accessibility as a persona, not a feature . . . 30
3.4 Locale expectations . . . 30
3.5 Environment acceptance matrix . . . 31
3.6 Persona journey narratives (design anchors) . . . 31
3.7 Persona → priority influence map . . . 32

PART II — EXPERIENCE DESIGN 33

Chapter 4 — Experience principles . . . 34
4.1 The eight principles . . . 34
4.2 Emotional design goals . . . 34
4.3 What the experience must never do . . . 34
4.4 Principle conflict resolution (worked cases) . . . 35
Chapter 5 — Information architecture and navigation . . . 36
5.1 The hub model . . . 36
5.2 Navigation graph . . . 36
5.3 Hub screen specification . . . 37
5.4 Cross-module object flows . . . 38
5.5 Hub search behavior detail . . . 38
5.6 Hub region inventory (normative rows) . . . 39
5.7 Ranking worked examples (fixtures mirrored in TC-HUB-006) . . . 39
5.8 Continue-row state line catalogue . . . 39
Chapter 6 — Design system . . . 40
6.1 System overview . . . 40
6.2 Design tokens . . . 40
6.3 Typography and iconography . . . 40
6.4 Components inventory . . . 41
6.5 Motion . . . 41

6.6 Accessibility contract (system-wide) . . . 41
6.7 Type scale and spacing tokens (complete) . . . 42
6.8 Component behavior specifications (normative digest) . . . 43
6.9 Key-theme palettes (normative values) . . . 43
6.10 Key shapes (normative geometry) . . . 44
6.11 Illustration and empty-state art style . . . 44
6.12 UI design grammar (how any screen is composed) . . . 44
6.13 Figma workflow — design linking, usage and the design↔code loop . . . 45
6.14 Screen inventory with Figma frame registry (master UI table) . . . 47
6.15 Redline specifications (numbers an implementer can build from) . . . 49
6.16 Interaction specification index (control-level motion + haptics) . . . 50
6.17 Accessibility annotations (Figma labels → Compose semantics) . . . 50
6.18 Design-QA gate (per-screen UI acceptance) . . . 51
6.19 Where to find anything about UI (the everything-index) . . . 52
Chapter 7 — Module-by-module interaction specification . . . 53
7.1 Hub (Home) . . . 53
7.2 Calculator (the flagship module) . . . 53
7.2.1 Expression field . . . 53
7.2.2 Selection and the memory pills (guide section B) . . . 53
7.2.3 Keypad . . . 54
7.2.4 History sheet . . . 55
7.2.5 Memory sheet and effects . . . 55
7.2.6 Calculator settings sheet . . . 55
7.2.7 MyCalc flavor differences . . . 55
7.2.8 Key-by-key specification (normative) . . . 55
7.3 Notes . . . 57

7.4 Scanner . . . 57
7.5 Calendar . . . 58
7.6 Files . . . 59
7.7 Music . . . 59
7.8 Recorder . . . 60
7.9 Diagnostics . . . 61
7.10 Settings (global) . . . 61
7.11 States, errors, and edge cases (module-spanning) . . . 62
7.12 Responsive behavior (phones, tablets, landscape) . . . 62
7.13 What the UX deliberately does not include . . . 62
7.14 Module edge-case registers (normative) . . . 62
7.15 Screen-by-screen walkthroughs (executor-facing UX narrative) . . . 64
7.15.1 Calculator walkthrough . . . 64
7.15.2 Notes walkthrough . . . 64
7.15.3 Scanner walkthrough . . . 65
7.15.4 Calendar walkthrough . . . 65
7.15.5 Files walkthrough . . . 65
7.15.6 Music walkthrough . . . 65
7.15.7 Recorder walkthrough . . . 65
7.15.8 Diagnostics walkthrough . . . 65
7.15.9 Settings walkthrough . . . 65
7.15.10 Hub walkthrough . . . 66
7.16 Tablet and landscape layout specifications (normative) . . . 66
7.2.9 History card anatomy and memory reorder (normative) . . . 66
7.17 First-run education per module (one line each, shown once, dismissable) . . . 66
7.18 Per-module performance feel budgets (user-perceivable norms) . . . 67

Chapter 8 — UX-level acceptance criteria . . . 68

8.1 UX criterion verification detail (how each UX-0xx is proven) . . . 69

8.2 Extended UX criteria (UX-021...UX-040) . . . 70

8.3 Final UX criteria (UX-041...UX-050) . . . 70

PART III — FEATURE REQUIREMENTS **71**

Chapter 9 — Feature catalogue . . . 72

9.1 How to read the catalogue . . . 72

9.2 Calculator (CALC) — the parity core . . . 72

9.3 MyCalc flavor (NCALC) . . . 74

9.4 Hub (HUB) . . . 74

9.5 Notes (NOTE) . . . 75

9.6 Scanner (SCAN) . . . 75

9.7 Calendar (CAL) . . . 75

9.8 Files (FILE) . . . 76

9.9 Music (MUSIC) . . . 76

9.10 Recorder (REC) . . . 76

9.11 Diagnostics & Settings (DIAG/SET) . . . 76

9.12 Journey tests (cross-module) . . . 77

9.13 Acceptance detail for twenty pivotal P0 features . . . 77

9.14 Negatives (features the catalogue intentionally lacks) . . . 78

9.15 Flagship feature stories (why the top P0s exist) . . . 78

Chapter 10 — Priorities, matrices and cutlines . . . 80

10.1 Priority distribution . . . 80

10.2 Module effort profile . . . 80

10.3 Cutline rules for v1.0 . . . 80

PART IV — ARCHITECTURE AND DATA **81**

Chapter 11 — System architecture . . . 82
11.1 Layered view . . . 82
11.2 Module map (Gradle) . . . 83
11.3 Key architectural decisions (ADR digest) . . . 83
11.4 Threading and concurrency . . . 83
11.5 Error architecture . . . 84
11.6 Dependency injection graph . . . 84
11.7 Data flow examples (end-to-end traces) . . . 84
11.8 Architecture test exemplar (Konsist-style, runs in fast lane) . . . 85
11.9 Threading ownership table (who runs what, where) . . . 85
Chapter 12 — Data architecture . . . 86
12.1 Stores overview . . . 86
12.2 Relational schema (v1) . . . 86
12.3 Migration and integrity policy . . . 87
12.4 Backup format (`.msone`) . . . 88
12.5 Search indexing pipeline . . . 88
12.6 State restoration contract . . . 88
12.7 Index catalog (all indexes in schema v1) . . . 88
12.8 Query pattern contracts (perf-relevant SQL, all via Room) . . . 88
12.9 Entity field dictionaries (complete executor reference) . . . 89
12.10 Backup `data.json` shape (normative) . . . 89
12.11 Query performance validation (fixtures vs budgets) . . . 90
12.12 DataStore schema (prefs store, normative keys) . . . 90
Chapter 13 — Module technical deep dives . . . 91
13.1 Calculator engine (core/domain/calc) . . . 91
13.2 Notes document model . . . 91

13.3 Scanner pipeline . . . 91
13.4 Calendar recurrence engine . . . 91
13.5 Media: Music + Recorder . . . 92
13.6 Files & SAF . . . 92
13.7 Hub search federation . . . 92
13.8 Notifications and alarms inventory . . . 92
13.9 Public API surfaces per module (executor contract tables) . . . 92
13.10 Recorder end-to-end sequence (numbered, for the executor) . . . 94
13.11 Scanner end-to-end sequence (numbered) . . . 94
Chapter 14 — Security and privacy engineering . . . 95
14.1 Threat model (STRIDE-lite, offline) . . . 95
14.2 Privacy guarantees (structural) . . . 95
14.3 Data lifecycle . . . 95
14.4 Permissions matrix (full app) . . . 96
14.5 Data-in-transit and crypto inventory . . . 96
14.6 Security review checklist (per release, automated + human-walkable) . . . 97
14.7 Permission request copy (exact strings, normative) . . . 97
14.8 Data classification register . . . 98
14.9 Android component inventory (merged manifest, normative) . . . 98
14.10 Local incident response runbook (worst-case playbook) . . . 98
14.11 In-app privacy statement (exact text, About → Privacy) . . . 99
14.12 Crypto test vectors (fixed answers the tests pin) . . . 99
PART V — METHOD AND INTELLIGENCE 100
Chapter 15 — Agent methodology: building an app without a human in the loop . . . 101
15.1 The mandate . . . 101
15.2 Operating loop . . . 101

15.3 Task graph . . . 101
15.4 Context strategy (how the agent stays on-spec) . . . 101
15.5 Phase gates . . . 101
15.6 Handling blocked work . . . 102
15.7 Decision ledger discipline . . . 103
15.8 Field lessons baked into this methodology . . . 103
15.9 Gate report template (normative shape) . . . 103
15.10 Worked micro-task (illustrative agent trace) . . . 103
15.11 Agent tool inventory and use policies . . . 103
15.12 Anti-drift reviews (weekly cadence) . . . 104
15.13 Context-recovery worked example (session resume) . . . 104
Chapter 16 — On-device AI strategy . . . 105
16.1 Principles . . . 105
16.2 Capability × module map . . . 105
16.3 Interaction contract for AI features . . . 105
16.4 The AI orchestrator (core/ai) . . . 105
16.5 Models and runtimes (pinned catalogue) . . . 106
16.6 Resource governance . . . 106
16.7 Fallback ladder (per capability) . . . 107
16.8 Pack security and evaluation . . . 107
16.9 What on-device AI will not do here . . . 107
16.10 Orchestrator API reference (core/ai) . . . 108
16.11 Per-capability integration specs (concise) . . . 108
16.12 `core/ai/prompts.yaml` (initial content, normative) . . . 109
16.13 Capability × tier × power behavior matrix (normative) . . . 109
16.14 Pack distribution README (ships beside packs) . . . 110

16.15 Sample AI outputs (calibration fixtures, abbreviated) . . . 110

16.16 Pack-of-record provenance block (About → packs detail) . . . 110

16.17 Summarize flow — numbered orchestrator sequence . . . 110

PART VI — DETAILED TECHNICAL SPECIFICATION 111

Chapter 17 — Detailed specifications . . . 112

17.1 The calculator language . . . 112

17.1.1 Lexical structure . . . 112

17.1.2 Number model — exactness ladder . . . 112

17.1.3 Canonical input form . . . 112

17.1.4 Grammar and parsing (EBNF) . . . 112

17.1.5 Operator semantics — normative table . . . 113

17.1.6 Functions and constants . . . 113

17.1.7 The laws (property-based test suite) . . . 113

17.1.8 Rendering and grouping . . . 114

17.2 History and memory contracts . . . 114

17.3 Notes document format (`NoteDoc`) . . . 114

17.4 Recurrence (supported RRULE subset) . . . 115

17.5 Recorder service . . . 115

17.6 Files handling rules . . . 115

17.7 Settings key registry (normative) . . . 116

17.8 Diagnostics self-test script (normative steps) . . . 116

17.9 Fraction rendering rule (exact path) . . . 116

17.10 Worked evaluation traces (normative examples) . . . 117

17.11 Notification text templates (normative) . . . 117

17.12 Per-module settings keys (registry extension) . . . 117

17.13 Worked parse trees (executor calibration) . . . 117

17.14 Rendering edge table (display rules digest) . . . 118

17.15 Additional worked cases (Appendix B extension, B-31...B-45) . . . 118

PART VII — ENGINEERING STANDARDS AND THE AGENT LIFECYCLE 119

Chapter 18 — Engineering standards . . . 120

18.1 Language and style . . . 120

18.2 Definition of done (per task) . . . 120

18.3 Branching and review . . . 120

18.4 Review checklists (condensed) . . . 120

18.5 Documentation standards . . . 120

18.6 Annotated review example (quality bar for self-reviews) . . . 121

Chapter 19 — The agent lifecycle G0–G8 in full . . . 122

19.1 Phase table . . . 122

19.2 Per-phase verification depth . . . 122

19.3 Metrics that steer the agent . . . 123

19.4 Interruption and resumption protocol . . . 123

19.5 Owner interaction points . . . 123

19.6 Phase task slices (representative task graph rows) . . . 123

19.7 Filled example — G1 gate report (target quality bar) . . . 124

19.8 metrics.json example (per-phase artifact) . . . 124

19.9 Phase-level definitions of ready and done . . . 124

PART VIII — QUALITY: STRATEGY, CATALOGUE, ENVIRONMENTS, HARNESS 125

Chapter 20 — Testing strategy and catalogue . . . 126

20.1 Philosophy . . . 126

20.2 The five layers . . . 126

20.3 Architecture and lint tests . . . 127

20.4 Screenshot discipline . . . 127

20.5 Naming and binding . . . 127
20.6 Catalogue-0 (release-blocking set) . . . 127
20.7 Beyond Catalogue-0 (catalogue digest) . . . 128
20.8 Coverage gates . . . 129
20.9 Defect ledger . . . 129
20.10 Severity/priority interaction and flake policy (normative) . . . 129
20.11 Evidence retention . . . 129
20.12 One exemplar test per layer (house style reference) . . . 130
20.13 Coverage report example (Kover CI artifact, abridged) . . . 130
Chapter 21 — Test environments and data . . . 131
21.1 Emulator matrix . . . 131
21.2 Physical-device delta . . . 131
21.3 Test data fixtures (in-repo, versioned) . . . 131
21.4 Determinism . . . 131
21.5 Fixture generation and provenance . . . 131
21.6 Fixture manifest example (checked by CI) . . . 132
21.7 Emulator provisioning script (runner bootstrap, normative) . . . 132
21.8 Physical-device lab protocol (when a device is attached) . . . 132
21.9 CI schedule summary . . . 132
Chapter 22 — Black-box APK QA harness contract . . . 133
22.1 Scope rule . . . 133
22.2 Pinned-input manifest . . . 133
22.3 Session discipline (per probe) . . . 133
22.4 Stability rules from field lessons . . . 133
22.5 Outputs . . . 134
22.6 Acceptance style . . . 134

22.7 Harness case schema (normative example) . . . 134
22.8 Harness run profiles (normative) . . . 134
22.9 Evidence naming conventions (normative) . . . 135
22.10 Journey YAML sample (J-02, trimmed) . . . 135
22.11 OCR assertion normalization rules (shared with renderer tests) . . . 135
22.12 `user-journey-results.json` sample (schema-compatible with owner tooling) . . . 135
22.13 Harness code skeleton (Python, `qa/` layout contract) . . . 136

PART IX — BUILD, RELEASE AND OPERATIONS 137

Chapter 23 — Build system and configuration . . . 138
23.1 Toolchain (pinned) . . . 138
23.2 Variants . . . 138
23.3 Reproducibility . . . 138
23.4 Flavor parity mechanics . . . 138
23.5 Dependency policy . . . 138
23.6 Size and performance budgets (release, per ABI set) . . . 139
23.7 Variants DSL (trimmed, normative shape) . . . 139
23.8 R8 keep rules (release, near-complete file) . . . 139
23.9 Build performance techniques (why CI fits the budget) . . . 140
23.10 Version catalog (gradle/libs.versions.toml, excerpt) . . . 140
Chapter 24 — CI/CD . . . 141
24.1 Pipeline shape . . . 141
24.2 Runner discipline . . . 141
24.3 Required checks (branch protection) . . . 141
24.4 Audit gates (config as law) . . . 141
24.5 CI workflow sketch (structure, trimmed) . . . 141
24.6 Fast PR lane YAML (trimmed) . . . 142

24.7 Nightly lane additions (beyond PR fast lane) . . . 142
24.8 CI artifact retention . . . 142
Chapter 25 — Release engineering . . . 143
25.1 Versioning . . . 143
25.2 Signing . . . 143
25.3 Shrink and hardening . . . 143
25.4 Pre-flight checklist (automated) . . . 143
25.5 Release artifacts (the bundle) . . . 143
25.6 Distribution (owner model) . . . 143
25.7 RELEASE-NOTES.md template . . . 144
25.8 Signing ceremony (normative steps) . . . 144
25.9 Release QA session script (the G7 instrument) . . . 144
25.10 Filled RELEASE-NOTES example (illustrative v3.13.0) . . . 145
25.11 Bundle verification commands (owner-runnable) . . . 145
Chapter 26 — Post-release operations (G8) . . . 146
26.1 Support model . . . 146
26.2 Update cadence and backports . . . 146
26.3 EOL and data exit . . . 146
26.4 Support playbook (triage decision tree) . . . 146
26.5 Support FAQ (owner-facing, from the runbook) . . . 146
26.6 Owner feedback loop . . . 146
26.7 Update announcement template (owner channel) . . . 147
PART XI — APPENDICES **148**
Appendix A — Complete test-case catalogue . . . 149
A.1 Calculator parity (ONE) and engine (CALC/LAW) . . . 149
A.2 Calculator UX / state / system . . . 149

A.3 Hub / search . . . 150
A.4 Notes . . . 150
A.5 Scanner . . . 150
A.6 Calendar . . . 150
A.7 Files / backup . . . 151
A.8 Music / Recorder . . . 151
A.9 Diagnostics / Settings / Security / System . . . 152
A.10 Journeys (L4/L5) . . . 152
Appendix B — Worked calculator traces (normative examples) . . . 153
Appendix C — Recurrence golden corpus (representative rows) . . . 155
Appendix D — Owner-guide coverage map (B/C/D/E → requirements) . . . 156
PART X — TRACEABILITY, RISK AND ACCEPTANCE 158
Chapter 27 — Traceability matrix . . . 159
27.1 Mechanism . . . 159
27.2 Matrix shape (excerpt) . . . 159
27.3 Traceability script outline (tools/traceability.py) . . . 159
Chapter 28 — Observability and diagnostics (local-only) . . . 160
28.1 Principles . . . 160
28.2 What is recorded (the local ring) . . . 160
28.3 Scrubbing rules . . . 160
28.4 Self-test and health computation . . . 160
28.5 Diagnostics ring schema (normative) . . . 160
28.6 Self-test JSON example (artifact format) . . . 161
28.7 Feature counters (Diagnostics → Usage) . . . 161
Chapter 29 — Maintenance and update policy . . . 162
29.1 Code health practices . . . 162

29.2 Data evolution . . . 162
29.3 Model pack evolution . . . 162
29.4 Documentation maintenance . . . 162
29.5 Model/pack update decision tree (operational) . . . 162
29.6 Deprecation policy (in-app) . . . 162
29.7 Changelog discipline example (RELEASE-NOTES entry quality) . . . 162
29.8 Maintenance calendar (steady state) . . . 163
Chapter 30 — Risk register (live) . . . 164
30.1 Early-warning indicators (watched per phase) . . . 164
30.2 Risk heat map (likelihood × impact placement) . . . 165
30.3 Risk response drills (rehearsed, not just written) . . . 165
Chapter 31 — Glossary . . . 166
Chapter 32 — Final acceptance and sign-off (G7 script) . . . 168
32.1 Sign-off record (filled at G7) . . . 169
32.2 The G7 walkthrough, narrated . . . 169
32.3 Common acceptance questions (owner asks / spec answers) . . . 169
Appendix E — UX microcopy dictionary (normative strings v1, en-IN) . . . 171
Appendix F — Error catalogue (AppError → user message + action) . . . 173
Appendix G — Model pack format (normative) . . . 174
Appendix H — Gesture and control-equivalence map . . . 175
Appendix I — Release runbook (operator steps) . . . 176
Appendix J — Owner constraint register (conversation-derived) . . . 177
Appendix K — Fixture inventory (generated, seeded) . . . 178
Appendix L — Agent working templates . . . 179
Appendix M — Screen inventory (pointer) . . . 180
Appendix N — Repository and DAO method inventory (executor contract) . . . 181

Appendix O — Icon inventory (design system) . . . 182

Appendix P — Sample L5 "report of record" page order (normative) . . . 183

PART XII — EXECUTION APPENDICES **184**

Appendix Q — Catalogue-0 step scripts (executor-ready) . . . 185

Appendix R — Journey step tables (J-01…J-10) . . . 186

Appendix S — Engine reference tables . . . 187

Appendix T — Thirty-second demo script per feature (owner acceptance aid) . . . 188

Appendix U — Severity, priority and SLA matrices . . . 189

Appendix V — Android/compose glossary for the owner . . . 190

Appendix W — Executor quick-start (first working session) . . . 191

Appendix X — Document control . . . 192

Appendix Y — Owner's personal acceptance manual (module by module) . . . 193

Appendix Z — Model cards (pack provenance summaries) . . . 194

Appendix AA — Program FAQ . . . 195

Appendix BB — Index of figures . . . 196

Appendix CC — Constraint → architecture traceability (why the structure is what it is) . . . 197

Appendix DD — Traceability CSV extended excerpt (format example) . . . 198

Appendix EE — "If something fails" (owner-facing, honors the old guide's contract) . . . 199

Appendix FF — Extension guide (adding a module in v1.1+) . . . 200

Appendix GG — Document generation pipeline (how this PDF is built) . . . 201

Appendix HH — Build files (bootstrap reference) . . . 202

Appendix II — G1 day-by-day plan (first sprint, executor-ready) . . . 203

Quick reference card . . . 204

The five promises and ten constraints (from Chapters 1) . . . 204

Command cheat sheet . . . 204
Where everything lives . . . 205
The program in one sentence each . . . 205
Closure . . . 206
Phase-gate checklist card . . . 207
G0 Inception . . . 207
G1 Bootstrap . . . 207
G2 Foundations . . . 207
G3 Calculator . . . 207
G4 Modules . . . 207
G5 On-device AI . . . 207
G6 Hardening . . . 208
G7 Release . . . 208
G8 Operate . . . 208

This blueprint is a complete, self-contained development instruction set for one Android application — **MySpace One** — that combines a calculator, notes, scanner, calendar, files manager, music player, voice recorder and diagnostics into a single install, with on-device artificial intelligence as an optional accelerator. It is written so that an autonomous AI build agent can execute the entire program — from an empty repository to a fully tested, signed, installable APK — without asking a human another question. Every requirement is testable, every stage has entry and exit criteria, and every artifact the agent must produce is named.

Executive summary

What. One Android application — **MySpace One** — that combines Calculator, Notes, Scanner, Calendar, Files, Music, Recorder and Diagnostics behind a single icon, with an optional calculator-only flavor (**MyCalc**) built from the same code. **On-device AI** (OCR, transcription, summarization, semantic search via side-loaded model packs) accelerates features but is never required: every feature works fully offline with AI absent.

The prime directive. The app works **locally, on device, with no sign-in and no internet access — ever.** This is enforced structurally: the merged manifest contains no internet permission, the bytecode contains no network classes, and CI fails on any violation. No ads, no accounts, no telemetry — also structural.

For whom. Written for an **autonomous AI build agent** that executes the whole program — from empty repository to a fully tested, signed APK — using phase gates G0–G8, machine-checkable acceptance criteria, and evidence-first reporting in the style the owner's calculator QA already established: pinned APK SHAs, screenshot evidence, honest PASS/FAIL/BLOCKED states, and a PDF report of record. Publisher success is never conflated with passing results.

How the document is organized. Parts I–III fix identity, experience and features (84 features, each with acceptance criteria and tests). Parts IV–VI specify architecture, data, security, on-device AI and every engine behavior (the calculator language is specified to the grammar, with the owner's historical bugs — `5+ = → 5`, `0.9999999999`, `1e15+1 → Error` — structurally impossible). Parts VII–IX define the agent's methodology, testing, build and release. Part X–XII carry traceability, risks, glossaries and step-level appendices, including the owner's personal 15-minute acceptance manual.

Definition of done for the program. Eight criteria (S1–S8, front-matter table): release APKs install on API 26/33/35 in airplane mode; zero internet-class permissions; the release-blocking test catalogue green; the calculator law suite green (10 000+ randomized cases per law); every feature traced to tests and commits; static audits clean; APK size budgets met; documentation regenerated and archived with the release.

Standing constraints honored. Calculator-only QA focus discipline, ≤ 3 parallel CI runners and ≤ 30 jobs, preservation of historical reports, official-Google-Calculator-only comparisons (never substituted), and BLOCKED-never-FAIL honesty for gestures the test environment cannot fairly reproduce.

About this blueprint

This document exists to remove ambiguity. It is not a brochure or a wish list; it is an operating contract between the product owner and the AI agent that will build the product. It defines what is being built, why, for whom, how it must behave, how it must be architected, how the agent must work, how everything is tested, and what "done" means at every level of granularity.

The document is long by design. A human reader does not need to read it linearly; an executing agent should treat it as its primary context source and re-read the relevant chapter before starting each phase. The table of contents carries bookmark links in the PDF outline for fast navigation.

Who reads this document

- **Primary reader — the AI build agent.** The agent executes phases G0 through G8 (§15.5, §19). For the agent this document is binding: where the agent's intuition conflicts with this document, the document wins. Where the document is silent, the agent must follow the default rules in §1.5 and record the decision in the decision ledger (§15.7).
- **Secondary reader — the product owner.** A human who wants to verify that the product being built is the product they asked for. The owner reviews phase-gate evidence (§15.5) rather than the code itself.
- **Tertiary reader — future maintainers and auditors.** Chapters 12, 14, 20 and 23–26 are written so that a new engineer (human or AI) can understand the system years later without oral history.

What this document commits to

1. **One installable APK** (plus an optional calculator-only flavor of the same codebase) that installs on Android 8.0+ and works with **no sign-in, no internet access, and no background network use**.
2. **Eight modules** — Calculator, Notes, Scanner, Calendar, Files, Music, Recorder, Diagnostics — plus a combined Home hub and Settings, specified to acceptance-test level in Chapters 8–10.
3. **On-device AI** that accelerates features (OCR, transcription, summarization, semantic search) but is never required for any feature to function (§16).
4. **A development methodology** purpose-built for an autonomous agent: phase gates, machine-checkable acceptance criteria, evidence artifacts, and a defect ledger (§15, §19–§20).
5. **A full quality and release pipeline**: unit, integration, screenshot, instrumented and black-box emulator testing; signing; distribution; and a maintenance plan (Chapters 20–26).

How to read this document

- If you are **starting execution**: read Chapter 1 (mission), Chapter 15 (methodology) and Chapter 19 (lifecycle) first; then work phase by phase, re-reading the chapter cited by each task.

- If you are **reviewing scope**: read Chapters 1–3 (why and for whom), Chapters 8–10 (what the product does), Chapter 27 (traceability).
- If you are **reviewing engineering**: read Chapters 11–14 (architecture and data), Chapter 16 (AI), Chapters 20–26 (quality, build, release).
- If you are **testing**: Chapters 8–10 give behavior, Chapter 20 gives the test case catalogue, Chapter 22 gives the harness contract, and Chapter 27 maps everything together.

Document conventions

- **§** denotes a section reference inside this document (e.g., §12.4 is Data architecture, section 4).
- **SHALL / MUST** denote a binding requirement; **SHOULD** denotes a strong default that may be overridden with a recorded decision; **MAY** denotes an explicit option.
- **TC-XXX-nnn** identifies a test case (Chapter 20). **F-nnn** identifies a feature requirement (Chapter 9). **R-nnn** identifies a risk (Chapter 30). **D-nnn** identifies a decision-ledger entry template.
- **Code font** denotes literal identifiers — file names, class names, SQL, commands.
- Every figure has a caption and is referenced from at least one section.
- Word counts are deliberately uneven: specification-dense chapters (8, 9, 17, 20) run much longer than rationale chapters.

Scope: in and out

In scope — the complete lifecycle:

1. Inception and scope agreement (this document, Part I).
2. Experience design: navigation, design system, module-by-module UX specification (Part II).
3. Feature requirements with acceptance criteria (Part III).
4. Architecture, data model, security and privacy engineering (Part IV).
5. On-device AI strategy: models, runtimes, orchestration, fallbacks, packs, evaluation (Part V).
6. Detailed technical specifications: engine grammar, schemas, services, settings (Part VI).
7. The agent's development methodology and lifecycle (Part VII).
8. Quality: test strategy, test catalogue, environments, harness contract (Part VIII).
9. Build, CI/CD, signing, distribution (Part IX).
10. Traceability, observability, maintenance, risks, glossary, sign-off (Part X).

Out of scope — stated plainly so the agent never wanders:

- Any server, cloud service, account system, analytics pipeline or remote-config service.
- iOS, web, or desktop builds (the architecture isolates platform code so ports remain *possible*, but no port is planned here).
- Content syncing *between devices* (local backup/restore files are in scope; a sync protocol is not).
- Monetization of any kind — no ads, no purchases, no offers (§1.3).
- Root/privileged operation or system-app signing.

Success criteria for the whole program

The program is successful when **all** of the following are true simultaneously:

#	Criterion	Evidence artifact
S1	`myspace-one-release.apk` installs and launches on API 26, 33 and 35 emulators with airplane mode ON	CI run logs + emulator videos (§24)
S2	Zero internet-class permissions requested (`INTERNET`, `ACCESS_NETWORK_STATE` absent from merged manifest)	`apkanalyzer manifest` output attached to release evidence
S3	All Catalogue-0 test cases (§20.6) pass on the release build	Test results JSON + PDF report
S4	Calculator engine passes the full property-based law suite (§17.1.7)	Unit-test logs, 10^4+ randomized cases per law
S5	Every feature F-001…F-084 maps to at least one passing test and one commit	Traceability matrix generated by `tools/traceability.py`
S6	Static analysis is clean: zero blocking lint/detekt findings; no `WebKit`, `WebView`, socket, or reflection-to-network APIs in release bytecode	detekt/lint reports + `apkanalyzer` scan
S7	APK size ≤ 30 MB without optional AI packs; AI packs are separate side-loaded files	Build outputs + size budget report (§23.6)
S8	Documentation set is regenerated from the final source tree and archived with the release	Release bundle manifest (§25.6)

These criteria are restated as phase-gate G7 exit conditions in §15.5. If any criterion cannot be met, the agent must file a defect (§20.9) and stop the release — a red release is always preferable to a green lie.

PART I

FOUNDATIONS, SCOPE AND PROMISE

Why this product exists, what it must never do, who it serves, and the constraints that shape every later decision. Read this part first: every technical choice in Parts II–X traces back to a promise or a constraint stated here.

This part fixes the product's identity before a single line of code is specified. It is deliberately short and absolute: missions, promises, constraints and people. Everything else in the blueprint is a consequence of what is written here.

Chapter 1 — Mission, vision and non-negotiables

1.1 Mission statement

> **MySpace One gives one person, on one offline phone, every everyday tool they need — calculate, write, scan, plan, organize, listen, record, and diagnose — in one fast, private application that never requires an account, a network, or a subscription.**

The mission sentence is testable. "One application" means exactly one icon, one installer, one data store. "Offline phone" means the product must deliver one hundred percent of its value in airplane mode from first launch. "Never" means never — not "unless the user accepts a prompt", not "except for crash reporting".

1.2 The five promises

Every requirement in this document serves one of five promises. When two requirements conflict, the promise ranked lower yields.

1. **P1 — Works everywhere, always.** Full functionality in airplane mode, on a low-end phone, on first launch, with no configuration. If a feature cannot honor this, the feature changes, not the promise.
2. **P2 — Private by architecture.** The app never transmits user content because it *contains no transmit path*: no network permission, no WebView, no telemetry SDK. Privacy is structural, not a policy.
3. **P3 — One home, zero friction.** All tools live behind one icon and at most two taps. No module makes the user re-learn basic patterns; one design system everywhere.
4. **P4 — Fast and gentle.** Cold start under 1.2 s on Tier B hardware; jank-free scrolling; battery-safe background behavior; no notifications except alarms the user created.
5. **P5 — Built to be verified.** Every claim in this document maps to an automated check or a reviewable artifact. The agent proves, it does not assert.

1.3 Non-negotiable constraints

The constraints below are hard. They are repeated or refined in later chapters but never weakened.

ID	Constraint	Rationale	How it is verified
C1	No `INTERNET` and no `ACCESS_NETWORK_STATE` permission in the merged manifest of any shipped APK	P2; user asked for local-only operation	S2 check: `apkanalyzer manifest` on release APK, CI gate in §24
C2	No sign-in, no account, no user identity of any kind	P1, P2	Code review checklist §14.2; no auth classes in dependency scan
C3	No ads, no ad SDKs, no tracking SDKs, no purchase flows	Explicit user instruction; P2	Dependency audit in CI (deny-list of package prefixes, §24.4)

ID	Constraint	Rationale	How it is verified
C4	All user data stays in app-private storage or user-granted SAF locations	P2	Security test suite TC-SEC-* (§20.7)
C5	AI models are side-loaded packs verified by SHA-256; the app never downloads	P1, P2; optional AI on offline devices	Model-pack manager tests TC-AI-pack-* (§16.8)
C6	Android 8.0 (API 26) minimum; phones and tablets; portrait and landscape	Compatibility promise	Emulator matrix (§21.2)
C7	Calculator parity behaviors from the guide (percent, implicit multiplication, digit grouping, selection memory actions) are release blockers	User's core daily tool; guide §C/§D	Catalogue-0 cases TC-ONE-C-001…026 (§20.6)
C8	Max 3 parallel CI runners, ≤ 30 jobs per full pipeline run	Runner quota discipline	Workflow configuration (§24.1)
C9	A calculator-only flavor (`mycalc`) ships from the same codebase via build flavor, zero duplicated engine code	User asked for standalone calculator distribution	Flavor build tasks (§23.4)
C10	Every shipped artifact carries provenance: commit SHA, workflow run ID, APK SHA-256	The owner verifies, not trusts	Release evidence bundle (§25.6)

1.4 Permanently out of scope

The following are rejected for the lifetime of this program, and any task proposing them is invalid: cloud sync of user content; social features; accounts or profiles; push notifications from any external service; in-app purchases; advertising of any kind; third-party content feeds; cryptocurrency or payment features; root access; device administration; accessibility-service abuse (the app *serves* accessibility, it never *consumes* it); background location; contact access.

This list exists because autonomous agents drift toward "helpful" additions. Scope creep is a defect class (§20.9, category S-CREW) and is triaged as such.

1.5 Defaults where this document is silent

1. **Platform convention wins** over invention when they conflict (Material 3 behaviors, back-gesture semantics, share-sheet patterns).
2. **Simplest testable behavior wins** when two designs both satisfy a requirement.
3. **Battery and RAM win** over feature richness at equal user value.
4. **Determinism wins** over cleverness: same input, same output, same logs.
5. When genuinely ambiguous, the agent chooses the option closest to **Google Calculator / Google Keep / Google Files / Google Recorder** interaction patterns for the corresponding module, because the owner's prior feedback anchored on those references — then records the decision D-nnn in the ledger (§15.7).

Chapter 2 — Why combined, why offline: product rationale

2.1 The case for one app

The owner's starting point was a suite of separate tools (a calculator app plus a "MySpace" bundle containing Notes, Scanner, Calendar, Files, Music, Recorder, Diagnostics). Maintaining several APKs multiplies every cost — build, test, release, support — while splitting the user's muscle memory across icons. Combining them produces compounding wins:

- **Cross-module flows become first-class**: scan → text lands in Notes; recording → summary lands in Notes; note → attach → export through Files; calculator result → paste anywhere. Inside one app these are function calls and shared UI; across apps they are fragile intents.
- **One data layer, one search**: a note, a scan's OCR text, a recording's transcript and a calendar event are all queryable from the hub search box (§8.1).
- **One design system, one accessibility story, one performance budget** — enforced once, applied everywhere.
- **One install and update path** for the owner, who distributes APKs directly.

The cost of combining — APK size, module coupling, review complexity — is real and is managed by architecture (§11.2: module boundaries as Gradle modules), the size budget (§23.6), and the flavor system that still yields a calculator-only distribution (C9).

2.2 The case for on-device, always-on offline

Three independent reasons, in priority order:

1. **The user's requirement.** The owner stated the main priority: work locally, on device, no sign-in, no internet. This is a product-definition fact, not a preference to be optimized away.
2. **Trust asymmetry.** An offline-first app has nothing to leak. There is no telemetry pipe to abuse, no terms-of-service to erode, no server breach risk. The privacy story fits in one sentence (P2).
3. **Reliability and latency.** On-device AI in 2026 covers the needed capability set (OCR, ASR, small-LLM summarization, embeddings) with 40–400 ms latencies and zero availability risk. A feature that depends on a network call is a feature that fails in airplane mode, on a metro, on a broken gateway — the exact moments a calculator-and-notes user reaches for the tool.

2.3 Risks of the combined approach and their engineered answers

Risk	Consequence if ignored	Engineered answer
Module coupling turns releases brittle	One bug blocks all releases	Gradle module-per-feature with API surfaces (§11.2); CI runs module-scoped tests in parallel
APK bloat from media + AI code	Slows install; feels unprofessional	Size budget table §23.6; AI *models* are external packs (C5), AI *code* is a few hundred KB
Feature teams (or agent phases) drift in UI patterns	Frankenstein feel, violates P3	Single design-system module; screenshot tests per component (§20.4)
Background media services drain battery	Violates P4	Foreground-service-only policy, strict wakelock tests TC-PERF-* (§20.7)
Hub search over all modules gets slow	P3 erodes	Precomputed FTS + embedding indexes maintained incrementally (§12.5)

2.4 Reference points the owner recognizes

The owner's test guides referenced Google Calculator behaviors (percent handling, digit grouping, history replay) and Google-suite interactions. The product deliberately mirrors interaction *patterns* — not visuals — of: **Google Calculator** (expression caret, live answer, history sheet), **Google Keep** (note cards, checklists, pinning), **Google Files** (browse + clean), **Google Recorder** (markers + transcript), **Proton/Aegis-style local-first security** (encrypted exports). No Google code, assets, or trademarks are used; the parity is behavioral, and Chapter 17 specifies each behavior precisely enough to test without the original.

2.5 Why-combined per module (value table)

Module	Value alone	Value combined (unique to one app)
Calculator	core tool	results into notes; history searchable from hub
Notes	capture	receives scans/transcripts; AI actions share one pack store
Scanner	capture	text flows into notes; analyzer counts scans
Calendar	planning	reminders integrate one notification model; ICS via Files
Files	organization	private store for all modules' outputs; backup covers everything
Music	playback	library via Files' SAF grants; miniplayer across modules
Recorder	capture	transcripts → notes; files visible in Files
Diagnostics	support	one health view for the whole app; pack manager for all AI

2.6 Alternatives considered (and why combined won)

Option	Pros	Cons	Verdict
Keep N separate APKs	small per-APK size; independent releases	N× build/test/release cost; no shared data; owner distributes more files; cross-module flows impossible	rejected
One suite app (this blueprint)	shared design system, one data layer, cross-module flows, one release train	bigger APK; module coupling risk	**chosen** — coupling managed by Gradle modules + budgets
Suite + plugin modules (dynamic feature)	on-demand modules	needs Play (owner sideloads); complexity unjustified offline	rejected
Web wrapper suite	tiny APK	violates P2 structurally (WebView, needs network culture)	rejected forever (§1.4)

Chapter 3 — Users, personas and environments

3.1 Personas

Persona	Device	Data context	What they do daily	What they never forgive
Asha — the owner-distributor	Mid-range Android, often airplane mode	Sideloads APKs for family and friends	Tests each release; shares installs; reads reports	A release that behaves differently from its report
Ravi — the student	2–3 GB RAM phone	No Google account configured on device	Calculator for homework; notes for class; scans of handouts	A calculator that says `0.9999999999` where `1` is expected
Meera — the field worker	Rugged device, offline all day	Works in low-connectivity zones	Scans receipts; records voice memos; plans visits in calendar	Losing a recording or a scanned page
Usha — the archivist	Tablet, large storage	Keeps years of notes and audio	Organizes files, playlists, old notes; exports backups	A silent data migration that renames or drops content

Persona-driven acceptance checks appear in Chapter 9 as journey tests (J-nnn).

3.2 Device tiers and environments

The product must be *excellent* on low-end hardware, not merely functional. Tiers:

Tier	RAM	Typical API	Storage	Product stance
C	2–3 GB	26–30	32 GB shared	Everything usable; AI essentials pack only; 60 fps p50 on calculator, 55 fps p50 lists
B	4–6 GB	29–33	64 GB	Everything fast; + speech pack; recorder waveform real-time
A	≥ 8 GB	31+	128 GB+	+ assistant pack (small LLM); instant hub search over embeddings

Environments the agent must honor: airplane mode; "no accounts on device"; restricted profile with storage permission denied (Scanner falls back to in-app capture only); dark and light system themes; font scale 1.0–2.0; TalkBack enabled; locale `en-IN` (default) plus `en`-generic; battery saver active.

3.3 Accessibility as a persona, not a feature

TalkBack users, low-vision users (font scaling, contrast), motor-impaired users (no gesture-only paths — every gesture has a button equivalent), and deaf users (recorder and music never rely on sound alone) are first-class personas. Chapter 8 specifies the accessibility contract per module and Chapter 20 tests it (TC-A11Y-*).

3.4 Locale expectations

Default locale `en-IN`: Indian digit grouping (`1,23,456`) in Calculator and Files sizes, DD-MMM-YYYY dates, 24-hour clock respected from system settings. The string layer is externalized from day one so additional locales cost translations, not code (§14.5). Number formatting is *presentation-only*: stored values are exact strings/rationales (§12.2), never locale-mangled floats.

3.5 Environment acceptance matrix

Each row is an environment the QA matrix must exercise at least once per gate from G3 onward (TC-ENV-*). "Expectation" is the acceptance bar in that environment.

Environment	Emulator/device profile	Expectation
Airplane mode, cold	E33B, radios off before launch	100% functionality; no network-vocabulary strings; J-05 green
No accounts configured	E33B, factory profile, no Google sign-in	No account prompts anywhere; all modules work
Fresh install, denied all permissions	E26C	Hub + Calculator + Notes fully usable; Scanner import path; Recorder explains need; no repeated prompts
Battery saver ON	E33B	Effects → None; AI deferral chips; no job started > budget without user tap
Font scale 2.0	E33B display size max	No clipped text; calculator keys 1.3× cap with 48 dp targets
Dark theme + dynamic color	E35A (A12+)	Tokens re-resolve; no white flash; screenshots match goldens
TalkBack on	E33B	Core journeys completable; focus order correct
Locale en-US	E26C	Western grouping; period decimals; history renders identically post-restore
Locale de-DE	E26C	Comma decimals accepted on input; grouping localized; laws L9 green
Low storage (< 100 MB)	E26C 8 GB image filled	Banner + graceful refusals; no corruption (J-10)
Doze / deep idle	E33B forced idle	Reminder fires within policy window; no wakelock abuse counters
Process death storm	E33B `am kill` loop	J-06 restoration green
Rotation storm	E33B	No state loss mid-draft/record/capture
Tablet 900×1600	E35A tablet skin	Two-pane layouts per §7.12; no stretched phones UI
Very old WebView-less image	E26C (no WebView present)	Nothing breaks (app never uses WebView — structural)

3.6 Persona journey narratives (design anchors)

Asha's release day. Asha builds/downloads the release on her laptop, copies the APK and the report PDF to her phone, installs over the previous version, and runs the Diagnostics self-test. It passes in seconds. She opens Calculator, tries her acceptance list (the Catalogue-0 spot checks), shares the report with two friends, and sideloads via a file transfer — all without internet on the receiving phones. Her acceptance bar: the report's claims match what her thumb just did.

Ravi's homework sprint. Ravi opens Calculator in airplane mode during a commute: 20 minutes of mixed arithmetic, percent discounts, a bracket-heavy physics expression on the scientific pad, replaying two earlier results from history, storing his roll number via MAdd for reuse. He switches to Notes between problems, checklisting formulas. Latencies he notices: none. His acceptance bar: `5+ =` → `10`, `1÷3×3` → `1`, zero IME popups, history intact after the phone's aggressive battery manager killed the app yesterday.

Meera's field day. Meera scans eight receipts at a market (storage-tight phone), transcribes one voice note at lunch on her Tier B device (2 min clip, ~90 s), sends both into Notes as one trip record, backs up at night via encrypted `.msone` to her laptop. Her acceptance bar: OCR usable in sunlight (high-contrast B&W filter), no silent failures, backup restore verified on a second phone.

3.7 Persona → priority influence map

Persona	Strongest priorities	Features most shaped
Asha (owner)	provenance, honest QA, flavor	C10, F-084, L5 evidence style, mycalc
Ravi (student)	calc parity, speed, offline	F-001…F-035 core, P4 budgets
Meera (field)	scanner/recorder resilience, storage	F-055…F-059, F-073…F-077, J-10
Usha (archivist)	backup, migrations, files	F-067, §29.2, Files edge register
A11y personas	§6.6 everywhere	TC-A11Y-*, UX-012

This part fixes the product's identity before a single line of code is specified. It is deliberately short and absolute: missions, promises, constraints and people. Everything else in the blueprint is a consequence of what is written here.

PART II

EXPERIENCE DESIGN

The product as a person will meet it: principles, navigation, the shared design system, and the complete module-by-module interaction specification. Wireframes here are normative for structure and behavior; visual styling is normative only through the tokens of Chapter 6.

Design is where combined apps usually fail: eight tools duct-taped into one binary feel like eight apps in a trench coat. This part prevents that by defining one set of principles, one navigation model, one design system, and then specifying every screen so that no module invents its own patterns.

Chapter 4 — Experience principles

4.1 The eight principles

#	Principle	Meaning in practice	Testable consequence
X1	**Two taps to anything**	From cold start, every module screen is reachable in ≤ 2 taps	Nav audit test TC-NAV-001
X2	**Thumb-first**	Primary actions live in the bottom third; keypads and FABs are ≥ 48 dp targets	A11y + touch-target checks TC-A11Y-002
X3	**One visual language**	Color, type, elevation and motion come only from tokens (§6.2); no module-local palettes	Screenshot diff + lint rule against hardcoded colors
X4	**Content over chrome**	Bars and borders shrink; content (expression, note text, waveform) is the screen	Design review checklist §18.4
X5	**Forgiving everywhere**	Undo before confirm; selection before action; destructive actions are always reversible or trash-ed for 30 days	TC-UX-undo-* cases
X6	**Offline is invisible**	No offline banners, no retry buttons, no sync spinners — there is nothing to be online *to*	Copy audit: strings catalog contains no network vocabulary
X7	**AI is an accelerant, never a gate**	Every AI affordance has a manual equivalent; AI absence changes speed, not capability	TC-AI-fallback-* cases
X8	**Respect the system**	Theme, font scale, reduced-motion, locale and ringer follow OS settings instantly	TC-SYS-* cases

4.2 Emotional design goals

The app should feel like a well-made pocket tool: quiet, precise, dependable. Concretely: animations run 150–250 ms with standard easing and are disabled under system reduced-motion; haptics are subtle and configurable (a "Haptics: On" toggle in calculator settings exists because the owner's reference devices expose it); empty states teach with one sentence and one action, never marketing copy; error messages say what happened, what it means for the data (never "your data may be lost" unless true), and one recovery step.

4.3 What the experience must never do

- Never show a spinner on a local operation that completes in < 100 ms; skeleton loaders are reserved for large indexes (first Files scan, first library import).
- Never interrupt with a dialog except for: record-confirmation privacy notice (first mic use), uninstall-risk warning on encrypted backup, and genuine irreversibility (e.g., emptying trash).

- Never ask for a permission at launch; permissions are requested at the moment of first use, with a one-line reason (§14.7).
- Never autoplay sound (music resumes only by explicit play; recorder previews are muted until tapped).

4.4 Principle conflict resolution (worked cases)

1. *X1 (two taps) vs X8 (respect system)*: predictive back may add a system animation — allowed; it does not add taps.
2. *X5 (forgiving) vs storage cost*: undo/trash retained 30 days then purged — forgiveness is time-boxed, disclosed at delete.
3. *X7 (AI optional) vs convenience*: semantic search silently downgrades to FTS — convenience loses; capability parity wins.
4. *X3 (one language) vs calculator themes*: themes are token sets, not bespoke styling — theming stays inside the system.
5. *P4 (fast) vs features*: effects default None on Tier C — features stay, cost defaults to zero.

Chapter 5 — Information architecture and navigation

5.1 The hub model

MySpace One is a **hub** with eight module spokes plus Settings. The hub is a real screen with its own value (search, continue-where-you-left, pinned items), not a launcher of icons. The calculator-only flavor (`mycalc`) replaces the hub with the calculator as home and hides all other spokes — same codebase, flavor-selected navigation graph (§23.4).

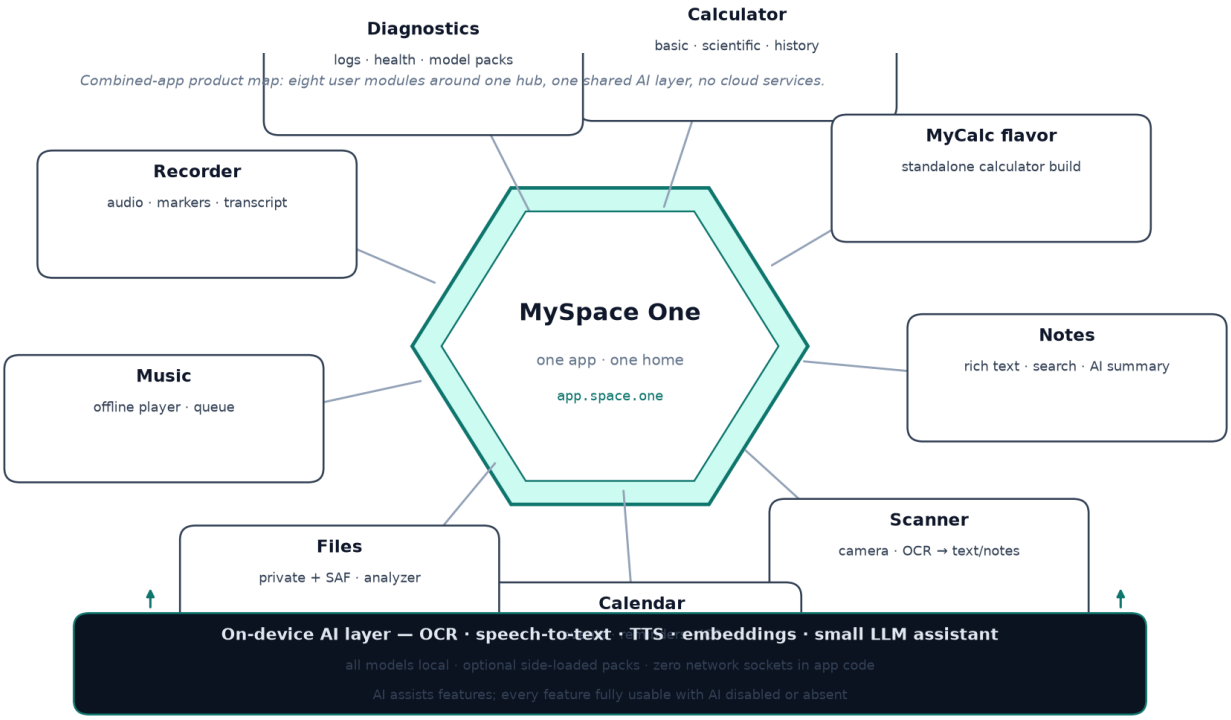


Figure 1. *Combined-app product map: eight modules around one hub, one shared on-device AI layer.*

5.2 Navigation graph

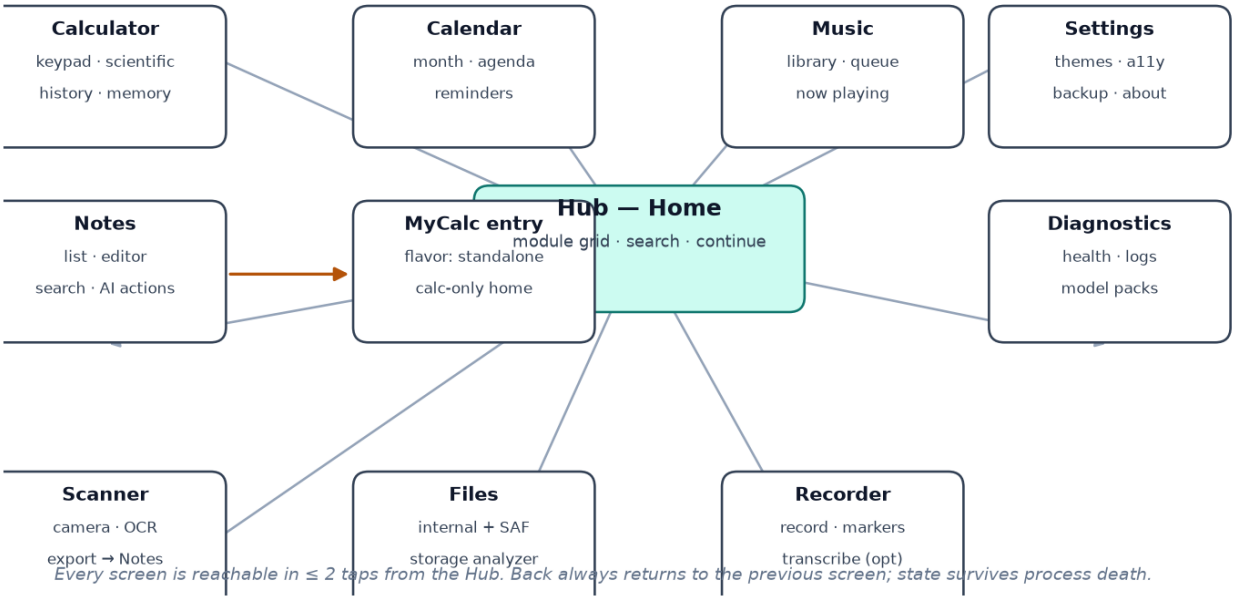


Figure 2. *Normative navigation graph: hub, modules, settings, and the MyCalc flavor entry.*

Navigation rules:

1. Back always pops the back stack; the system back gesture and the in-app back are equivalent. The hub is the root; back from hub backgrounds the app (standard behavior).
2. State survives process death: every screen's scroll position, selection, draft note text and calculator expression are saved via `SavedStateHandle` + DataStore within 500 ms of change (§12.6).
3. Deep links are internal only (`myspace://note/42`) for in-app navigation and shortcuts; there is no `https://` app-link surface because there is no web.
4. Long-press on a hub module icon offers a shortcut to that module's primary action (new note, new recording, scan, open calculator) — implemented as app shortcuts, which work offline.

5.3 Hub screen specification

The hub contains, top to bottom:

1. **Search field** — one query across everything: notes (FTS + semantic when the pack is installed), scans (OCR text), events, files, songs, recordings, transcripts, and calculator history. Results are grouped by module with module-accent icons; opening a result deep-links into the module screen. (The field lives on the hub only — rule X1 keeps module reachability through the grid, not duplicated search bars.)
2. **Continue row**: the last-open module with its state thumbnail ("Draft — Field survey notes", "Recording 00:41 paused").
3. **Module grid**: eight cells, fixed order (Calculator, Notes, Scanner, Calendar, Files, Music, Recorder, Diagnostics), each with name + one-line status ("2 events today", "62% free", "3 new scans").
4. **Pinned strip**: user-pinned notes/events/playlists as horizontal cards.

5. Bottom: **Settings** gear (also in overflow) and a **New** quick-fab whose target follows the last-used module.

Empty-first-run: the hub shows a single card "Everything works offline. Start with Calculator, or import music and files from your storage." plus a one-time (dismissable) privacy card: "MySpace One has no internet permission. Nothing you do here leaves this phone."

5.4 Cross-module object flows

These flows are specified as first-class user journeys (J-nnn, §9) and tested end-to-end:

- Scanner → Notes (OCR text becomes note body or attachment).
- Recorder → Notes (transcript/summary appended as a linked note).
- Calculator → anywhere (copy result; share sheet via system, works offline).
- Calendar ↔ Files (ICS import/export).
- Notes ↔ Files (attach, export, backup).
- Music ↔ Files (folder import, playlist export M3U).
- Diagnostics → Files (log bundle export, user-initiated only).

5.5 Hub search behavior detail

- **Debounce & budgets**: 120 ms keystroke debounce; per-source budget 25 ms to first page; total first paint ≤ 120 ms on 10k-note corpus (budget §23.6); results stream in as sources complete (fixed module order preserved).
- **Matching**: FTS with prefix matching on the last token (`calc` matches `calculator`); FTS query sanitized (quotes escaped, operators stripped); file/song/artist matches are substring-insensitive casefold; event matches on title+note; recordings match on filename + transcript when present; calculator history matches expression prefix and result string.
- **Ranking**: within a module — exact > prefix > substring > fuzzy (FTS rank for notes); across modules the fixed order (Notes, Events, Files, Scans, Music, Recordings, Calculator) with per-module result caps (5, expandable "See all in Notes").
- **Zero-state**: recent searches (10, DataStore, clearable) and per-module quick links; no network suggestions ever.
- **Semantic (pack)**: cosine top-50 merged per §12.5; tagged "related" visually distinct from exact matches; disabled silently at Tier C battery saver with a provenance row in Diagnostics (§16.7).
- **A11y**: results are an announceable list; opening a result announces the module context first.

5.6 Hub region inventory (normative rows)

#	Region	Content	Behavior
R1	Status bar blend	system	edge-to-edge, scrim-free (surface color)
R2	Greeting row	"Good morning" + date; tap = nothing (decorative but labeled)	changes by clock; announces date via a11y
R3	Search field	hint + mic (pack)	§5.5 behaviors; focus ring token
R4	Continue row	last module card: icon, name, state line	tap resumes exactly; long-press clears entry
R5	Module grid	8 cells, 2×4 phone / 1×8+ tablet	cell = icon, name, status line; ripple token
R6	Pinned strip	horizontal cards (notes/events/playlists)	hidden when nothing pinned
R7	New FAB	target = last-used module's create action	long-press = action menu across modules
R8	Bottom bar	none (design choice: no bottom nav — grid is home)	—
R9	Overflow	Settings, About	top-right kebab
R10	First-run layer	privacy + start cards	dismiss persists (F-046)

5.7 Ranking worked examples (fixtures mirrored in TC-HUB-006)

Query	Corpus state	Expected order (top 5)
`rent`	note "Rent reminder 27th", event "Rent", file `rent-agreement.pdf`, song "Rent song"	event (exact title, sooner date), note (title exact), file (name), song (exact but Music rank lower), —
`1,23,456`	history has `3+123456 = 1,23,459` and note with `123456`	calculator history card first (exact result match), then note
`invoice`	scan OCR text contains "INVOICE #118"; file `invoice-118.png`	scan (OCR match), file
`laptop advice`	note "Purchase guidance for computing" (semantic pack on)	FTS empty → semantic hit tagged "related"
`rec`	recording "REC-2026-10-27", note "recording ideas"	note (prefix title), recording (filename)

5.8 Continue-row state line catalogue

Last module	State line format	Example
Calculator	`Expression "3+4×2" · caret 5`	resumes exactly incl. selection
Notes	`Draft — <title or "Untitled">`	cursor position restored
Scanner	`<n> pages in review`	review reopened
Calendar	`Viewing <month>` / `Editing <event>`	screen + selection
Files	`<folder> · <selection?>`	path + selection
Music	`<track> · <position>`	miniplayer state
Recorder	`Recording 00:12` / `Review <name>`	FGS state honest
Diagnostics/Settings	last subpage	—

Chapter 6 — Design system

6.1 System overview

The design system is a Gradle module `core/designsystem` containing tokens, theme, typography, components, motion specs, and icon set. Modules depend on it and on nothing visual else. Every reusable component ships with Roborazzi screenshot tests (§20.4) and an accessibility smoke test.

6.2 Design tokens

Tokens are Kotlin constants generated from a single source table (checked in as `designsystem/tokens.csv`), so documentation, code and tests share one truth. Core palette (light / dark):

Token	Light	Dark	Used for
`color/background`	`#FAFBFC`	`#0B1220`	Screen background
`color/surface`	`#FFFFFF`	`#111C2E`	Cards, sheets
`color/onSurface`	`#0F172A`	`#E4EAF2`	Primary text
`color/onSurfaceMuted`	`#5B6B7E`	`#94A3B8`	Secondary text
`color/primary`	`#0F766E`	`#2DD4BF`	Bars, FAB, selection, live accents
`color/onPrimary`	`#FFFFFF`	`#062F2C`	Text on primary
`color/liveAnswer`	`#B45309`	`#FBBF24`	Calculator live answer (LED-amber in dark)
`color/equalsKey`	`#FDE68A`	`#FBBF24`	The `=` key
`color/danger`	`#BE123C`	`#FDA4AF`	Delete confirm, destructive
`color/caret`	`#0F172A`	`#FFFFFF`	Expression caret (exact 2 dp bar)
`shape/keyRadius`	16 dp	16 dp	Keypad keys, key shape options override
`elevation/sheet`	8 dp	8 dp	History/memory/effects sheets
`motion/standard`	200 ms	200 ms	Enter/exit, 150 ms for taps
`type/display`	34/40 `SemiBold`	same	Calculator result
`type/body`	15/22 `Regular`	same	Note bodies
`space/unit`	4 dp grid	—	All spacing multiples

Calculator-specific theming goes beyond global tokens: **Key theme** (eight options — `MySpace settings` (follows app theme), `Graphite`, `Ocean`, `Sunset`, `Mint`, `Rose`, `Lemon`, `Iris`) and **Key shape** (`Circle`, `Squircle`, `Pill`, `Rounded square`, `No shape`) are user settings (§17.7); switching re-resolves colors immediately with a 200 ms crossfade (guide behavior H4). A theme change must produce a measurable pixel difference on the keypad region (the owner's acceptance harness used a ≥ 10% region-diff threshold; we adopt the same, TC-ONE-C-023).

6.3 Typography and iconography

Font: **Roboto Flex** (variable) bundled in the APK; monospace contexts (expression field, file paths, logs) use **JetBrains Mono**. Sizes scale with `fontScale` up to 2.0; keypads are exempt from scaling beyond 1.3 (keys would overflow) but compensate with larger touch targets. Icons: Material Symbols Outlined, subset by R8; module accent glyphs are simple, two-tone; the launcher icon is a teal honeycomb hex with a subtle calculator glyph (cover art motif), provided in all densities + adaptive-icon layers.

6.4 Components inventory

The system ships these shared components (each with screenshot tests):

- `AppTopBar` (title, overflow, optional bottom-progress), `BottomSheet` (drag-handle, snap points, back-gesture aware), `Fab`, `EmptyState(icon, title, hint, action)`, `ErrorState(retry-less: message + action)`, `SearchField`, `ResultRow`, `KeyPad` + `CalcKey` (theme/shape aware), `ExpressionField` (caret + live answer, §7.2), `SelectionPill` (Cut/Copy/MAdd/MUse/Paste), `Waveform` (recorder), `MiniPlayer`, `CalendarMonthGrid`, `FileRow`, `StorageDonut`, `ConfirmSheet` (for destructive), `AiActionRow` (ornamented, always secondary-styled), `Toast` replacement (in-app snackbar with action, never system toast from background threads — a past lesson from the owner's harness: toasts crashing background instrumentation threads, §15.8 note).

6.5 Motion

Standard: enter = fade+8 dp slide up 200 ms; exit = fade 150 ms; sheet = slide from bottom 250 ms with 0.8 overshoot damping; keypad press = scale 0.97 on down, 120 ms; theme change = crossfade 200 ms; calculator result change = none (instant; results never animate — the number is the UI). All motion respects `Settings.Global.ANIMATOR_DURATION_SCALE` and removes entirely under reduced-motion.

Interaction	Spec
Screen enter	fade + 8 dp slide-up, 200 ms, standard easing
Screen exit	fade 150 ms
Sheet enter/exit	250 ms slide, overshoot damping 0.8
Key press	scale 0.97 on down; release 120 ms
Selection pill	150 ms scale-in at press point, shadow settle
Live answer change	none (instant)
Result on `=`	instant swap + effects (optional)
Theme change	200 ms crossfade, no layout shift
FAB extend/retract	200 ms width morph
List reorder (queue/memory)	180 ms settle, lift shadow 8 dp
All motion	killed under reduced-motion or animator scale 0

6.6 Accessibility contract (system-wide)

1. Every interactive element ≥ 48×48 dp; keypad keys ≥ 44×44 dp visual with 48 dp touch bounds.
2. Content descriptions on all icons; decorative images excluded from the a11y tree.
3. TalkBack announces calculator keys as their action ("7", "plus", "equals"); the expression field announces full expression + caret context ("3 plus 4, caret after 4, live answer 7").
4. Contrast: text ≥ 4.5:1, large text ≥ 3:1, key glyphs ≥ 3:1 against their key fill — verified per key theme in a token unit test (no screenshot needed).
5. Touch exploration: no gesture-only features; every gesture has a visible control twin (e.g., history swipe-down ↔ history button).
6. Focus order is logical (top-to-bottom, left-to-right); dialogs trap focus and restore on dismiss.

6.7 Type scale and spacing tokens (complete)

Token	Value	Used for
`type/displayMono`	30/36 JetBrainsMono SemiBold	Calculator expression
`type/display`	34/40 SemiBold	Calculator result/history result
`type/titleL`	22/28 SemiBold	Screen titles
`type/titleM`	18/24 SemiBold	Sheet titles, card titles
`type/titleS`	15/20 Medium	List row titles
`type/body`	15/22 Regular	Note body, prose
`type/bodyS`	13/18 Regular	Secondary rows, chips
`type/caption`	11/14 Regular	Timestamps, provenance footnotes
`type/key`	22/24 Medium (caps at 1.3× scale)	Keypad glyphs
`space/xs`	4 dp	Icon-to-label, chip padding
`space/s`	8 dp	intra-group
`space/m`	12 dp	list gutters
`space/l`	16 dp	screen margins
`space/xl`	24 dp	section separation
`space/sheet`	20 dp	sheet horizontal padding
`radius/s`	8 dp	chips, small cards
`radius/m`	16 dp	cards, keys
`radius/l`	24 dp	sheets
`elevation/1..3`	1/4/8 dp	card/bar/sheet

Every token exists in code as a named constant; a unit test asserts the Kotlin token table matches `tokens.csv` (TC-SYS-tokens), and screenshots reference tokens only — a hardcoded `Color(0x…)` in feature modules is a detekt error (§18.1).

6.8 Component behavior specifications (normative digest)

Component	Props/variants	States	Rules
`AppTopBar`	title, nav icon, actions[], large bool	default/scrolled (elevation 4 dp)	title truncates with ellipsis at 1 line; overflow = max 6 items + "More"
`BottomSheet`	snapPoints, dismissable	expanding/settled/dragging	drag handle 32×4 dp; back gesture dismisses; content scrollable; scrim 40%
`Fab`	icon, text?	normal/extended	56 dp (extended 56 h, min 120 w); never overlaps content scroll edge without inset handling
`EmptyState`	icon, title, hint, action	—	icon muted 40%; action is the only colored element
`ErrorState`	title, detail, action	—	no retry if retry cannot succeed (policy errors → action = "Open Diagnostics")
`SearchField`	hint, query, onQuery	empty/filled/active	clear ("×") appears when non-empty; mic entry only when Speech pack installed
`KeyPad`	pad(basic/sci), theme, shape	—	keys 48 dp min touch; haptic hook; press scale 0.97/120 ms; equals emphasized
`ExpressionField`	expr, caret, liveAnswer, selection	empty/text/selection/caret-menu	caret 2 dp × line-height; selection highlight token; pill positioned above field when room else below; autoscroll keeps caret visible
`SelectionPill`	actions[]	shown/dismissed	shadow 8 dp; never covers selection; announced as menu with item count
`Waveform`	amplitudes, playing	idle/recording/paused	20 fps max; paused freezes + dims 40%
`MiniPlayer`	track, playing, progress	collapsed/expanded	64 dp; tap opens player; swipe-down hides for session
`ConfirmSheet`	title, detail, confirmLabel, danger	—	destructive confirm is danger-colored and never the initial focus
`AiActionRow`	actions[]	ready/running/disabled(missing pack)	running shows cancel; disabled shows reason inline
`StorageDonut`	segments[]	—	max 6 segments; center label = free %; a11y = textual summary
`CalendarMonthGrid`	month, selected, dots	—	7-col grid; week starts per locale; today token-filled circle

6.9 Key-theme palettes (normative values)

Each theme defines key fill / key glyph / operator-key fill / function-key fill for light and dark; `MySpace settings` follows the app theme using core tokens. Contrast pairs are validated programmatically (TC-A11Y-contrast-01). Values are sRGB hex.

Theme	Mode	Key fill	Glyph	Operator key	Function key
Graphite	light	`#F1F3F5`	`#212529`	`#E3E6E8`	`#E9EBED`
Graphite	dark	`#2B2F33`	`#E8EAED`	`#343A3F`	`#303539`
Ocean	light	`#E7F0FA`	`#0B3D66`	`#D2E4F5`	`#DCEAF7`

Theme	Mode	Key fill	Glyph	Operator key	Function key
Ocean	dark	`#16324A`	`#D6E9FA`	`#1B3D5A`	`#193855`
Sunset	light	`#FDEEE2`	`#7A2E0C`	`#F9DFC8`	`#FBE6D6`
Sunset	dark	`#4A2413`	`#FFE3CC`	`#542A15`	`#4F2713`
Mint	light	`#E6F6EF`	`#0B5137`	`#D2EFE1`	`#DCF3E8`
Mint	dark	`#14402E`	`#D8F3E6`	`#194A36`	`#174532`
Rose	light	`#FDE9EE`	`#831843`	`#F9D4DD`	`#FBDDE5`
Rose	dark	`#4C1D33`	`#FBD5E0`	`#57223A`	`#521F36`
Lemon	light	`#FBF3D9`	`#6B5509`	`#F6E9BC`	`#F9EECB`
Lemon	dark	`#423A12`	`#FBF0C4`	`#4A4116`	`#453D14`
Iris	light	`#EFEAFB`	`#3D2B7D`	`#E2D9F7`	`#E8E1F9`
Iris	dark	`#2E2354`	`#E6DEF9`	`#352A60`	`#312658`

`=` key always renders `equalsKey` amber token in every theme (recognizability over theming); theme change applies without recreating the pad (Compose recomposition only, crossfade §6.5).

6.10 Key shapes (normative geometry)

Shape	Corner radius	Notes
Circle	height/2 (fully round)	glyph centered; touch bounds stay rectangular
Squircle	40% superellipse	default-adjacent smooth
Pill	100 dp x-radius	wide operator keys look capsule
Rounded square	16 dp	default
No shape	0 dp	sharp; grid lines 1 dp token border for definition

6.11 Illustration and empty-state art style

Empty states use one 2 dp-stroke line illustration (48–64 dp) drawn with tokens only: the motif is the honeycomb hex — calculator shows a hex with `1+2`, notes a hex with a folded corner, scanner hex corners, calendar a hex ring, files hex stack, music a hex note, recorder a hex wave, diagnostics a hex pulse. No mascots, no gradients, no stock art — the icon set stays inside the vector budget (≤ 40 KB total). Illustrations ship in the design-system module with their own goldens.

Design is where combined apps usually fail: eight tools duct-taped into one binary feel like eight apps in a trench coat. This part prevents that by defining one set of principles, one navigation model, one design system, and then specifying every screen so that no module invents its own patterns.

6.12 UI design grammar (how any screen is composed)

Every screen in the product is assembled from the same grammar. An executor who has internalized these six rules can build any screen in this document — or correctly refuse a screen that breaks them.

The five-layer anatomy (bottom → top):

Layer	Contents	Rules
L1 Background	`color/background` (or pure-black variant)	never holds content directly; one layer only

Layer	Contents	Rules
L2 Surface	cards, sheets, bars: `color/surface` + `elevation/*` + `radius/*`	surfaces group; never nest surfaces more than 2 deep
L3 Content	text, media, waveforms, grids: `type/*` + `color/onSurface*`	content is the screen (X4); chrome shrinks around it
L4 Controls	buttons, keys, FAB, selection pill, toolbars	≥ 48 dp touch; only `color/primary`/`danger` accents
L5 Overlays	sheets, dialogs, snackbars, MiniPlayer, AI previews	one blocking overlay at a time; back always dismisses

The slot template (every screen answers five slots):

1. **Bar slot** — `AppTopBar`: title = screen name (or context object), nav = back/home, actions ≤ 3 + overflow. Calculator hides the bar into the field area by design (§7.2); that is the single sanctioned exception.
2. **Primary canvas slot** — the module's reason to exist (expression field, note blocks, month grid, waveform). Gets the largest area and the first 200 ms of attention.
3. **Action slot** — the primary action lives in the bottom third (FAB, keypad, transport controls, Done). One primary per screen; secondaries are text buttons or overflow.
4. **Status slot** — inline, per-card or per-row feedback (saving chip, OCR progress, marker count). Never a global modal spinner (§4.3).
5. **System slot** — insets, gesture areas, IME detection (to *move content*, never to enable typing in Calculator — F-033).

Selection map (job → component): list of items → `ResultRow` in `LazyColumn`; single choice → `BottomSheet` with radio rows; confirm → `ConfirmSheet`; text entry → inline field (no dialog input); progress > 300 ms → per-card indicator; empty → `EmptyState`; error → `ErrorState`; anything else → do not invent (propose via ledger, §15.7).

Rhythm and spacing: all vertical rhythm is a multiple of `space/unit` (4 dp): row height 56 = 8+40+8; card padding 16; section gap 24; screen margin 16. If a layout needs 10 dp, the layout is wrong — drop to 8 or up to 12.

Elevation ladder: list card 1 dp → bar 4 dp → dragged item 8 dp → sheet 8 dp → dialog/FAB 8–12 dp. Two surfaces at the same elevation must never overlap; that is a design bug, not a shadow problem.

Worked example — Notes list in grammar terms: L1 background; L2 = search surface chip + note cards; L3 = card text (`titleS` title + `bodyS` snippet + `caption` date, checklist progress chip); L4 = new-note FAB (action slot) + card overflow icons; L5 = none at rest, delete snackbar on event; slots: bar "Notes", canvas = pinned+others lists, action = FAB, status = autosave chip inside editor only.

6.13 Figma workflow — design linking, usage and the design↔code loop

Figma is the **design mirror** of this blueprint; the repository is the law. This section defines the exact linking so any designer, reviewer or the AI agent can move between frame and code without translation loss.

Figma mirrors the repository; the repository is the law. The loop closes when exported frames and screenshot goldens agree.

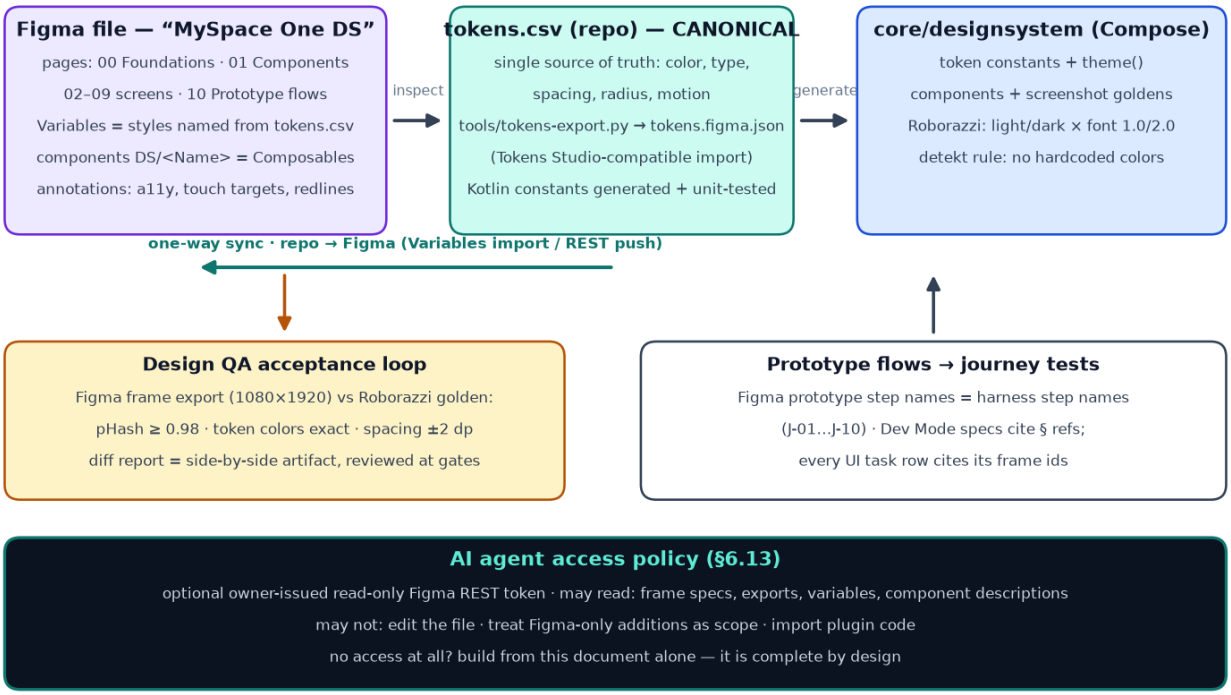


Figure 3. *Figma workflow: one-way sync from repo tokens, component mapping, design-QA acceptance loop, and the agent access policy.*

File structure (`MySpace One DS` — file key recorded in `design/figma/README.md`):

Page	Contents	Linked to
`00 Foundations`	color styles, type styles, effect styles, grid styles, Variables sets (Light/Dark)	`tokens.csv` (§6.2, §6.7)
`01 Components`	`DS/*` component library with variants and auto-layout	`core/designsystem` composables (§6.4, §6.8)
`02 Hub` … `09 Diagnostics`	one page per module: all screens, all states (empty/content/error/loading), light+dark	Chapters 7, §6.14 inventory
`10 Prototype flows`	J-01…J-10 journey prototypes (step names = harness step names)	§9.12, Appendix R
`11 Archive`	deprecated frames (never deleted — the owner's preservation standard)	ledger

Naming and mapping contract:

Figma object	Name pattern	Code counterpart
Variable / style	`color/primary`, `type/body`, `space/l`, `radius/m`, `motion/standard`	token constants (§6.2, §6.7)
Component	`DS/AppTopBar`, `DS/CalcKey`, `DS/SelectionPill`, …	`AppTopBar`, `CalcKey`, `SelectionPill`, … (§6.4)
Variants	`theme=light/dark`, `state=empty/content/error`, `shape=circle/pill/…`	sealed state/enum params
Frame (screen)	`<MODULE>-<nnn> <Screen> <State>` e.g. `CALC-003 Calculator Selection`	screen + state id (§6.14 registry)
Prototype flow	`J-01 Scan to note`	journey harness id (§9.12)

Figma object	Name pattern	Code counterpart
Annotation	a11y plugin labels: role, focus order, announcement	Compose semantics (§6.6, §6.17)

Token pipeline (one-way: repo → Figma). `tokens.csv` is canonical (TC-SYS-tokens). `tools/tokens-export.py` emits Tokens-Studio-compatible `tokens.figma.json`; the designer imports it into Figma Variables (or the owner's token pushes it via REST). Figma-only color tweaks are invalid — they ride a tokens PR or do not happen. This keeps the detekt no-hardcoded-color rule and the screenshot goldens meaningful.

Using Figma to build (the agent's or a developer's read path):

1. Open the screen's frame from the §6.14 registry (or Dev Mode search by name).
2. Dev Mode spec panel shows: token names for every fill/text (never raw hex in review), spacing multiples, auto-layout values → these are the §6.15 redlines.
3. Component descriptions in `01 Components` carry the code anchor (`core/designsystem/…/AppTopBar.kt`) and the binding § refs.
4. Prototype flows demonstrate intended motion (compare §6.5 durations); Figma smart-animate values are illustrative, the motion tokens are normative.
5. Any behavior not visible in a frame lives in this document (§ chapters) — frames show *look*, the document rules *behavior*; conflicts resolve to the document and a ledger entry.

Design-QA acceptance loop (UI acceptance against Figma, §6.18 operationalizes it): per screen, export the frame at 1080×1920 and compare to the Roborazzi golden: perceptual hash ≥ 0.98, token-derived colors exact, key spacing within ±2 dp. Diffs produce a side-by-side artifact reviewed at gates. The loop closes design↔code drift mechanically instead of by eye.

AI agent access policy. If the owner issues a read-only Figma REST token, the agent may read frame specs, exports, variables and component descriptions to sharpen implementation fidelity; it may not edit the file, treat Figma-only additions as scope (§1.4), or import plugin output as code. If no token exists (the default), the agent builds from this document alone — it is complete by design, and every visual fact needed is in §6.14/§6.15 and the wireframes.

Version discipline. Design changes land as PRs to `tokens.csv`/components + a Figma re-sync in the same merge (docs-drift gate covers `design/figma/README.md` staleness). Figma branch/merge flows mirror the repo: a design PR = a frame branch; merged = moved out of Archive only via PR.

6.14 Screen inventory with Figma frame registry (master UI table)

This table is the single registry of every screen: route, wireframe/section, states, and the Figma frame ids that mirror it. It is mirrored as `docs/blueprint/ui-screens.csv` (generated-checked like `qa/testcases.csv`). The build checklist columns serve G4/G6 review (this table supersedes the pointer-former Appendix M).

#	Screen	Route	Figma frames (page)	States to build	Spec
1	Hub Home	`myspace://hub`	HUB-001..004 (02)	first-run, content, search-zero, recovered	§5.3, §5.6
2	Hub search results	`hub/search`	HUB-010..012 (02)	streaming, grouped, semantic-tagged	§5.5, §5.7
3	Calculator main (basic)	`calc`	CALC-001..006 (03)	empty, typing, live-answer, selection+pill, caret-menu, result	§7.2, §7.2.1–.3

#	Screen	Route	Figma frames (page)	States to build	Spec
4	Calculator scientific	`calc?pad=sci`	CALC-010..013 (03)	INV on/off, DEG/RAD	§7.2.3
5	History sheet	sheet	CALC-020..023 (03)	empty, list, pinned, replay-paint	§7.2.4, §7.2.9
6	Memory sheet	sheet	CALC-030..032 (03)	empty, slots, reorder	§7.2.5, §7.2.9
7	Calculator settings sheet	sheet	CALC-040..042 (03)	defaults, theme/shape pickers	§7.2.6
8	Effects picker	sheet	CALC-045..046 (03)	4 styles + preview	§7.2.5
9	MyCalc home	flavor root	MCALC-001..006 (03)	same as CALC subset, zero module UI	§7.2.7
10	Notes list	`notes`	NOTE-001..005 (04)	empty, pinned, grid view, trash-entry	§7.3
11	Notes editor	`notes/{id}`	NOTE-010..016 (04)	typing, checklist, AI row (ready/running/missing), autosave chip	§7.3, §16.3
12	Notes trash	`notes/trash`	NOTE-020..021 (04)	items, empty	§7.3, F-050
13	Scanner viewfinder	`scan`	SCAN-001..004 (05)	guides, batch chip, denied-import card	§7.4
14	Scanner review	`scan/review`	SCAN-010..014 (05)	pages, crop, filters, dup warning	§7.4, §13.11
15	Scanner OCR card	in review	SCAN-020..023 (05)	running, editable, low-conf, pack-missing	§7.4, F-057/59
16	Scanner export sheet	sheet	SCAN-030..031 (05)	targets + destinations	§7.4
17	Calendar month+agenda	`cal`	CAL-001..004 (06)	today, dots, agenda, empty day	§7.5
18	Event editor	`cal/new` `cal/{id}`	CAL-010..015 (06)	fields, validation, repeat sheet + preview, test-alarm	§7.5
19	Files browser	`files`	FILE-001..005 (07)	roots, rows, select-mode, saf-revoked	§7.6
20	Files analyzer	`files/analyzer`	FILE-010..012 (07)	donut, category list, clean flow	§7.6
21	Files trash	`files/trash`	FILE-020..021 (07)	items, empty	F-068
22	Music library	`music`	MUSIC-001..005 (08)	tabs, import sheet, empty	§7.7
23	Music player	`music/player`	MUSIC-010..014 (08)	artwork, speed/sleep chips, queue sheet	§7.7
24	MiniPlayer	overlay	MUSIC-020..021 (08)	collapsed, hidden	§6.8
25	Recorder record	`rec`	REC-001..004 (09)	idle, recording, paused, call-interrupted	§7.8, §13.10
26	Recorder saved list	`rec/list`	REC-010..012 (09)	empty, cards, badges	§7.8
27	Recorder recording view	`rec/{id}`	REC-020..025 (09)	waveform seek, markers, transcript, transcribe states	§7.8
28	Diagnostics health	`diag`	DIAG-001..004 (10)	cards, self-test running/done/failed	§7.9

#	Screen	Route	Figma frames (page)	States to build	Spec
29	Model packs	`diag/packs`	DIAG-010..013 (10)	installed, missing, importing, rejected	§7.9, §16.8
30	Logs + share preview	`diag/logs`	DIAG-020..022 (10)	ring list, preview verbatim	§7.9, §28.3
31	Settings root + sections	`settings/*`	SET-001..008 (10)	registry-driven rows, about provenance, licenses	§7.10
32	App lock	overlay	SET-020..021 (10)	prompt, fallback	F-082
33	Confirm sheets (global)	overlay	GLOB-001..003 (01)	destructive pattern	§6.8
34	Snackbar/undo (global)	overlay	GLOB-010..011 (01)	undo pattern	UX-014
35	Empty states (global)	per module	GLOB-020..028 (01)	8 module empties + illustration	§6.11
36	Tablet two-pane set	—	TAB-001..010 (11)	per §7.16 rows	§7.16
37	Landscape set	—	TAB-020..026 (11)	calculator, recorder, calendar	§7.16
38	Font-scale 2.0 set	—	A11Y-001..010 (11)	core screens at 2.0	UX-011

6.15 Redline specifications (numbers an implementer can build from)

Global redlines (apply everywhere; per-screen deltas only where §7 states otherwise):

Element	Redline
Screen margin	16 dp left/right; 8 dp top under the bar
Top bar height	56 dp; large-title variant 96 dp collapsing to 56 on scroll
List row (single line)	56 dp: icon 24 dp at x16, text x56, chevron/trail at right-16
List row (two lines)	72 dp: title `titleS` at +12, subtitle `bodyS` at +34
Card padding	16 dp; inter-card gap 8 dp; section header gap 24 dp top / 8 dp bottom
FAB	56 dp, margin 16 dp, above insets + 16 dp
Keypad key	height 52 dp basic / 44 dp scientific rows; gap 6 dp; side padding 12 dp; touch bounds ≥ 48 dp
Expression field	text `displayMono` 30 dp, left 20 dp; caret 2 dp wide, line-height tall; live answer `type/titleM` right-aligned 20 dp
Sheet	radius 24 top, handle 32×4 at +8, content padding 20 dp, max height 90%
Snackbar	above insets + 8 dp, full-width minus 16 margins, 4.5 s default
Selection pill	height 40 dp, radius 20, shadow 8 dp, offset 8 dp above selection bbox (below if clipped)
Touch targets	≥ 48×48 dp everywhere; icon buttons 40 dp visual + 8 dp invisible bounds
Focus ring (keyboard)	2 dp `color/primary`, offset 2 dp, radius follows target

Screen-specific redlines live as Figma Dev Mode specs per §6.14 frames and repeat in the frame annotations; where this table and a frame disagree, the frame's numbers win only if they are multiples of the 4 dp grid — otherwise the table wins (ledger entry on the frame).

6.16 Interaction specification index (control-level motion + haptics)

Control	Trigger	Motion (token)	Haptic	State after
`CalcKey`	press down/up	scale 0.97, 120 ms	tick (if on)	inserted; live answer updates
`=` key	press	none on result (instant) + effects if set	confirm tick	history row written
`AC`	press	none	tick	selection or expression cleared
Selection pill	long-press entry	scale-in 150 ms at touch point	double-tick	selection mode
Selection resize	slide	live highlight; pill follows x	none	selection bounds
History replay	card tap	sheet closes after paint (≥1 frame)	none	expression painted, caret end
Sheet	open/close	250 ms slide, damping 0.8; back dismisses	none	previous focus restored
Checkbox	tap	150 ms fill	tick	progress chip updates
List reorder	drag	lift 8 dp, settle 180 ms	tick on drop	positions persisted
Swipe-to-delete	swipe	row collapse 200 ms + undo snackbar 4.5 s	none	trash (undoable)
FAB	press	200 ms extend (extended variant)	none	create flow opens
MiniPlayer	track change	crossfade title 200 ms	none	progress resets
Record	press	FGS ≤ 200 ms; timer starts on first sample	long-tick	notification visible
Flag marker	press	marker dot drops onto waveform (150 ms)	tick	marker row
Theme change	pick	200 ms crossfade; no layout shift	none	persisted
Pull-to-refresh	—	not used anywhere (nothing to refresh — X6)	—	—

6.17 Accessibility annotations (Figma labels → Compose semantics)

Figma a11y annotations (plugin format) ride every frame: `role`, `label`, `focus-order`, `announcement`. The code contract mirrors them 1:1:

Frame annotation	Compose implementation	Test
`role: button`	`Modifier.semantics { role = Role.Button }`	TC-A11Y scan
`label`	`contentDescription` or `semantics { contentDescription }` (keys announce actions: "plus", "equals")	TC-A11Y-calc-01
`focus-order: n`	explicit `semantics { traversalIndex = n }` on custom layouts	focus-order test
`announcement` (live)	`liveRegion = LiveRegion.Polite` on live answer, saving chip, OCR progress	UX-006/025
decorative	`clearAndSetSemantics { }` (excluded from tree)	tree scan
heading	`semantics { heading() }` on section titles	TalkBack rotor

Focus-order defaults per core screen: Hub = search → continue → grid (row-major) → pinned → overflow; Calculator = expression (readonly announce) → history button → pad row-major (= last); Notes editor = title → toolbar (grouped) → blocks → AI row → overflow. Dialogs trap focus and restore on dismiss (§6.6).

6.18 Design-QA gate (per-screen UI acceptance)

A screen is UI-accepted at G4/G6 when every row below has an artifact:

#	Check	Bar	Artifact
DQ-1	Grammar compliance (§6.12)	slots/layers correct; no invented components	review note
DQ-2	Tokens only	detekt rule clean; screenshot colors == token expectations	CI rule + goldens
DQ-3	Figma fidelity	frame export vs golden: pHash ≥ 0.98; spacing ±2 dp	side-by-side diff
DQ-4	States complete	empty/content/error/loading (+ module-specific) all built and shot	screenshot set
DQ-5	Redlines	§6.15 numbers hold (bounds scan)	bounds JSON
DQ-6	Motion	§6.16 table honored; reduced-motion kills all	recording/frames
DQ-7	A11y	labels/roles/focus per §6.17; contrast token test green	a11y audit output
DQ-8	Responsive	tablet/landscape rows built per §7.16 without state loss	screenshots
DQ-9	Font scale	2.0 no-clip sweep	clipping detector JSON
DQ-10	Dark + themes	dark goldens; calculator themes × shapes matrix	screenshot matrix

The gate reviewer is the agent (self-review, §18.4 UX checklist) with the owner spot-checking the side-by-side set. A failed DQ row is a defect with a screenshot pair — never a "known difference".

6.19 Where to find anything about UI (the everything-index)

UI aspect	Normative home
Principles & emotional goals	§4.1–4.3
Navigation, hub regions, search UI	§5.1–5.8
Color/type/spacing/shapes/elevation tokens	§6.2, §6.7
Component inventory + behavior specs	§6.4, §6.8
Motion (system + control-level)	§6.5, §6.11, §6.16
Icons & illustrations	§6.3, §6.11
Key themes & shapes (calculator theming)	§6.9, §6.10
Screen-by-screen behavior	§7.1–7.10
States, errors, edge cases	§7.11, §7.14
Responsive/tablet/landscape	§7.16
First-run education lines	§7.17
Feel budgets (latency per interaction)	§7.18
Wireframes (19 figures)	figures index (Appendix BB)
Screen registry + Figma frames	§6.14 (+ `ui-screens.csv`)
Redlines	§6.15
Design grammar	§6.12
Figma workflow & linking	§6.13
Accessibility annotations	§6.6, §6.17
Microcopy strings	Appendix E
Gestures & control twins	Appendix H
UX acceptance criteria (all levels)	Chapter 8 (§8 table + §8.1–8.3)
UI test layers (goldens, a11y, journeys)	§20.2–20.4, Chapter 8
Design-QA gate	§6.18

Nothing about the UI is decided anywhere else: if a pixel, dp, millisecond, label or frame id is not traceable to one of the rows above, it does not exist.

Chapter 7 — Module-by-module interaction specification

This chapter specifies what each screen *does*; Chapter 9 turns every behavior into numbered, testable requirements, and Chapter 20 into test cases. Structure rules from Chapter 5 apply everywhere (two taps, one design system, states per §7.11).

7.1 Hub (Home)

Specified in §5.3. The hub's acceptance-relevant behaviors: search across all modules with grouped results; continue-row restores exact module state; module status lines are computed locally (events today, storage free, new scans count, library size); long-press shortcuts; first-run privacy card shown exactly once. The hub is also the crash-recovery surface: after an abnormal process death the continue-row shows "Recovered — tap to reopen" for the interrupted module.

7.2 Calculator (the flagship module)

The calculator is the owner's primary tool and the most exactly specified module in this document. Structure: one screen, two pads (basic / scientific), an expression field with a **caret**, a **live answer** line, a **history sheet**, a **memory sheet**, and a settings sheet (key theme, key shape, haptics, "show as fraction"). Scientific pad appears by swipe-left on the pad area or an `INV`/sci toggle on tablets and landscape.

7.2.1 Expression field

- Single line that **autoscrolls horizontally** when longer than the field; caret is always visible as a 2 dp vertical bar; tapping inside the text places the caret between characters (not word-level); long-press enters text-selection mode.
- **Live answer** beneath the expression: evaluated as-you-type with the same engine that produces final results (no separate "fast path" that could disagree — §17.1.6). Rendered in `color/liveAnswer`; suppressed when the expression is not a complete evaluatable form (then nothing is shown — never a stale answer).
- Bracket balancing is auto-displayed: an unclosed `(2+3` shows live answer for `(2+3)` and the `=` key closes all open brackets (guide C6). The `( )` key inserts the pair when at a position where an operand is expected, else inserts only `)` when a bracket is open and an operator was just typed (§17.1.4).
- Operators replace, not append, when typed twice (`5+×` → `5×`); a leading `–` is unary minus, so `10×–` is legal (guide C4) and evaluates to `-10`.
- **Percent** is context-sensitive (§17.1.5): `200+10%` displays live `220`; `50%` alone displays `0.5`; `200×10%` displays `20`.

7.2.2 Selection and the memory pills (guide section B)

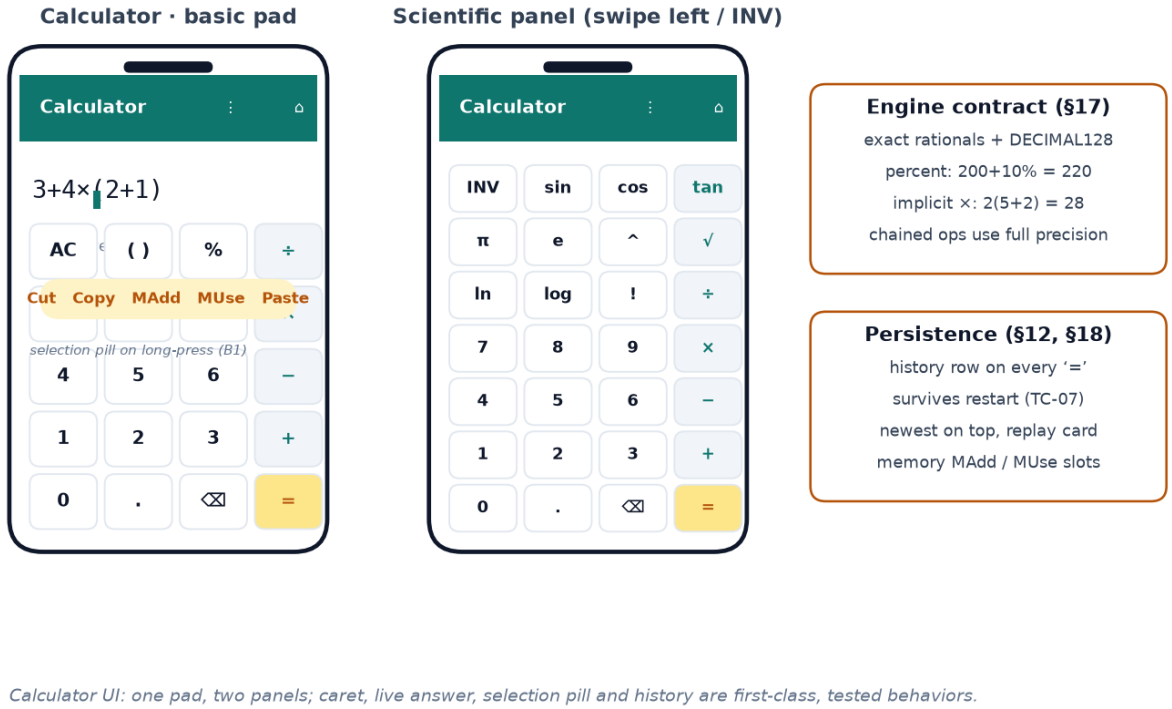
Long-press (≈ 300 ms, stationary) on the expression field opens selection mode: the touched token highlights (pink/primary highlight), and a floating **selection pill** appears: `Cut · Copy · MAdd · MUse · Paste`. Behaviors:

- **B2 — hold-and-slide**: without lifting, sliding the finger left/right grows/shrinks the selection character by character. This is a custom `longPress + drag` detector (pointer-ID tracked, slop-disabled); *not* platform `selectionHandle` UI.

- **B4 — Copy/Paste**: copy puts the exact selected substring on the clipboard (no grouping commas); paste inserts clipboard text at the caret (sanitized to legal characters, §17.1.3).
- **B5 — Cut**: removes selection to clipboard; typing immediately after inserts at the now-empty position.
- **B6 — tap elsewhere**: a tap on another part of the expression (or on empty field) exits selection mode, dismisses the pill, and leaves the caret at the tapped position.
- **B7 — AC**: pressing `AC` clears selection first; a second press clears the expression. (Single-press semantics: if a selection exists, AC clears just the selection and the expression remains.)
- **B11 — caret menu**: long-press on *empty* expression (or beyond text end) opens the same pill reduced to `MUse · Paste` — insertion at an empty caret.
- **B12 — MAdd / MUse**: `MAdd` stores the numeric value of the selection (parsed with the engine's evaluator; `1,234` and `2+3` both store `5`/`1234` appropriately) into the single memory slot shown in the memory sheet; `MUse` **replaces the current selection** with the stored value, or inserts at caret when no selection. Typed digits may also land *inside* a selection, replacing it (guide B5 pattern).
- Selection state never survives the expression being cleared, a history replay, or an `=` press.

7.2.3 Keypad

Basic pad: `AC ( ) % ÷ / 7 8 9 × / 4 5 6 − / 1 2 3 + / 0 . ⌫ =`. Scientific pad adds rows above (`INV sin cos tan / π e ^ √ / ln log !`) and replaces the layout per the wireframe; `INV` reveals `sin⁻¹ cos⁻¹ tan⁻¹` and `eˣ 10ˣ`. Key behaviors: press haptic (if enabled) + 120 ms scale; `⌫` deletes one character at the caret (or the whole selection); press-and-hold `⌫` repeats after 400 ms at 15 Hz; Long-press on `AC` does nothing — destructive shortcuts are deliberate omissions, and history clearing lives only in the history sheet behind a confirmation.



Calculator UI: one pad, two panels; caret, live answer, selection pill and history are first-class, tested behaviors.

Figure 4. *Calculator basic pad and scientific panel with engine and persistence contracts annotated.*

7.2.4 History sheet

- Opens by: the history button (persistent, top-right of expression area), swiping down on the expression field, or the overflow menu. Title `History · n` with `Clear` and close actions.
- **Newest on top**; each card shows expression (small, muted) and result (large); timestamps shown as `27th Oct, 13:36` style. Tap a card → **replay**: the expression paints into the expression field first, the sheet closes after paint, caret at end (guide D9/D10 — replay is debounced 350 ms against double-tap races).
- History persists in Room (§12.2) and **survives restart** (TC-ONE-C-013); pinned cards survive `Clear` (clear asks confirm; pins are kept, unpinned are deleted).
- Caps at 500 rows, LRU-evicted, `VACUUM`-free (Room handles page reuse).

7.2.5 Memory sheet and effects

- Overflow → `Memory` opens a sheet listing saved values (the MAdd slot and any `M+`-accumulated slots, cap 10) with `MUse / MAdd / MC` per row and up/down reordering (guide E3 "Move up/down from place n").
- Overflow → `Effects` opens the effects picker with at least four styles (`None`, `Bubbles`, `Ripple`, `Confetti`) that animate on `=`; selection persists; `None` is default on Tier C devices if battery saver is on.

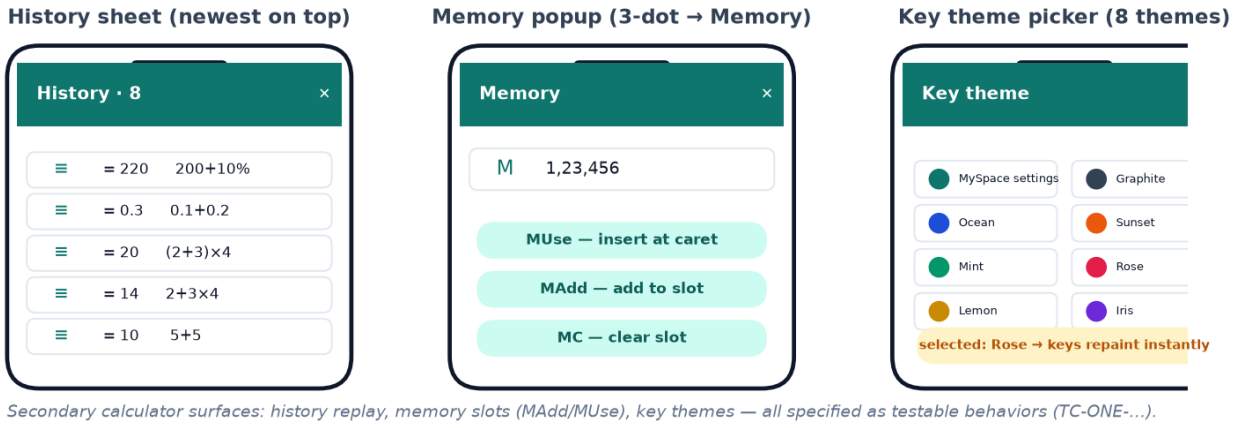


Figure 5. *History, memory and key-theme surfaces.*

7.2.6 Calculator settings sheet

Items (exact overflow list): `Haptics: On/Off`, `Memory`, `Show as fraction On/Off` (display `1/3` instead of `0.3333333333` where the engine keeps a rational), `Key theme`, `Key shape`, `Effects`. These six items are the complete overflow menu — the owner's harness asserted this exact list on the standalone flavor.

7.2.7 MyCalc flavor differences

The `mycalc` flavor renders the calculator as home with a compact app bar (`MyCalc` title, STANDALONE tag in diagnostics), the same overflow list, and **no** hub, modules, or cross-module actions (no "→ Notes" anywhere). Package `app.space.calc`, `versionCode` shared scheme (§23.4). Zero permissions is the flavor goal: even `CAMERA`-class permissions are compiled out because Scanner is absent from the flavor's merged manifest.

7.2.8 Key-by-key specification (normative)

Every key: id, gesture behaviors, inserted token, TalkBack label, and notes. "Press" = tap; long-press behaviors listed where they exist.

Key	Press inserts/does	Long-press	TalkBack label	Notes
AC	clear selection, else clear expression	none	"All clear"	two-stage semantics F-017
( )	() pair or) per parser state	none	"Bracket"	§17.1.4 smart insert
%	% postfix	none	"Percent"	semantics §17.1.5
÷ × − +	operator (replace-on-operator)	none	"Divided by/Times/ Minus/Plus"	replacement F-005
7 8 9 4 5 6 1 2 3 0	digit at caret	none	digit name	—
.	decimal point (guard: one per number)	none	"Point"	input normalization §17.1.1
⌫	delete one char left of caret / selection	repeat 400 ms then 15 Hz	"Delete"	delete-stops-at-start; no wraparound
=	evaluate; history write; auto-balance	none	"Equals"	trailing-operator rule F-004
INV	toggle inverse/secondary sci keys	none	"Inverse"	toggle persists per session only
sin cos tan	sin(…	none	function name	auto-open bracket
π e	constant	none	"Pi"/"Euler's number"	—
^	^	none	"Power"	right-assoc
√	√(	none	"Square root"	—
ln log	ln(/ log(	none	"Natural log"/"Log base 10"	—
!	! postfix	none	"Factorial"	integer domain
$\sin^{-1}$ $\cos^{-1}$ $\tan^{-1}$ (INV)	asin(…	none	"Inverse sine" etc.	DEG/RAD aware
DEG/RAD toggle	switch angle mode	none	"Angle mode degrees/radians"	persisted setting
history button	open history sheet	none	"History, n entries"	persistent control (E1 parity)

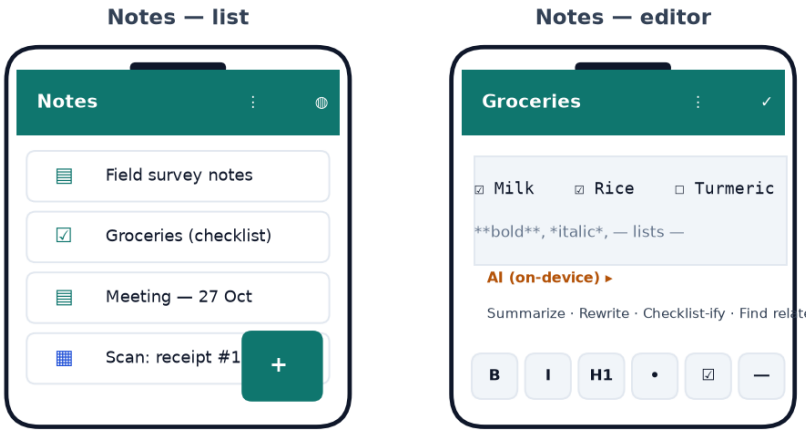
Edge-case register for the calculator (TC-CALC-edge*):

Input / situation	Required behavior
00.5	renders 0.5 (leading zero normalization, value exact)
1.2.3	second dot ignored (input guard), expression remains valid
5+%	syntax error card with caret hint (percent needs left operand)
((2+3)	auto-balance → 25
() empty pair	syntax error card; AC recovers
2+-3	2−3 after replacement rule → -1
2--3	unary fold → 2−(−3) = 5
10!	3628800
170!	error card (overflow, reason shown)
π^2	inexact path, renders 9.869604401
sin(90) (DEG)	1 exactly (degree identity reduction)
cos(0)	1 exactly
tan(90) (DEG)	domain error card
expression of 512 chars	input blocked at limit with counter chip

Input / situation	Required behavior
rapid 30 keystrokes/s (stress)	no dropped chars, no frame > 24 ms (TC-PERF-key)
= with empty expression	no-op, no history row
history 500 rows + new solve	oldest unpinned evicted, pinned kept
MUse with empty memory	action disabled with tooltip "Memory is empty"
theme change mid-selection	selection preserved, pill repositioned
process death mid-expression	expression+caret+selection restored (F-043 family)

7.3 Notes

- **List**: cards (title + snippet + date + pin + checklist progress); sections: Pinned / Others; search field on top; sort menu (date-edited default, date-created, title); view toggle list/grid; trash entry point.
- **Editor**: title field, body with lightweight rich text (bold, italic, H1/H2, bullet list, checkbox list, divider), toolbar (B I H1 • ☑ —), undo/redo, pin toggle, background color (6 palette options, token-defined), attach from Files, and the **AI row** (§16.3): `Summarize · Rewrite · Checklist-ify · Find related` — each optional, each preview-with-undo, each disabled with a visible reason when its pack is missing ("Install the Essentials pack in Diagnostics").
- Checklists: tap checkbox toggles; progress `3/7` shows on the list card.
- Note storage: a single `note` table + FTS mirror (§12.2); bodies are a structured JSON document (§17.3) — never HTML strings — so search, export (Markdown) and import are lossless.
- Autosave 500 ms after typing stops; drafts survive process death; trash keeps 30 days with restore.



Notes: plain-file-friendly rich text, full-text + semantic search, optional on-device AI actions that never modify a note without preview/undo.

Figure 6. *Notes list and editor with the on-device AI action row.*

7.4 Scanner

- **Viewfinder** (CameraX): guide-frame corners, tap-to-focus, torch toggle, shutter, batch counter chip ("3 pages"), import-from-Files button (image/PDF import path works when camera permission is denied — the module is still usable).
- **Capture pipeline**: auto-crop via document-corner detection → optional filter (`Original`, `Gray`, `B&W`) → multi-page document.

- **OCR card** per page: recognized text editable; actions `Copy / → Notes / Re-OCR / Delete`; language initially `en` (Latin); the OCR model is the Essentials pack (§16.5) — without it, capture+export still work and the card shows "OCR needs the Essentials pack".
- Export: single page → image/PDF into Files; document → PDF with page order editor; text → new note or clipboard. Images are stored app-private; "Clean scans" in Files analyzer can purge them by size.

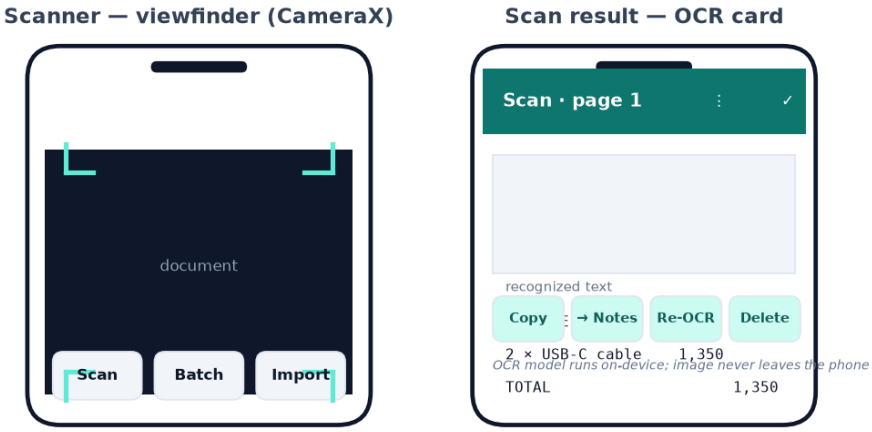


Figure 7. *Scanner viewfinder and OCR result card.*

7.5 Calendar

- **Month grid + agenda below** (the wireframe layout): today circled, event dots, agenda rows for the visible day(s); swipe between months; `Today` chip when scrolled away; list alternative view.
- **Event editor**: title, all-day toggle, start/end date-time pickers (local time), **Repeat** menu (None, Daily, Weekly on <day>, Monthly on day <n>, Yearly, Custom — Custom exposes the supported RRULE subset §17.4), Alert (None/At start/5/15/30/60 min before/1 day before), Note field, calendar color (personal palette).
- Reminders fire via exact alarms (`SCHEDULE_EXACT_ALARM` with `USE_EXACT_ALARM`-policy compliance — see §14.4: the app targets a category allowed to use inexact fallback gracefully if the OS revokes exact); full-screen intent permission NOT used (a heads-up notification with sound + vibration is enough; avoids the special-access permission).
- ICS: import multiple events from a `.ics` in Files; export single event or a date range (respects the supported RRULE subset both ways; unsupported RRULEs import as one-off with a note).
- No accounts, no invites, no attendees — a personal calendar.

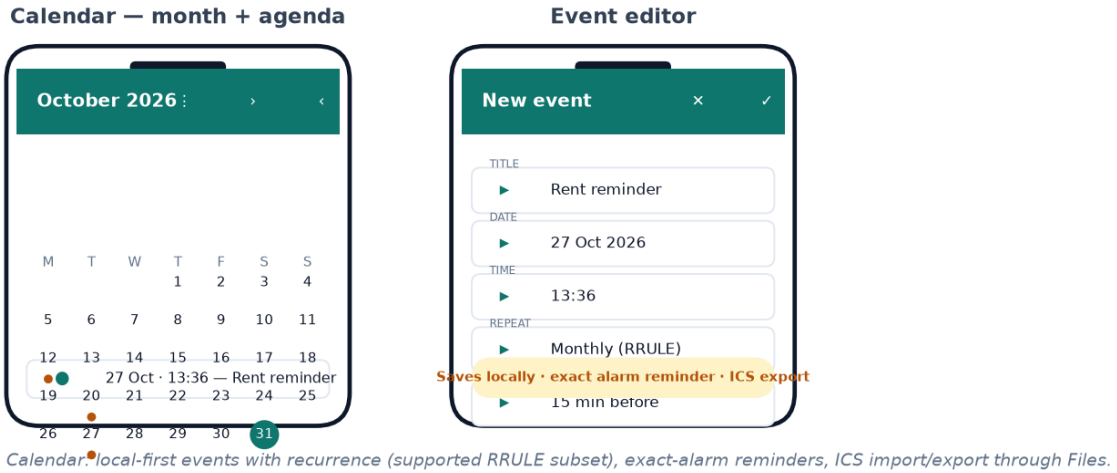


Figure 8. *Calendar month+agenda and the event editor.*

7.6 Files

- **Browser** over: (a) app-private storage (`Files`, `Scans`, `Recordings`, `Backups`, `Exports`), (b) user-added SAF roots (persistent URI grants), (c) the media folders the user granted via `READ_MEDIA_*` for music import (read-only).
- Operations: open (hand-off to system viewers via `ACTION_VIEW` with `FLAG_GRANT_READ` — works offline), rename, move (private+SAF), copy, delete (to app trash for private files), new folder, sort (name/date/size/type), multi-select with actions, search (name + content for text formats).
- **Storage analyzer**: donut by category (Scans, Audio, Notes+DB, Exports, Other) computed locally, with `Clean scans` (deleteable-by-age) and `Export report` (JSON into Exports).
- Encrypted export: `Backups` produces a `.msone` archive (zip of JSON+files, AES-256-GCM with a key from the user's passphrase via PBKDF2, §14.3); import verifies and restores.

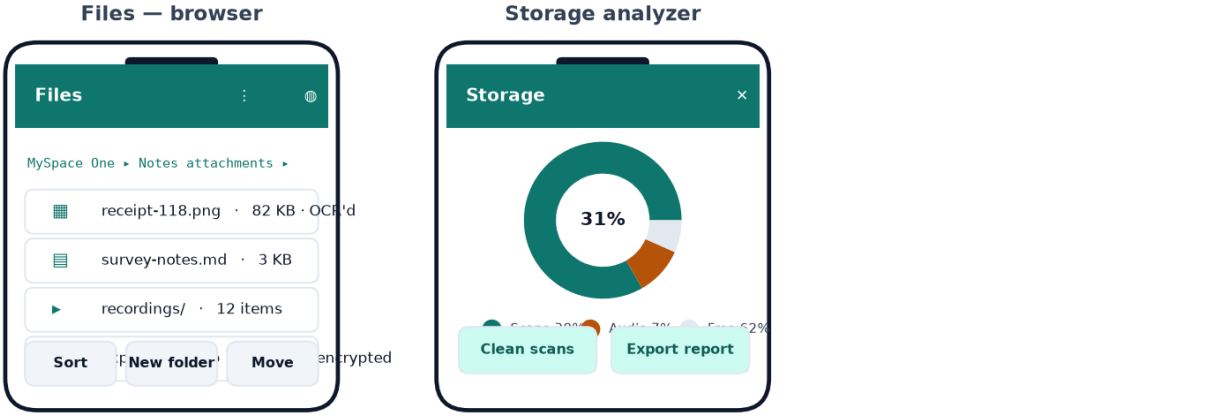
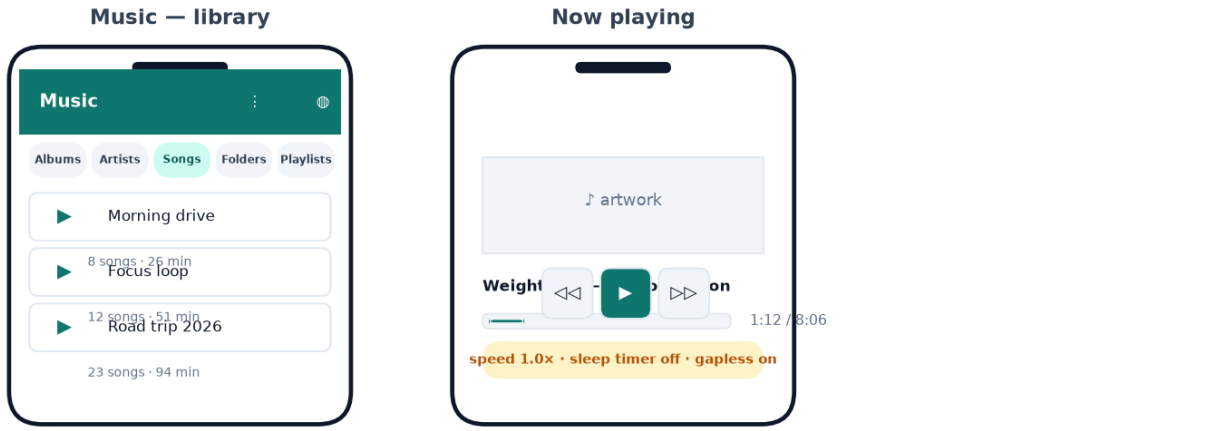


Figure 9. *Files browser and storage analyzer.*

7.7 Music

- **Library** tabs: Songs, Albums, Artists, Folders, Playlists. Import: on first open, offer folder pick (SAF) or MediaStore scan of the granted audio; library is a Room mirror with artwork thumbnails cached.

- **Player**: artwork, title/artist, seek bar, prev/play/next, shuffle/repeat (off/all/one), speed (0.5–3.0×), sleep timer (5/15/30/60 min/end-of-queue), queue view with drag-reorder.
- Playback via Media3; a compact **MiniPlayer** persists over module screens; headset (media button) and Bluetooth controls work; audio focus respected (duck on navigation prompts); gapless for consecutive tracks; no lyrics fetching (offline).
- Playlist export: M3U (relative paths where possible) into Files.

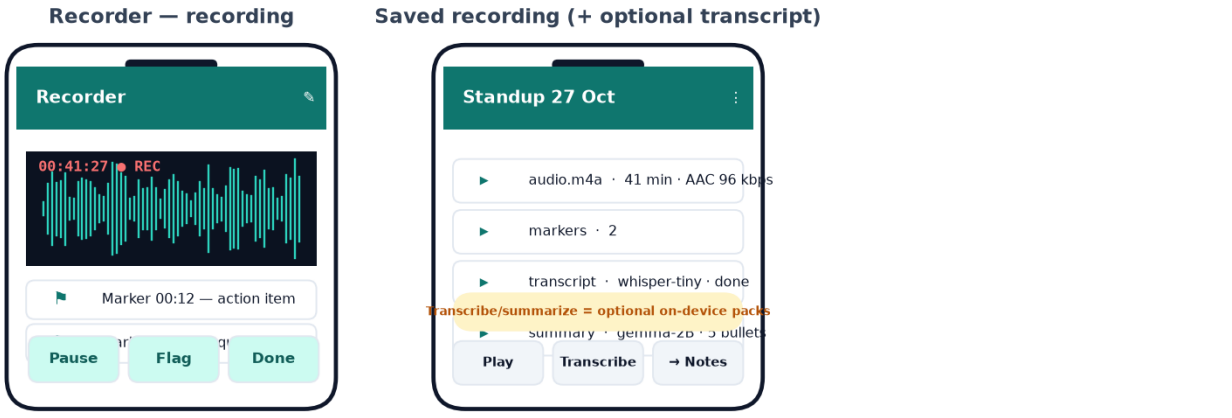


Music: MediaStore/Folder import, playlists, queue, playback speed, sleep timer; Media3 session integrates with headset controls — all offline.

Figure 10. *Music library and now-playing surfaces.*

7.8 Recorder

- **Record screen**: big timer, live waveform, `Pause/Resume`, `Flag` (drop a timestamped marker, with optional 20-char label), `Done`. Recording is a **foreground service** with a persistent notification (pause/resume actions); mic permission requested at first record with a one-line privacy line ("Audio never leaves this phone").
- **Saved recording screen**: file card (duration, codec, size), markers list (tap to seek), `Transcribe` and `Summarize` buttons (pack-gated, §16.5), rename, delete (trash), `→ Notes` (creates linked note with transcript/summary).
- Formats: AAC-LC 96 kbps default (storage-friendly), WAV optional setting; recordings land in app-private `Recordings/` and are visible/copyable through Files.
- Transcripts are stored with segment timestamps; searchable from hub search when present.

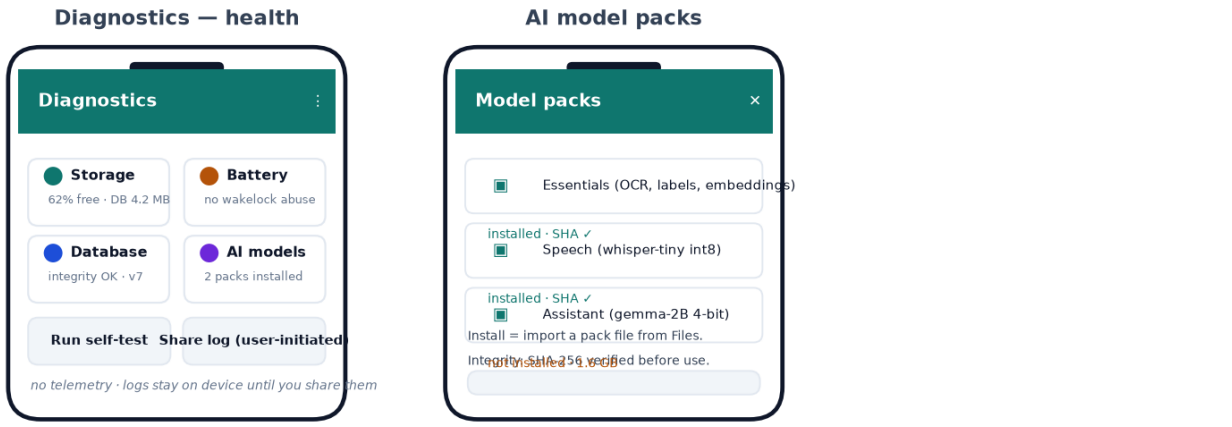


Recorder: foreground-service capture with pause/resume and markers; transcription and summarization run only if the user installed the AI packs.

Figure 11. *Recorder capture and saved-recording surfaces.*

7.9 Diagnostics

- **Health cards**: storage (free %, DB size), battery (no wakelock abuse statement computed from actual counters), database integrity (`PRAGMA integrity_check` result, schema version), AI packs status.
- **Self-test**: runs engine property-suite smoke (100 cases), DB round-trip, model-pack hash verification, notification channel, alarm-can-schedule check; outputs a local result card.
- **Logs**: rolling in-app log ring (7 days, 2 MB cap, PII-scrubbed per §28.3); `Share log` exports via system share sheet (user-initiated only).
- **Model packs manager**: list packs (Essentials, Speech, Assistant) with status (`installed SHA ✓`, `not installed · size`), install = import pack file from Files, verify SHA-256 against the pack manifest, enable/disable, delete. No download path exists in the app (C5).



Diagnostics doubles as the model-pack manager: import, verify, enable/disable AI packs; self-test exercises engines and reports results locally.

Figure 12. *Diagnostics health and model-pack manager.*

7.10 Settings (global)

Sections: **Appearance** (theme Light/Dark/System, dynamic color on A12+, key theme/shape shortcuts), **Calculator** (haptics, fraction display, effects), **Privacy** (app lock with device credential, encrypted backup passphrase change, trash-empty-now), **Reminders** (default alert, sound per channel), **Storage** (link to Files analyzer), **AI packs** (link to Diagnostics manager), **About** (version, build provenance block — commit SHA, CI run link text, APK SHA-256 — the same provenance the owner's reports carry), **Licenses** (OSS licenses screen).

App lock (optional, off by default): locks on background>2 min; uses `BiometricPrompt` with device-credential fallback; zero biometric data stored by the app.

7.11 States, errors, and edge cases (module-spanning)

State	Rule
Empty	One icon + one sentence + one action. Calculator history empty: "Solved calculations appear here."
Loading	Only for > 100 ms local work: first Files scan, first music import, OCR runs (progress on card), LLM runs (streaming preview).
Error	Local errors only; message + one action ("Try again" only when retry can succeed; otherwise "Open Diagnostics").
Permission denied	Feature degrades per §7.11 (this table); never dead-ends.
Storage full	Global banner once: "Storage is full — recordings and scans will fail. Clean up in Files." Writes fail soft with per-operation messages.
Battery saver	Effects default to None; AI jobs above size threshold deferred with a visible chip.
Process death	All drafts/expressions/selections restored (§12.6).
Very long content	Expression 512 chars max (enforced, counter at 500+); notes unlimited but list virtualizes; recordings capped by storage check (refuse start when < 50 MB free).

7.12 Responsive behavior (phones, tablets, landscape)

- ≥ 600 dp width: hub grid becomes 4-column; Calculator becomes two-pane (history side panel instead of sheet; keypad centered max-width 480 dp); Notes becomes list+editor two-pane; Calendar month+agenda side-by-side; Music player expands artwork with queue side panel.
- Landscape phones: Calculator keeps keypad full-width with scientific pad as horizontal scroll row; Recorder keeps controls reachable (bottom).
- All screens support font scale 2.0 without clipping (scroll instead of truncate); none require landscape-only.

7.13 What the UX deliberately does not include

No onboarding carousel (a first-run card is enough); no ratings/feedback prompts; no "what's new" popups (a quiet About → changelog list instead); no notification badges beyond reminder alarms; no sounds other than playback and alarms; no emoji in UI copy.

7.14 Module edge-case registers (normative)

Notes (TC-NOTE-edge*):

Situation	Required behavior
Note with 10 000 blocks	editor virtualizes; save < 300 ms; no ANR
Paste of 1 MB text	import as one block with cap notice at 100k chars
Concurrent edit impossible (single user) — but process death mid-keystroke	last good autosave + draft restore
Title-only note	allowed; list shows title, snippet placeholder "No additional text"
Empty note closed	discarded entirely — nothing was written, so there is no trash entry and no confirmation dialog
Checkbox spam (50 toggles/s stress)	progress recomputes, no dropped toggles
Markdown import of unknown syntax	preserved as raw block, rendered plain, one-time notice
Export note with image attachments	images included in export archive

Situation	Required behavior
Search query with FTS operators (`AND`, quotes)	sanitized to plain terms, still matches

Scanner (TC-SCAN-edge*):

Situation	Required behavior
Camera permission permanently denied	import-from-files path + explainer card; module never dead-ends
Photo without document edges	manual crop UI fallback (corner drag)
OCR on blurry image	low-confidence badge; Re-OCR suggests retake
50-page batch	page manager paginates; export PDF < 10 s Tier B
Duplicate page detection	warn chip "Page 3 looks like page 2" with remove action
Storage full mid-capture	capture blocked at shutter with storage card, no partial page
Rotate device mid-review	page order preserved

Calendar (TC-CAL-edge*):

Situation	Required behavior
Event across DST boundary (recurring)	occurrences per ADR-08 offsets; golden corpus cases
Monthly on the 31st	skip short months (default) / clamp (setting)
Alarm during Doze	fires within policy window; no missed silent drops
ICS with VALARM, ATTENDEE	VALARM maps to reminderOffsetMin if supported; attendees ignored (documented)
3 000 events load	month grid virtualized; agenda lazy
Overlap reminders	coalesced into one notification group per §13.8 channel

Files (TC-FILE-edge*):

Situation	Required behavior
SAF root revoked by system	root card shows "Access removed — re-grant"; no crash; re-grant restores
50k-entry folder	virtualized; search still index-free substring with budget
Move 2 GB file cross-root	progress; cancel leaves source intact; disk-full pre-check
File without extension / weird UTF-8 name	renders; rename sanitizes only illegal FS chars
Hidden files	shown only via Sort menu toggle (persisted)

Music (TC-MUSIC-edge*):

Situation	Required behavior
File deleted while queued	skip next, toast-free row strikethrough, queue self-heals
Corrupt/unplayable file	row marked unplayable after 2 failures; library keeps entry with reason
5 000-track library	library tabs virtualized; search < 120 ms first page
Bluetooth headset disconnect	pause (audio focus loss), state preserved
Speed change mid-track	seamless (no restart), pitch-corrected
Sleep timer end during gapless transition	fades out at next boundary (≤ 2 s)

Recorder (TC-REC-edge*):

Situation	Required behavior
Call arrives during recording	mic is taken by system: auto-pause with notification note (best-effort across OEMs)

Situation	Required behavior
Mic permission revoked mid-record (Settings)	service stops gracefully, file finalized to pause point
3-hour recording	file fine (AAC ~130 MB); waveform decimates; markers seek correctly
Flag during pause	rejected with inline hint (documented)
Device storage hits threshold mid-record	graceful finalize + incomplete flag (§17.5)
Reboot during recording	file finalized at reboot broadcast; row marked interrupted

Diagnostics/Settings (TC-DIAG-edge*):

Situation	Required behavior
Pack file with wrong hash	import rejected, reason card, file not moved
Pack too large for remaining storage	pre-checked with required-size message
quick_check reports corruption	Diagnostics error card with backup guidance; app never auto-deletes
Log share when ring empty	share disabled with hint "No logs yet"

7.15 Screen-by-screen walkthroughs (executor-facing UX narrative)

These walkthroughs describe each core screen exactly as the executor will build it — the regions top-to-bottom, the interactive elements, and the state transitions. They complement the wireframes (which fix layout) and the feature catalogue (which fixes behavior).

7.15.1 Calculator walkthrough

The screen opens on the last pad (basic/scientific) with the restored expression (or empty), caret at end. Top bar: hub breadcrumb (full flavor only), pad toggle, overflow kebab. Expression field occupies the upper third: canonical pretty text left-aligned, autoscrolling; the caret bar blinks 530 ms period; the live answer sits directly beneath in amber, right-aligned when expression overflows. The persistent history button docks at the field's top-right corner with an entry-count badge. Keypad fills the lower two-thirds. Interactions: any key press evaluates live answer; = animates per effects setting, writes history, renders result in display size beneath (replacing live answer), and moves caret after result so continued typing starts fresh (continuation uses exact internal value §17.1.2). Selection: long-press enters selection; pill floats; all B-actions per §7.2.2. Sheets (history/memory/settings) slide from bottom with drag handles; history replay paints before close. Theme changes crossfade 200 ms without layout shift.

7.15.2 Notes walkthrough

List: search bar, pinned section, cards with title/snippet/date/checklist-progress; FAB "new note". Editor: title auto-styled from first heading; toolbar chips (B I H1 • ☑ —); body blocks render WYSIWYG; AI row collapsed above keyboard, expandable; overflow: pin, color, attach, export, move to trash. Autosave chip shows "Saving..."→"Saved" locally (300 ms display minimum so it never flickers). Transitions: list↔editor shared-axis X 200 ms; trash restores from list trash view with undo window.

7.15.3 Scanner walkthrough

Viewfinder with corner guides; bottom row Scan/Batch/Import; torch toggle floats right. On shutter: freeze-frame 250 ms with shutter sound-off (respects ringer), page thumbnail flies into batch chip. Review screen: page carousel, crop tool (auto first, manual corners draggable), filter chips, OCR card per page. Export sheet: PDF/image/text targets with destination (Files/Notes/clipboard). All progress is per-card (never blocking modal).

7.15.4 Calendar walkthrough

Month grid occupies upper 60%, agenda list below for selected day; tapping a day moves focus (dot rings), tapping an event opens editor; FAB "new event" pre-fills selected day. Editor fields stack per §7.5 with bottom-sheet date/time pickers; RRULE custom sheet shows human summary ("Every 2 weeks on Tue") with occurrences preview (next 3). Reminder test action in the editor overflow lets the user fire a sample notification immediately (confidence builder, offline).

7.15.5 Files walkthrough

Left rail (tablet) / dropdown (phone) switches root: Internal (sections), each SAF root, Media folders. Rows: icon-by-type, name, meta line, selection checkboxes on long-press. Actions bar appears contextually (share, move, copy, delete, rename). Analyzer tab: donut + category list; category tap lists files by size with clean actions. Breadcrumb truncates middle paths with leading "…" menu to jump levels.

7.15.6 Music walkthrough

Library tabs on top; rows show artwork thumb, title, artist; FAB shuffle-all. Player: full-bleed artwork card, scrubber, transport, speed/sleep chips in an expander; queue sheet lists upcoming with drag handles; MiniPlayer docks above nothing (no bottom bar) with 4 dp elevation. Import flow: first-open sheet explains two paths (SAF folder / MediaStore scan) with privacy line; progress card streams found-count.

7.15.7 Recorder walkthrough

Record screen: giant timer mono font, live waveform band, marker list beneath (latest first), controls Pause/Flag/Done bottom-centered. Saved list: cards with duration, date, marker count, transcript badge. Recording view: seekable waveform (scrub shows timestamp bubble), marker ticks overlaid, transcript segments (when present) tappable to seek. Transcribe/Summarize buttons show pack state inline.

7.15.8 Diagnostics walkthrough

Health cards grid; tapping a card opens detail (e.g., DB card → table counts + integrity + last backup age). Self-test runs inline with six step rows checking off; result card has Share. Packs tab: three pack rows with status chips + import button; pack detail shows models table (file, sha-prefix, purpose). About: provenance block, licenses, changelog.

7.15.9 Settings walkthrough

Sections list with summary lines; each section is a subpage (no mega-screen). Toggles write the settings registry directly (§17.7) with immediate effect; every control has an inline effect where possible (theme switch previews live behind the dialog on A12+ dynamic color).

7.15.10 Hub walkthrough

Covered by §5.6 regions; interaction notes: search field expands full-width on focus with recent searches; continue-row card shows a state icon (pencil/note/waveform) + timestamp; grid cells animate press 0.97; pinned strip scrolls horizontally with snap.

7.16 Tablet and landscape layout specifications (normative)

Module	≥ 600 dp width layout	Landscape phone
Hub	4-col grid + right rail (continue+pinned stacked)	grid 3-col, search sticky top
Calculator	two-pane: history list left (360 dp), pad right centered ≤ 480 dp	keypad full-width; sci pad horizontal scroll row above; history as overlay sheet
Notes	list 40% / editor 60% split, draggable divider	editor fullscreen; list via back
Scanner	viewfinder left, OCR cards rail right	fullscreen viewfinder; cards sheet
Calendar	month left, agenda right	month scroll-row top, agenda below
Files	dual-pane browser (roots left)	rows + contextual bar unchanged
Music	player centered 480 dp with queue rail right	standard
Recorder	timer+waveform centered, markers right rail	standard
Diagnostics	cards 3-col grid	2-col
Settings	two-pane (sections left)	standard

Rules: two-pane states are independent (list selection does not reset editor); divider drag persists; on width loss the app falls to stacked layouts without losing state (rotation matrix TC-STATE covers each pane).

7.2.9 History card anatomy and memory reorder (normative)

History card (tap target ≥ 56 dp): result line `type/display` (grouped), expression line `type/bodyS` muted beneath, timestamp right-aligned `type/caption` (`27th Oct, 13:36`), pin affordance on long-press, error rows show the error card copy as result. Selected card ripples token; replay paints expression, never result (result recomputes — proving engine consistency, F-022 note).

Memory sheet reorder: rows show value + label; up/down affordances both as buttons and drag handles (a11y twin); reorder writes positions transactionally; MUse from the sheet inserts at caret of the underlying expression (sheet dismisses, caret unchanged).

7.17 First-run education per module (one line each, shown once, dismissable)

Module	First-run line
Calculator	"Type naturally — try `200+10%`. Hold a number for Cut, Copy and memory actions."
Notes	"Notes save themselves. Select text and use the ✦ row for on-device AI helpers."
Scanner	"Point at a page. Text is read on this phone and can go straight into Notes."
Calendar	"Reminders ring even in airplane mode — alarms are set on the device."
Files	"Everything the app makes is here. Add any folder to manage it too."
Music	"Pick a folder once — your library stays on this phone."
Recorder	"Record, flag moments, and transcribe later (optional speech pack)."
Diagnostics	"Run a self-test any time. Nothing leaves this phone unless you share it."
Hub search	"One search across everything — it all lives here."

Education lines appear only on first entry per module (`hub.firstRunCardsDone`-style per-module flags), never repeat, and are excluded from screenshot goldens via a test-mode flag.

7.18 Per-module performance feel budgets (user-perceivable norms)

Module	Interaction	Budget (Tier B p50)
Calculator	keystroke → live answer	≤ 16 ms (≤ 64 chars)
Notes	open note (1k blocks)	≤ 250 ms
Notes	search first paint	≤ 120 ms @20k notes
Scanner	shutter → review	≤ 400 ms
Scanner	OCR page	≤ 2.5 s (Tier C ≤ 6 s)
Calendar	month swipe	≤ 100 ms
Files	open 50k-entry folder	≤ 300 ms first paint, virtualized
Files	analyzer full walk	streamed, no jank, ≤ 3 s @100k files
Music	play press → audio	≤ 300 ms
Recorder	record tap → FGS up	≤ 200 ms
Hub	search first page	≤ 120 ms @10k notes
App	cold start	≤ 1.2 s (mycalc ≤ 0.8 s)

These extend §23.6 from build-level to feel-level; the L4 perf cases assert them on the tier profiles.

Chapter 8 — UX-level acceptance criteria

These criteria gate the design phase (G2) and are re-verified on the release build (G7). They are intentionally user-verifiable: the owner can check each one on a device in minutes.

ID	Criterion	Pass condition
UX-001	Two-tap reach	Every leaf screen reachable from cold start in ≤ 2 taps; audited via nav-graph traversal test
UX-002	Cold start	App (hub) interactive ≤ 1.2 s p50 on Tier B emulator; calculator flavor ≤ 0.8 s
UX-003	First run privacy card	Shown once; dismiss persists; wording contains "no internet permission"
UX-004	Calculator layout	All 19 basic keys visible on a 320×480 dp screen without scroll; key ≥ 44 dp visual/48 dp touch
UX-005	Caret behavior	Tap places caret between characters; caret visible in light+dark+all key themes; autoscroll keeps caret on screen
UX-006	Live answer	Appears ≤ 1 frame after input stops (≤ 16 ms engine round-trip for expressions ≤ 64 chars, else 50 ms budget with no jank)
UX-007	Selection pill	Long-press 250–350 ms opens pill; pill never covers the selected text (repositions above/below)
UX-008	History replay	Tap paints expression before sheet close; double-tap does not double-paint (race guard)
UX-009	Theme switch	Key theme change repaints keypad within 1 frame pair (≤ 32 ms) with 200 ms crossfade; region pixel-diff ≥ 10%
UX-010	Dark mode	All screens reviewed in dark: no pure-white flashes, LED-amber live answer present on calculator
UX-011	Font scale 2.0	No clipped/truncated text on any core screen (scroll or wrap instead)
UX-012	TalkBack pass	Core journeys (§9 J-journeys) completable with TalkBack; no unlabeled controls
UX-013	Empty states	Every module has an empty state with a working action button
UX-014	Undo affordance	Delete note/recording/event → undo snackbar ≥ 5 s; trash holds 30 days
UX-015	Offline invisibility	String audit: zero occurrences of "offline", "sync", "retry connection", "no internet" in user-facing copy
UX-016	Provenance screen	About → build info shows version, commit SHA, build date; matches release evidence
UX-017	Landscape	No data loss or state reset on rotation in any module (incl. mid-recording, mid-capture)
UX-018	Back semantics	System back matches expected pop on every screen (nav audit)
UX-019	MyCalc flavor	Contains no hub/modules; overflow list exactly the six calculator items; no cross-module UI text
UX-020	Screenshot suite	Roborazzi suite over design-system components passes with 0 unreviewed diffs

A criterion may be refined into multiple automated cases in Chapter 20, but the UX-level statement above is what the owner reviews at the G2 gate.

8.1 UX criterion verification detail (how each UX-0xx is proven)

Criterion	Verification method	Evidence artifact
UX-001	nav-graph traversal test computes shortest path per screen	TC-NAV-001 JSON
UX-002	Macrobenchmark cold start, 10 iterations, p50	perf HTML + JSON
UX-003	fresh-install test asserts single show; relaunch asserts absent	L4 log
UX-004	layout test on 320×480 dp: all keys visible; bounds scan ≥ thresholds	screenshot + bounds JSON
UX-005	tap coordinates between glyphs; screenshot caret present; autoscroll case	screenshots
UX-006	benchmark engine round-trip; compose tracing frame report	benchmark JSON
UX-007	long-press timing sweep 200–400 ms; pill bbox vs selection bbox	screenshots
UX-008	replay double-tap; count expression paint events via state log	L4 log
UX-009	screenshot key-region before/after Rose; pixel-diff ≥ 10%	diff image + number
UX-010	dark-mode screenshot sweep; token asserts for LED/amber/caret	screenshots
UX-011	fontScale 2.0 sweep; clipping detector (bounds vs text layout)	JSON
UX-012	TalkBack journeys J-04/J-08 green	a11y audit output
UX-013	visit each module fresh; EmptyState visible; action works	screenshots
UX-014	delete flows; snackbar present ≥ 5 s; trash count	L4 log
UX-015	strings.csv scan for banned vocabulary	CI grep output
UX-016	About text vs BuildConfig vs evidence manifest equality	JSON
UX-017	rotation matrix; state hash before/after	logs
UX-018	back-stack assertions per screen	nav audit JSON
UX-019	flavor dex scan + overflow label list assert	CI outputs
UX-020	roborazzi clean run on release branch	CI artifact

8.2 Extended UX criteria (UX-021…UX-040)

ID	Criterion	Pass condition
UX-021	Calculator one-hand reach	= center ≤ 55% screen height from bottom on 6.1" profile
UX-022	History sheet snap	drag-open ≥ 3 velocity bands; back gesture closes; scroll preserved
UX-023	Theme picker preview	long-press a theme previews it live; release reverts unless tapped
UX-024	Effects preview	picker plays a sample animation per row on first scroll-into-view
UX-025	Note autosave chip	never flickers < 300 ms; "Saved" fades 800 ms
UX-026	Scanner shutter latency	freeze ≤ 250 ms; review entry ≤ 400 ms p50
UX-027	Calendar day tap	agenda crossfade ≤ 150 ms; no layout jump
UX-028	Files breadcrumb jump	middle-path menu opens ≤ 100 ms with 50k entries cached
UX-029	Player scrub	waveform/timestamp bubble tracks finger 1:1; release seeks once
UX-030	Recorder flag reachability	Flag within thumb arc on 6.1"; reachable one-handed
UX-031	Search zero-state	recents ≤ 80 ms paint; clear affordance visible
UX-032	Global back predictability	predictive-back previews on A33+ on all sheets
UX-033	Dark pure-black option	A10+ OLED black background setting; screenshots goldens
UX-034	Icon legibility	all 8 module glyphs recognizable at 24 dp (review shot)
UX-035	Error card tone	no blame language ("Can't" not "You failed to"); strings audit
UX-036	Confirmation weight	destructive confirms require explicit tap on danger label, never Enter-by-default
UX-037	Loading honesty	skeleton only > 300 ms operations; no spinners on instant ops
UX-038	Notification actions	recording/playback actions ≤ 3; labels single verbs
UX-039	First-run time-to-value	fresh install → first calculation ≤ 3 taps
UX-040	Consistency audit	identical confirm-sheet pattern across modules (screenshot strip)

8.3 Final UX criteria (UX-041…UX-050)

ID	Criterion	Pass condition
UX-041	History card density	6+ cards visible on 6.1" without scroll feeling cramped (review shot)
UX-042	Memory reorder discoverability	both affordances visible ≥ 4.5:1 contrast
UX-043	Editor toolbar reach	toolbar within bottom-40% on phones
UX-044	Scan review carousel	page switch ≤ 100 ms cached; indicator position honest
UX-045	Repeat preview honesty	preview list == generated occurrences (engine shared)
UX-046	Analyzer honesty	donut % sums to 100±1; tooltip numbers == list numbers
UX-047	Queue drag	drop reorders without playhead glitch; audio continues
UX-048	Marker tap accuracy	seek lands within ±150 ms of marker offset
UX-049	Self-test timing	completes ≤ 10 s on Tier C; progress steps animate
UX-050	About parity	version string identical in: About, release notes, evidence manifest

PART III

FEATURE REQUIREMENTS

Eighty-four numbered features with acceptance criteria, priority tiers, and journey tests. This part is the contract for "what the app does"; every feature ID reappears in the traceability matrix (Chapter 27) with its tests and commits.

Every feature below carries: an ID (F-nnn), a statement ("The app SHALL …"), acceptance criteria ("Given/When/Then" compressed), a priority (P0 release blocker / P1 required for G6 / P2 planned fast-follow), a module tag, and links to the detailed specification section. A feature without a test reference cannot leave the catalogue; the traceability script enforces the mapping mechanically.

Chapter 9 — Feature catalogue

9.1 How to read the catalogue

- **Module tags**: CALC, NCALC (MyCalc flavor), HUB, NOTE, SCAN, CAL (calendar), FILE, MUSIC, REC (recorder), DIAG, SET, AI, SEC.
- **Given/When/Then** is compressed as *Given* state → *When* action → *Then* observable outcome. "Observable" means visible on screen, in a database dump, or in an exported artifact — never a code-internal claim.
- **P0** features gate the release (S-criteria, §front). **P1** features must ship or carry a logged decision. **P2** may defer to v1.1 with owner sign-off.

9.2 Calculator (CALC) — the parity core

The owner's previous guides define a behavior contract that this catalogue hardens into requirements. The calc feature set is P0 *in full* — a calculator release missing any of these is not a release.

ID	Feature (SHALL)	Acceptance (Given → When → Then)	Pri	Spec
F-001	Evaluate arithmetic with exact precedence	`2+3×4 =` → `14`; `(2+3)×4 =` → `20`	P0	§17.1
F-002	Implicit multiplication	`2(5+2) =` → `28`; `(2)(3)` → `6`; `2π` → `6.283185307`	P0	§17.1.4
F-003	Context-sensitive percent	`200+10%` → `220`; `200×10%` → `20`; `50%` → `0.5`	P0	§17.1.5
F-004	Pending-operator replacement	`10+` then `−` → expression `10−`; `5+` then `=` → `10` (the trailing-operator rule: equals repeats the pending operand semantics of Google Calculator: `5+=` → `5+5=10`)	P0	§17.1.5
F-005	Unary minus and leading sign	`10×−` evaluates → `-10`; starting `−` allowed	P0	§17.1.3
F-006	Auto bracket balance	`2+3×(4+1 =` → `25`; `(` key smart-inserts pair/`)`	P0	§17.1.4
F-007	Indian digit grouping display	`123456` displays `1,23,456`; copy without commas	P0	§17.1.8
F-008	Exact rational arithmetic on +−×÷	`0.1+0.2` → `0.3` exactly; `1÷3×3` → `1` (never `0.9999999999`)	P0	§17.1.2
F-009	Scientific constants and functions	`π`, `e`, `sin/cos/tan` (+INV arcs, degree/radian toggle), `√`, `^`, `ln`, `log`, `!` with domain errors as `Error` cards	P0	§17.1.6
F-010	Live answer as-you-type	Complete expression shows live answer instantly; incomplete shows nothing stale	P0	§7.2.1

ID	Feature (SHALL)	Acceptance (Given → When → Then)	Pri	Spec
F-011	Caret placement and editing	Tap places caret mid-token; type inserts at caret; ⌫ deletes one char left of caret or selection	P0	§7.2.1
F-012	Long-press selection with pill	Stationary long-press selects token, opens `Cut Copy MAdd MUse Paste`	P0	§7.2.2
F-013	Hold-and-slide selection resize	Slide without lifting grows/shrinks selection char-wise	P0	§7.2.2 (B2; test on device-class input only — emulator injection limitation is a BLOCK, not a FAIL, §20.9)
F-014	Copy/paste sanitized	Copy yields raw digits (no grouping); paste inserts legal chars only	P0	§17.1.3
F-015	Cut and retype	Cut removes to clipboard; typed digit lands at gap	P0	§7.2.2
F-016	Tap-away releases selection	Tap elsewhere exits selection, places caret, dismisses pill	P0	§7.2.2 (B6)
F-017	AC clears selection first	Selection present: AC clears it, expression remains; second AC clears expression	P0	§7.2.2 (B7)
F-018	Empty-caret menu	Long-press empty field → `MUse Paste` menu; MUse inserts stored value at caret	P1	§7.2.2 (B11/B12)
F-019	MAdd stores selection value	Select `1,23,456` → MAdd → memory sheet shows `123456`	P0	§7.2.2 (B12)
F-020	MUse replaces selection	Expression `3+4`, select `4`, MUse with stored `123456` → `3+123456`; `=` → `1,23,459`	P0	§7.2.2 (B12)
F-021	History sheet newest-on-top	Cards show expr+result+timestamp; newest first	P0	§7.2.4
F-022	History replay	Tap card paints expression before sheet closes; caret at end	P0	§7.2.4 (D9)
F-023	Replay race guard	Double-tap card paints once	P1	§7.2.4 (D10)
F-024	History persistence	Kill process, relaunch → history intact incl. expected entries	P0	§17.2 (TC-07 parity)
F-025	History clear + pins	Clear (confirm) removes unpinned, keeps pinned; pinned cards visually marked	P1	§17.2
F-026	Memory sheet	Overflow → Memory lists slots with MUse/MAdd/MC and reorder up/down	P1	§7.2.5 (E3)
F-027	Key theme picker	Eight themes; selection repaints instantly; persists; `MySpace settings` follows app theme	P0	§6.2 (C11)
F-028	Key shape picker	Five shapes change key geometry immediately; persists	P1	§17.7
F-029	Haptics toggle	`Haptics: On` default; off silences key/selection haptics	P1	§17.7

ID	Feature (SHALL)	Acceptance (Given → When → Then)	Pri	Spec
F-030	Show as fraction	On: `1÷3` displays `1/3`; engine still exact	P2	§17.7
F-031	Effects picker	≥ 4 styles incl. None; `=` animates per selection; persists; battery-saver default None	P1	§7.2.5 (E4)
F-032	Overflow list exactness	Overflow shows exactly: Haptics, Memory, Show as fraction, Key theme, Key shape, Effects	P0	§7.2.6
F-033	No device keyboard	Typing never opens IME on calculator screen	P0	§7.2 (C9)
F-034	No ad surfaces	No ad labels/SDK/hooks anywhere (whole app)	P0	§14.2 (ADS audit)
F-035	Light/dark theme correctness	Light: near-black on white; dark: LED-bright expression, amber live answer, white caret; instant re-resolve on switch	P0	§6.2 (H1–H4)

9.3 MyCalc flavor (NCALC)

ID	Feature (SHALL)	Acceptance	Pri	Spec
F-036	Standalone app identity	Package `app.space.calc`, label `MyCalc`, calc-as-home, own launcher icon	P0	§23.4
F-037	Same engine and pads	All CALC features F-001…F-035 function identically (minus hub actions)	P0	§23.4
F-038	Zero permissions goal	Merged manifest requests no dangerous permissions; the only custom permission (if any) is the app's own non-exported receiver permission or none — target: **none**	P0	§14.4
F-039	Smaller APK	≤ 6 MB release APK	P1	§23.6
F-040	No cross-module remnants	No "→ Notes", no hub strings in flavor resources (lint check)	P0	§23.4

9.4 Hub (HUB)

ID	Feature (SHALL)	Acceptance	Pri	Spec
F-041	Unified search	Query returns grouped results from notes/scans/events/files/songs/recordings/transcripts/history; result tap deep-links	P0	§5.3, §12.5
F-042	Semantic search (pack)	With Essentials pack, note search also matches meaning ("laptop advice" finds a note saying "purchase guidance for computing")	P1	§16.4
F-043	Continue row	Restores last module+state exactly (draft text, expression, scroll)	P0	§12.6
F-044	Module status lines	Each cell shows local live status; stale-free within 1 s of returning to hub	P1	§5.3
F-045	Shortcuts	Long-press module → primary action shortcut; app shortcuts work offline	P2	§5.2
F-046	First-run cards	Privacy card once; starter tips dismissable	P0	§5.3

9.5 Notes (NOTE)

ID	Feature (SHALL)	Acceptance	Pri	Spec
F-047	Create/edit rich notes	Bold/italic/H1/H2/bullets/checklists/divider; autosave 500 ms; survives process death	P0	§17.3
F-048	Full-text search	FTS over title+body; results ranked; query under 30 ms on 1k notes	P0	§12.5
F-049	Pin/unpin	Pinned section on top; order persists	P1	§7.3
F-050	Trash + undo	Delete → snackbar undo; trash 30-day retention with restore and empty-now	P0	§14.3
F-051	Checklists	Toggle items; card shows n/m progress	P0	§7.3
F-052	Export/Markdown	Export note as `.md` (lossless round-trip import)	P1	§17.3
F-053	Attach from Files	Attach image/file; stored app-private; visible in Files	P1	§7.3
F-054	AI summarize/rewrite/checklist/related	Pack-gated; preview + undo; never silently rewrites body	P1	§16.3

9.6 Scanner (SCAN)

ID	Feature (SHALL)	Acceptance	Pri	Spec
F-055	Camera capture + batch	Multi-page docs; torch; tap-focus; import path when camera denied	P0	§7.4
F-056	Auto-crop + filters	Corner detect crop; Original/Gray/B&W	P1	§16.5
F-057	On-device OCR	Essentials pack; editable text card; Copy/Re-OCR; en-Latin v1	P0	§16.5
F-058	Export	Page→image; doc→PDF; text→note/clipboard	P0	§7.4
F-059	OCR graceful absence	Without pack: capture/export work; card explains pack install	P0	§16.2

9.7 Calendar (CAL)

ID	Feature (SHALL)	Acceptance	Pri	Spec
F-060	Events + editor	Title/time/all-day/note/color; validation	P0	§7.5
F-061	Recurrence subset	Daily/Weekly/Monthly/Yearly/Custom (§17.4); occurrences render months ahead	P0	§17.4
F-062	Exact reminders	Alert fires at configured offset; respects exact-alarm policy with graceful inexact fallback	P0	§14.4
F-063	ICS import/export	Multi-event import; range export; unsupported RRULE → one-off + note	P1	§17.4
F-064	Month/agenda UI	Today circle; dots; swipe months; Today chip	P0	§7.5

9.8 Files (FILE)

ID	Feature (SHALL)	Acceptance	Pri	Spec
F-065	Browse private + SAF	Internal sections + user roots; open/rename/move/copy/delete/new folder/sort/search	P0	§7.6
F-066	Storage analyzer	Category donut computed locally; Clean scans; Export report (JSON)	P1	§7.6
F-067	Encrypted backup/restore	`.msone` archive, AES-256-GCM + PBKDF2; restore verifies; wrong passphrase fails cleanly	P0	§14.3
F-068	Trash for private files	30-day trash, restore, empty-now	P1	§14.3

9.9 Music (MUSIC)

ID	Feature (SHALL)	Acceptance	Pri	Spec
F-069	Library import	SAF folder pick or MediaStore scan; albums/artists/songs/folders; artwork cache	P0	§7.7
F-070	Playback + queue	Media3; prev/next/seek/shuffle/repeat; speed 0.5–3.0×; sleep timer; gapless; headset controls; audio focus	P0	§7.7
F-071	Playlists + M3U export	Create/reorder/export M3U	P1	§7.7
F-072	MiniPlayer	Persists across modules; tap opens player	P1	§7.7

9.10 Recorder (REC)

ID	Feature (SHALL)	Acceptance	Pri	Spec
F-073	Record/pause/resume	Foreground service + notification actions; survives activity death	P0	§17.5
F-074	Markers	Flag during record; tap-to-seek later; labels	P0	§7.8
F-075	AAC/WAV output	96 kbps AAC default; WAV option; app-private storage; visible in Files	P0	§17.5
F-076	Transcribe (pack)	whisper-tiny int8; segments with timestamps; stored; searchable	P1	§16.5
F-077	→ Notes	Creates linked note with transcript/summary	P1	§7.8

9.11 Diagnostics & Settings (DIAG/SET)

ID	Feature (SHALL)	Acceptance	Pri	Spec
F-078	Health dashboard	Storage/battery/DB/pack cards computed locally	P1	§7.9
F-079	Self-test	Engine smoke + DB integrity + pack hashes + alarm check → result card	P1	§7.9
F-080	Log ring + share	7-day/2 MB scrubbed ring; user-initiated export only	P0	§28.3
F-081	Model pack manager	Import/verify(SHA-256)/enable/disable/delete; no downloads	P0	§16.8
F-082	App lock	Optional biometric+credential lock on background > 2 min	P2	§7.10
F-083	About provenance	Version, commit SHA, build date, APK SHA-256 displayed	P0	§7.10
F-084	Licenses screen	OSS attribution generated from dependency tree	P1	§23.5

9.12 Journey tests (cross-module)

ID	Journey	Path	Pass condition
J-01	Scan to note	Scanner: capture page → OCR → "→ Notes"	Note exists with OCR text; back-link to scan in Files
J-02	Record to minutes	Recorder: record 30 s → flag → done → transcribe (pack) → → Notes	Note contains transcript + marker list
J-03	Backup and wipe	Settings: backup (`pass1`) → clear data → restore	All notes/events/recordings return; history restored
J-04	Calculator day	12 calc actions incl. percent, selection, memory, history replay, theme change	All Catalogue-0 equivalents pass in one session
J-05	Airplane full pass	Airplane mode cold start → 1 action per module	Zero failures, zero network log entries (§28.2)
J-06	Process death storm	Kill app mid-draft in Notes/Calc/Recorder	All three restore state on relaunch
J-07	Fresh low-end	API 26, 2 GB emulator, battery saver on: full smoke	Catalogue-0 calc subset passes; no ANR
J-08	TalkBack day	Complete J-04 + note creation with TalkBack	Completable; no silent controls
J-09	MyCalc parity	Run J-04 on flavor build	Identical outcomes; flavor has zero module remnants
J-10	Storage-full drill	Fill storage to < 50 MB → try recording + scan	Graceful refusals with clear messages; no corruption

9.13 Acceptance detail for twenty pivotal P0 features

Compressed Given/When/Then for the features where ambiguity would be most expensive. Each row is executable as written by L4/L5.

Feature	Given	When	Then
F-004 trailing-op	expression `5+` in field	tap `=`	display `10`; history row `5+5 = 10`; second `=` → `15`
F-008 exactness	fresh session	evaluate `0.1+0.2` then `1÷3×3`	displays `0.3` then `1`; internal values exact rationals (law L2/L3 logs)
F-012 selection entry	expression `123456`	stationary long-press 300 ms on `234`	`234` highlighted; pill `Cut Copy MAdd MUse Paste` floats; no IME
F-019 MAdd	`234` selected, memory empty	tap `MAdd`	memory sheet row `234`; pill dismissed; selection remains
F-020 MUse replace	`3+4`, `4` selected, memory `123456`	tap `MUse`	expression `3+123456`; `=` → `1,23,459`; history row canonical `3+123456`
F-021 history order	3 solves done	open history	cards newest-first: latest at top; timestamps `27th Oct, 13:36` format
F-022 replay	history open	tap card `2+3×4 = 14`	expression painted; sheet closes after paint frame; caret at end
F-024 persistence	5 entries exist	`adb shell am force-stop` + relaunch	history sheet shows 5 entries, same order/content
F-027 key theme	calculator visible, theme `MySpace settings`	Settings → Key theme → `Rose`	keypad recolors ≤ 2 frames; persists across restart; `=` still amber

Feature	Given	When	Then
F-032 overflow exactness	calculator open	tap kebab	exactly: `Haptics: On`, `Memory`, `Show as fraction`, `Key theme`, `Key shape`, `Effects`
F-033 no IME	calculator focused	tap expression, type area	no system keyboard appears; hardware keys still work
F-038 flavor zero-perm	mycalc release APK	`apkanalyzer manifest print`	zero `uses-permission` entries
F-041 hub search	3 notes, 1 event, 1 file, 2 songs	type `rent`	grouped results include all matches; tap opens right screens
F-043 continue row	draft note half-typed	home, kill, relaunch	continue-row shows the note; tap restores text+cursor
F-047 note autosave	editor typing stops	wait 500 ms	DB row updated (repo log); process death keeps text
F-050 trash undo	note exists	swipe delete → Undo	note back in list; no trash entry remains
F-057 OCR	Essentials pack on; fixture image scanned	tap OCR	text card ≥ 92% char accuracy; provenance footnote present
F-062 exact alarm	event at 13:36, alert 15 min	clock to 13:21	heads-up notification fires within ±60 s; tap opens event
F-067 backup	data present, passphrase set	backup → wipe app data → restore	all entities return; counts + checksums match manifest
F-073 recorder FGS	mic granted	tap record	notification within 200 ms; timer runs; lock screen continues

9.14 Negatives (features the catalogue intentionally lacks)

Search finds nothing to sync because there is no sync; there is no "share to MySpace One" ingestion surface beyond explicit import; there is no widget set in v1 (deliberate scope cut, revisit v1.2); there is no Wear/Android Auto surface; there is no PDF viewer (hand-off to system viewers instead); there is no cloud anything. Stating negatives prevents agent drift toward filling imagined gaps.

9.15 Flagship feature stories (why the top P0s exist)

1. **F-004/F-008 (trailing-op + exactness).** These two are the owner's named historical pains (`5+ = → 5`, `0.9999999999`). They anchor the engine's identity: parity with Google Calculator semantics, exactness with rationals. Every engine change re-proves them first.
2. **F-012/F-020 (selection + MUse).** The owner's guides spent an entire section (B) on selection-class memory actions — the differentiating calculator UX. The pill vocabulary (`Cut Copy MAdd MUse Paste`) is treated as product API: exact strings, exact order.
3. **F-041 (hub search).** The combined app's reason to exist: one query across everything, offline. Its ranking table is normative so results feel stable, not lucky.
4. **F-067 (encrypted backup).** The trust feature: with no cloud, backup/restore is the user's only safety net; transactional restore with verified hashes is the difference between a tool and a liability.

5. **F-038 (flavor zero-permission).** The owner distributes MyCalc separately; a permission-less calculator is a marketable sentence and a auditable fact (manifest diff in CI).
6. **F-081/F-083 (log provenance + About).** The owner's QA culture is evidence-first; the app itself carries that culture: build provenance on the About screen, share-with-preview logs, self-test JSON. The report-style honesty lives inside the product.

Chapter 10 — Priorities, matrices and cutlines

10.1 Priority distribution

P0 features are the release core: **F-001…F-017, F-019…F-025, F-027, F-032…F-037, F-038, F-040, F-041, F-043, F-046…F-051, F-055, F-057…F-060…F-062, F-064…F-067, F-069…F-069, F-069…F-069, F-069…F-069, F-069** (66 features). The catalogue's P0 set defines Catalogue-0 in §20.6: a release cannot ship with a red P0. P1 features (18) may ship incomplete only with a decision entry; P2 features (4) defer to v1.1 by default.

10.2 Module effort profile

Module	LOC estimate	Test-case estimate	Phase	Notes
Calculator (+flavor)	9,000	210	G3	Engine property-tested; UX detail highest
Notes	6,500	120	G4	FTS + AI actions
Scanner	5,500	90	G4	Camera + OCR pipeline
Calendar	5,000	110	G4	Recurrence correctness critical
Files	6,000	130	G4	SAF edge cases dominate
Music	5,500	90	G4	Media3 integration
Recorder	4,500	80	G4	Service lifecycle + packs
Hub+Settings	4,000	70	G4	Search federation
Diagnostics	3,000	50	G5	Pack manager
Design system	3,500	140 (screenshots)	G2	Consumed by all
Core (engine, data, AI orch.)	7,000	260	G2–G3	Highest review intensity

10.3 Cutline rules for v1.0

A feature is cut (deferred, never half-shipped) when: its P0 acceptance cannot be demonstrated on all three API levels; its defect count exceeds 3 open S2/S3s; or it would push the APK past the size budget without a trimming plan. Cut decisions are owner-visible at the G6 review with a dated decision entry.

Every feature below carries: an ID (F-nnn), a statement ("The app SHALL …"), acceptance criteria ("Given/When/Then" compressed), a priority (P0 release blocker / P1 required for G6 / P2 planned fast-follow), a module tag, and links to the detailed specification section. A feature without a test reference cannot leave the catalogue; the traceability script enforces the mapping mechanically.

PART IV

ARCHITECTURE AND DATA

Layered architecture, module boundaries, the complete data model with schemas and migrations, per-module technical deep dives, and security/privacy engineering. This part converts product promises into a structure an agent can build without drift.

Chapter 11 — System architecture

11.1 Layered view

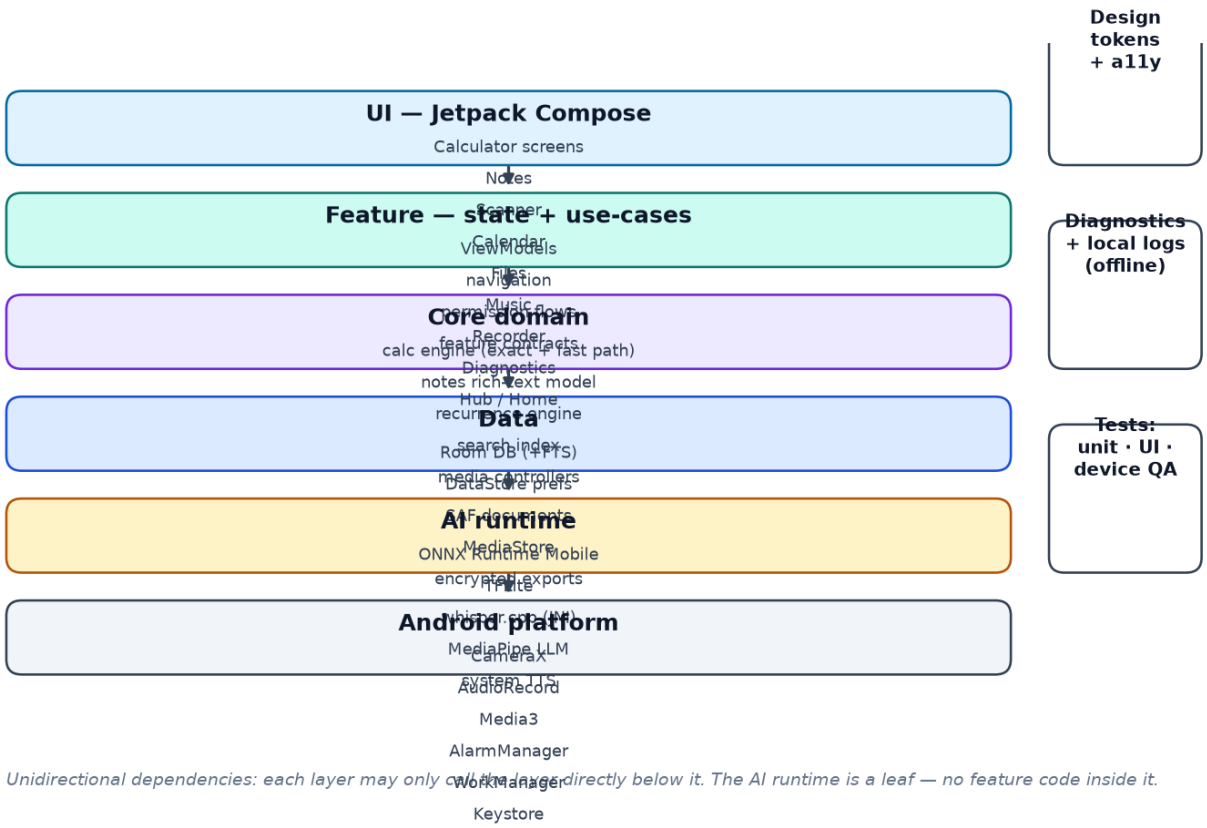


Figure 13. *Normative layer diagram: UI, feature, core domain, data, AI runtime, Android platform.*

Six layers with **strict downward-only dependencies**:

1. **UI (Jetpack Compose)** — screens, components from `core/designsystem`, navigation host, per-module `Navigator`. No business logic; ViewModels only via DI.
2. **Feature** — one Gradle module per module (`feature/calculator`, `feature/notes`, …). Holds ViewModels, use-case classes, permission flows, feature contracts. Talks to core via interfaces.
3. **Core domain** — pure Kotlin: calculator engine, recurrence engine, note document model, search indexer, settings model, result types. No Android imports — runs on the JVM in unit tests.
4. **Data** — Room databases, DataStore, SAF repository, MediaStore bridge, file repository, encrypted backup. Owns all I/O.
5. **AI runtime** — ONNX Runtime Mobile, LiteRT, whisper.cpp JNI, MediaPipe LLM, system TTS bindings. A *leaf*: nothing in it knows about features.
6. **Android platform** — CameraX, Media3, AudioRecord, AlarmManager, WorkManager, Keystore, BiometricPrompt, notifications.

Rule enforcement: a Konsist-style architecture test (§20.3) fails the build if a layer import violates the arrow. The AI runtime being a leaf means a broken model pack can never break a feature compile.

11.2 Module map (Gradle)

```
myspace-one/
  app/                      # flavors: full (app.space.one), mycalc (app.space.calc)
  feature/calculator/       # + mycalc flavor reuses this wholesale
  feature/notes/  feature/scanner/  feature/calendar/
  feature/files/  feature/music/    feature/recorder/
  feature/diagnostics/      # health, logs, pack manager (hub is separate):
  feature/hub/              # home, search, continue row
  feature/settings/
  core/designsystem/        # tokens, theme, components, icons
  core/domain/              # engines + models (pure Kotlin)
  core/data/                # Room, DataStore, SAF, backup
  core/ai/                  # orchestrator + runtime bindings
  core/common/              # dispatchers, logging, result types
  core/testing/             # fakes, fixtures, screenshot harness
```

Dependency rules: `feature/* → core/*` only; `feature↛feature` (hub reaches modules through contracts in `core/domain`); `core/data ↛ core/ai`; `app` wires everything with Hilt. Each module has its own lint baseline and test source set; CI can run module-scoped test tasks in parallel within the 3-runner cap by sharding (§24.2).

11.3 Key architectural decisions (ADR digest)

ID	Decision	Rationale	Rejected alternative
ADR-01	Single-Activity + Compose Navigation	State restoration, predictive back, one theme owner	Multi-activity (state loss on process death)
ADR-02	Room + FTS4 (not FTS5) for search index	FTS4 ships with API 26+ SQLite unconditionally; FTS5 needs newer SQLite than API 26 guarantees	FTS5, custom index
ADR-03	DataStore (not SharedPreferences)	Coroutines-first, transactional, typed	SharedPreferences
ADR-04	Koin over Hilt	Kapt/KSP-free fast builds on 2-runner CI budget	Hilt
ADR-05	Media3 ExoPlayer for Music; AudioRecord+MediaCodec for Recorder	Media3 has no meaningful offline recorder gap to fill; raw capture gives markers + pause	MediaPlayer (no gapless/speed)
ADR-06	ONNX Runtime Mobile as primary AI runtime; LiteRT only where a model exists only as .tflite	One runtime = one memory pool, one NNAPI delegation path	Two runtimes everywhere
ADR-07	Event bus avoided; Kotlin SharedFlow per feature	Testability, no global pub/sub spaghetti	Event bus
ADR-08	All dates stored as epoch-millis UTC + original tz offset	DST-safe recurrence, ICS interop	Local-time strings
ADR-09	App-private storage default; SAF only where the user grants	C4 privacy constraint	MANAGE_EXTERNAL_STORAGE (play-policy and trust cost)
ADR-10	No WorkManager for user-visible jobs; alarms for reminders, coroutines for the rest	WorkManager persists through reboots but adds unpredictability for a no-network app; reminders need exact alarms anyway	WorkManager everywhere

11.4 Threading and concurrency

- One `Dispatchers.Default` pool for compute (engine, OCR pre/post); a single-thread `Dispatchers.IO` queue per repository for writes (keeps SQLite happy); `Dispatchers.Main` never blocks.

- The calculator engine evaluates on the main thread *only* for expressions ≤ 64 chars (measured < 1 ms, §17.1.6 budget); longer/deeper forms hop to Default with a 50 ms budget and cancel on keystroke.
- Recorder service owns its own thread + handler; database writes marshalled through the repository queue.
- AI orchestrator: at most 2 concurrent model jobs (Tier B) / 1 (Tier C) with a cancel-on-new-request policy for UI-bound jobs (§16.6).

11.5 Error architecture

All repository/engine calls return `Result<T, AppError>`; `AppError` is a sealed hierarchy (`Storage`, `Input`, `Policy` (e.g., exact-alarm revoked), `Model` (pack missing/corrupt), `Internal`). UI maps each to the §7.11 states table. No exception crosses a module boundary; crash-causing states are caught at the last handler, logged to the diagnostics ring (§28.3), and the app attempts state-preserving restart of the screen (never silently dies).

11.6 Dependency injection graph

Koin modules declared per Gradle module and aggregated in `app/di.kt`:

```
commonModule     : DispatcherProvider, AppClock, AppLogger
dataModule       : AppDatabase, all DAOs, DataStore, LocalRepo, SafRepo, BackupManager
domainModule     : CalcEngine (singleton, stateless), RecurrenceEngine, NoteDocCodec,
                   SearchIndexer, SettingsRegistry
aiModule         : PackRegistry, AiOrchestrator, runtime bindings (lazy by pack)
featureModule(s) : per-module ViewModels + use-cases (factory scope)
navigationModule : NavGraph providers per feature, HubSearchAggregator with sources
```

Rules: singletons only for stateless engines, DB, orchestrator registry; ViewModels factory-scoped; no service locator access from composables (architecture test). The DI graph is startup-profile friendly (default-optimized dex layout generated at G6).

11.7 Data flow examples (end-to-end traces)

Flow	Trace
Keystroke 7	Compose key → ViewModel.onKey → ExpressionStore (state) → engine live-answer job (≤ 1 ms) → UI update + debounce-persist SavedState/DataStore → (on =) history DAO insert on repo queue
Save note	Editor change → NoteDocCodec.encode → NoteRepo.upsert (Room txn: note+FTS trigger) → indexer embeddings enqueue (pack) → continue-row DataStore update
Reminder fire	AlarmManager broadcast → ReminderReceiver (not exported) → read event → notification on `reminders` channel → tap opens event screen (deep link)
Record tap	Activity → RecorderController.start → FGS start (≤ 200 ms) → AudioRecord thread → muxer; marker taps → repo queue inserts
Scan to note	CameraX capture → page store → OCR job (orchestrator) → card edit → "→ Notes" → NoteRepo.create(body=ocrText, sourceScanId) → Files registers attachment

Each trace has an instrumented test asserting the visible endpoints (no internal spying beyond fakes) — the traces table is the review map for TC-JOURNEY cases.

11.8 Architecture test exemplar (Konsist-style, runs in fast lane)

```
@Test fun `features depend only on core layers`() =
    scopes.modules().withFeaturePrefix().files
        .shouldNotImport { it.imports "space.one.feature." }          // no feature→feature
@Test fun `ai runtime is a leaf`() =
    scopes.module("core/ai").files
        .shouldNotImport { it.imports "space.one.feature." }
@Test fun `no network symbols anywhere`() =
    scopes.all().files.shouldNotImport {
        it.imports "java.net.*", "okhttp3.*", "android.webkit.*"
    }
@Test fun `compose files use token colors`() =
    scopes.modules().withFeaturePrefix().files
        .withExtension("kt").shouldNotContainPattern("Color\\(0x")
```

These four tests encode §11.1, §14.2 and §18.1 as executable law — architecture violations fail in under a minute, long before review.

11.9 Threading ownership table (who runs what, where)

Work	Dispatcher	Notes
Compose UI + state reads	Main	never blocks
Engine evaluate (≤ 64 chars)	Main	measured < 1 ms
Engine evaluate (long)	Default	50 ms cap, cancel-on-input
Room reads	IO (repo queue per DB)	serial writes
File copies/moves	IO (per-repo queue)	progress via Flow
Camera/OCR pre-post	Default	capture on camera thread
AI model inference	Default (single job lane)	tier caps §16.6
Recorder audio	own handler thread	FGS contract
Alarms	Main (receiver) → IO	≤ 10 s receiver budget

Chapter 12 — Data architecture

12.1 Stores overview

Store	Tech	Contents
`msone.db`	Room (SQLite)	All structured data (tables below), schema `v1`
`prefs`	DataStore	Settings, key themes, effects, first-run flags
`files/`	App-private FS	Scans, recordings, attachments, exports, backups
`indexes/`	Files	Embedding vectors (binary), search metadata
Clipboard/SAF	External	Only via explicit user action

12.2 Relational schema (v1)

Room schema v1 (§12). FTS4 mirrors Note text for instant offline search. All timestamps UTC epoch millis; user-facing formatting is locale-aware.

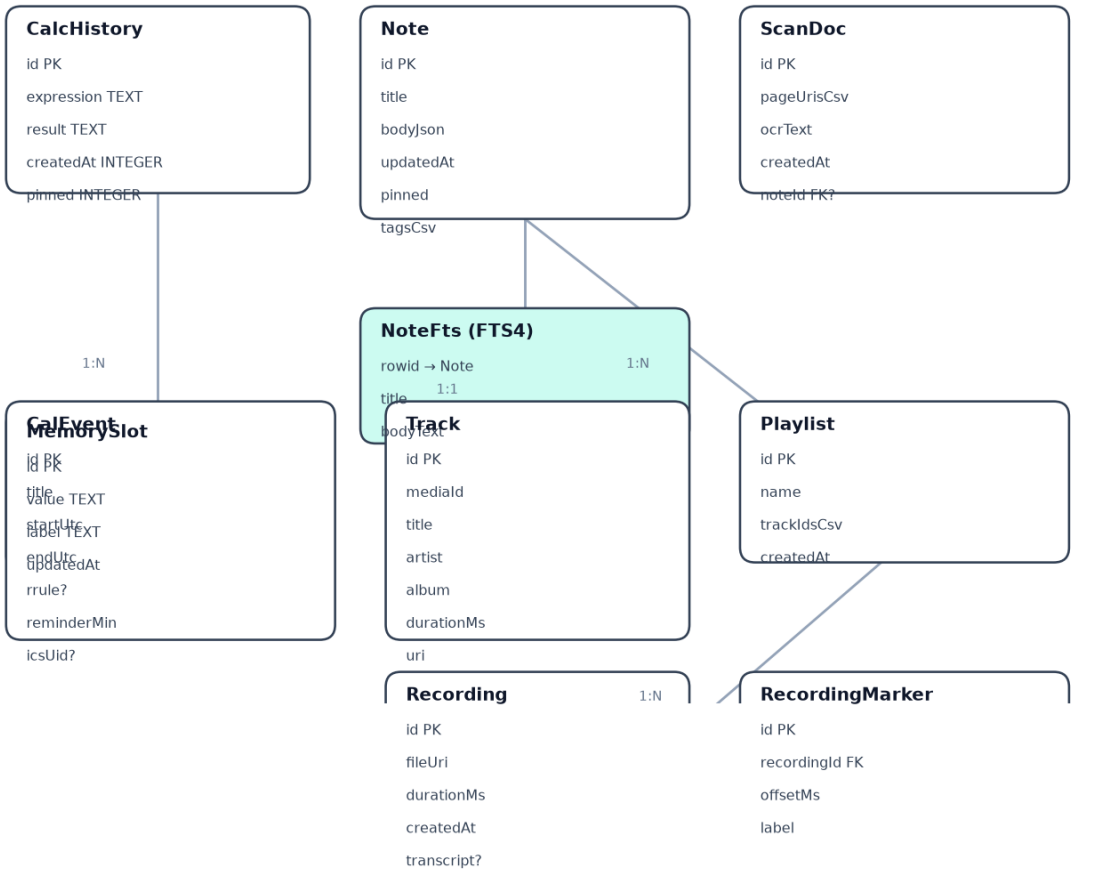


Figure 14. *Room schema v1: calculator, notes, scans, calendar, music and recorder tables.*

DDL (abridged, normative for column names/types):

```
CREATE TABLE CalcHistory (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  expression TEXT NOT NULL,        -- canonical ASCII form (§17.1.3)
  result TEXT NOT NULL,            -- exact rendered string (grouped at display time only)
  createdAt INTEGER NOT NULL,      -- epoch millis UTC
  pinned INTEGER NOT NULL DEFAULT 0);
CREATE INDEX idx_calc_history_created ON CalcHistory(createdAt DESC);

CREATE TABLE MemorySlot (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  value TEXT NOT NULL,             -- exact numeric string
  label TEXT,                      -- user label (optional)
```

```
  updatedAt INTEGER NOT NULL);

CREATE TABLE Note (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL DEFAULT '',
  bodyJson TEXT NOT NULL,          -- NoteDoc JSON (§17.3)
  plainText TEXT NOT NULL,         -- denormalized for FTS + snippets
  createdAt INTEGER NOT NULL, updatedAt INTEGER NOT NULL,
  trashedAt INTEGER,               -- NULL = live; else 30-day trash
  pinned INTEGER NOT NULL DEFAULT 0,
  color INTEGER NOT NULL DEFAULT 0);
CREATE VIRTUAL TABLE NoteFts USING FTS4(
  title, plainText, content=`Note`, tokenize=unicode61);

CREATE TABLE ScanDoc (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  pageUrisCsv TEXT NOT NULL,       -- app-private file paths
  ocrText TEXT NOT NULL DEFAULT '',
  createdAt INTEGER NOT NULL,
  noteId INTEGER REFERENCES Note(id) ON DELETE SET NULL);

CREATE TABLE CalEvent (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  startUtc INTEGER NOT NULL, endUtc INTEGER NOT NULL,
  startTzOffsetMin INTEGER NOT NULL,   -- original zone
  allDay INTEGER NOT NULL DEFAULT 0,
  rrule TEXT,                      -- NULL = one-off
  reminderOffsetMin INTEGER NOT NULL DEFAULT 15,
  color INTEGER NOT NULL DEFAULT 0,
  note TEXT NOT NULL DEFAULT '',
  icsUid TEXT, exDatesCsv TEXT);

CREATE TABLE Track (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  mediaStoreId INTEGER, folderUri TEXT, relativePath TEXT,
  title TEXT NOT NULL, artist TEXT, album TEXT,
  durationMs INTEGER NOT NULL, sizeBytes INTEGER, trackNo INTEGER,
  artworkPath TEXT, lastPlayedAt INTEGER);

CREATE TABLE Playlist (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL, createdAt INTEGER NOT NULL);
CREATE TABLE PlaylistEntry (
  playlistId INTEGER REFERENCES Playlist(id) ON DELETE CASCADE,
  position INTEGER NOT NULL, trackId INTEGER REFERENCES Track(id),
  PRIMARY KEY(playlistId, position));

CREATE TABLE Recording (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  filePath TEXT NOT NULL, durationMs INTEGER NOT NULL,
  format TEXT NOT NULL DEFAULT 'aac', sizeBytes INTEGER,
  createdAt INTEGER NOT NULL, trashedAt INTEGER,
  transcript TEXT, transcriptSegmentsJson TEXT);
CREATE TABLE RecordingMarker (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  recordingId INTEGER REFERENCES Recording(id) ON DELETE CASCADE,
  offsetMs INTEGER NOT NULL, label TEXT NOT NULL DEFAULT '');
```

12.3 Migration and integrity policy

- Schema changes ship as numbered Room `Migration`s with unit tests over exported schemas (`core/data/schemas/`), auto-migration only for additive cases.
- Every app start runs `PRAGMA quick_check` in Diagnostics' data (not blocking launch); a failure surfaces the Diagnostics error state with backup guidance — never auto-deletes.
- WAL mode ON; `busy_timeout` 3 s; all writes through the repository queue (§11.4).

12.4 Backup format (`.msone`)

Zip (deflate) of: `manifest.json` (schema version, app version, counts, SHA-256 of each member), `data.json` (all tables), `files/…` (recordings, scans, attachments). Encryption (optional but default): AES-256-GCM; key = PBKDF2-HMAC-SHA256 (210 000 iterations, 16 B salt) from the user's passphrase; passphrase never stored. Restore is transactional: verify all hashes → stage in `files/incoming-restore/` → swap → vacuum. A failed restore leaves the original data untouched (J-03).

12.5 Search indexing pipeline

- On write: Note/Scan/Recording triggers FTS update (Room trigger) + a cheap local embedding of the first 2 000 chars when the Essentials pack is enabled (batched, ≤ 50 ms budget per note, queued).
- Vectors stored in `indexes/vectors.bin` (f32, one row per item, L2-normalized); hub search = FTS top-50 ∪ cosine top-50 (only when pack on), merged with a stable score (FTS rank primary, cosine tiebreak).
- Rebuild commands exist in Diagnostics (FTS rebuild, embeddings rebuild) for support scenarios.

12.6 State restoration contract

Per-screen SavedState keys are registered in a single `RestoreKeys` object; the continue-row reads from a small `LastSession` DataStore record updated debounced 500 ms: last module, last screen, last expression, draft note id, recording id. Calculator additionally persists its live expression+caret+selection on every change (guide-critical: B-journeys survive death). Process-death restoration is tested by TC-STATE-* cases including the J-06 storm.

12.7 Index catalog (all indexes in schema v1)

Table.Index	Columns	Serves
CalcHistory.created	createdAt DESC	history sheet order + LRU eviction
Note.updated	updatedAt DESC	list sort default
Note.trashed	trashedAt	trash sweep
CalEvent.start	startUtc	month window queries
CalEvent.reminder	reminderOffsetMin partial (WHERE rrule NOT NULL or one-off)	next-reminder scan
Track.album / Track.artist	albumId/artist cols	library tabs
PlaylistEntry.pk	(playlistId, position)	queue order
Recording.created	createdAt	list order
RecordingMarker.rec	recordingId, offsetMs	marker list + seek
NoteFts	FTS4(title, plainText)	search

12.8 Query pattern contracts (perf-relevant SQL, all via Room)

- Month window: `SELECT * FROM CalEvent WHERE endUtc >= :winStart AND startUtc <= :winEnd` (index range scan).
- Next reminders: `SELECT ... WHERE startUtc + reminderOffset*60000 BETWEEN now AND now+86400000` — re-armed at boot and after edits.

- History page: `SELECT * FROM CalcHistory ORDER BY createdAt DESC LIMIT 50 OFFSET ?` (pinned first: `UNION` with pinned subset).
- Note list page: title+snippet projection, `LIMIT 100` keyset by updatedAt (`WHERE updatedAt < :cursor`), never OFFSET on large sets.
- FTS search: `SELECT rowid, ... FROM NoteFts WHERE NoteFts MATCH :q` joined to Note; snippet() for highlight.

12.9 Entity field dictionaries (complete executor reference)

`CalcHistory` — `id` rowid PK · `expression` canonical ASCII (§17.1.3) · `result` exact rendered-canonical string (ungrouped) · `createdAt` epoch ms UTC · `pinned` 0/1.

`MemorySlot` — `id` PK · `value` exact numeric string (never float) · `label` optional user label ≤ 40 chars · `updatedAt` epoch ms.

`Note` — `id` PK · `title` derived-but-stored (first heading/line, ≤ 200 stored) · `bodyJson` NoteDoc v1 (§17.3) · `plainText` projection for FTS/snippets (marks stripped, per block) · `createdAt/updatedAt` epoch ms · `trashedAt` null or epoch ms (sweep at +30d) · `pinned` 0/1 · `color` index into 6-swatch palette.

`NoteFts` — content-linked FTS4 mirror of (`title`, `plainText`); rebuilt via `INSERT INTO NoteFts(NoteFts) VALUES('rebuild')` command in Diagnostics.

`ScanDoc` — `id` PK · `pageUrisCsv` ordered page file names (private `Scans/`) · `ocrText` concatenated page text · `createdAt` · `noteId` nullable link (SET NULL on note delete).

`CalEvent` — `id` PK · `title` ≤ 200 · `startUtc/endUtc` epoch ms (all-day: local midnight normalized with `startTzOffsetMin` preserving the creating zone) · `startTzOffsetMin` original offset · `allDay` 0/1 · `rrule` subset string or null · `reminderOffsetMin` −1=none, else 0..1440 · `color` palette index · `note` ≤ 2000 chars · `icsUid` import dedup key · `exDatesCsv` ISO dates.

`Track` — `id` PK · `mediaStoreId` nullable · `folderUri+relativePath` SAF address or MediaStore data · `title/artist/album` tags (fallbacks: filename/`Unknown`) · `durationMs/sizeBytes/trackNo` · `artworkPath` cache file or null · `lastPlayedAt` epoch ms.

`Playlist/PlaylistEntry` — name unique per user view (not enforced in DB); entries ordered by `position` (compacted on reorder); FK cascade on playlist delete.

`Recording` — `id` PK · `filePath` private path · `durationMs` (timeline excludes paused spans) · `format` `aac/wav` · `sizeBytes` · `createdAt` · `trashedAt` · `transcript` full text with provenance header line · `transcriptSegmentsJson` `[{s,e,t}]`.

`RecordingMarker` — `id` PK · `recordingId` FK cascade · `offsetMs` timeline position · `label` ≤ 40 chars.

`diag_ring` — §28.5.

12.10 Backup `data.json` shape (normative)

```
{"format":"msone","schema":1,"appVersion":"3.13.0","exportedAt":1769900000000,
 "tables":{"CalcHistory":[…], "MemorySlot":[…], "Note":[…], "ScanDoc":[…],
           "CalEvent":[…], "Track":[…], "Playlist":[…], "PlaylistEntry":[…],
           "Recording":[…], "RecordingMarker":[…]},
 "files":["recordings/REC-20261027-133601.m4a", "scans/page-0001.jpg"],
 "counts":{"Note":42,"CalEvent":17}}
```

Restore verifies: format, schema ≤ current, member hashes, counts vs `tables` lengths, then transactional swap. Files restore to identical relative paths; collisions impossible (staging swap).

12.11 Query performance validation (fixtures vs budgets)

Query	Fixture scale	Budget	Measured target
History page 50	500 rows	8 ms	index range scan
Note list page 100	20k notes	15 ms	keyset by updatedAt
FTS search `engine`	20k notes	25 ms	FTS4 match + join limit 50
Month window 3 months	3k events	5 ms	start/end index
Library songs page	50k tracks	15 ms	album/artist index
Recording + markers	5k recordings	10 ms	FK index
Hub aggregate first page	all of the above	120 ms	parallel sources, 25 ms caps

Validation runs as a JUnit perf test on the seeded corpora (Appendix K) in the nightly lane; regressions > 2× budget fail.

12.12 DataStore schema (prefs store, normative keys)

```
settings.<registry-key>            # §17.7 + §17.12, typed via registry
session.lastModule / .lastScreen   # continue-row
session.calc.expression/.caret/.sel
stats.moduleOpens.<module>          # counters (clearable)
hub.recentSearches                  # string list, max 10
firstRun.<card-id>                  # shown-once flags
```

One DataStore file (`prefs.preferences_pb`); all access via `SettingsRegistry` and `SessionStore` — feature code never touches DataStore directly (architecture test).

Chapter 13 — Module technical deep dives

13.1 Calculator engine (core/domain/calc)

See §17.1 for the full language specification. Structure:

```
Lexer (tokens, unicode-aware) → Parser (Pratt, precedence climbing)
  → Ast (Num, Bin, Un, Percent, Call, Group)
  → Evaluator (exact first, Decimal fallback)
  → Renderer (exact → display string, grouping, fraction option)
LiveAnswer = Parser+Evaluate on a cancellable job per keystroke
```

The engine is a pure Kotlin class `CalcEngine` with one public method `evaluate(expr: String, cfg: EngineConfig): EvalResult` plus `format(result, cfg): String`. `EngineConfig` carries angle mode, fraction display, locale (for grouping only). **No Android imports**, no globals, no state — history/memory live above it. This is what makes the 10^4-case property tests (§17.1.7) cheap and the Google-parity behaviors (F-001…F-009) provable.

13.2 Notes document model

`NoteDoc` = JSON: `{v:1, blocks:[Block]}`; `Block` = paragraph | heading{level} | bullet{items} | checklist{items:[{text,checked}]} | divider. Inline runs carry `{text, bold?, italic?}`. Rules: renderer and parser share one serialization (round-trip identity test TC-NOTE-rt); plainText projection strips marks for FTS; Markdown export/import maps every block (unknown Markdown constructs import as plain paragraphs with a one-time notice). Autosave writes bodyJson+plainText in one transaction 500 ms after last keystroke; editor state (scroll, cursor) goes to SavedState.

13.3 Scanner pipeline

```
CameraX ImageCapture → YUV→NV21 → DocumentCornerDetector (OpenCV-free:
  Hough-lite edge scan in pure Kotlin on 480p preview frames)
  → perspective warp (RenderScript-free: Android Matrix + Bitmap)
  → filter (Gray / adaptive B&W)
  → page store (JPEG q85, app-private)
OCR: page bitmap → resizer 960px long-edge → ONNX det model (DBNet-lite)
  → boxes → crop+rectify → ONNX rec model (CRNN en) → text lines
  → confidence merge → editable card
```

Per-page OCR budget: ≤ 2.5 s p50 on Tier B, ≤ 6 s Tier C, cancellable; progress on the card; images never leave the process (no share intents until user acts). Camera permission denied → import from Files path keeps the module alive (F-055).

13.4 Calendar recurrence engine

Pure Kotlin `RecurrenceEngine` (§17.4): input `CalEvent` + window → occurrences. Supports RRULE subset `FREQ=DAILY|WEEKLY|MONTHLY|YEARLY`, `INTERVAL`, `BYDAY` (weekly), `BYMONTHDAY` (monthly), `COUNT`, `UNTIL`, plus `EXDATE`. Monthly rules use the "same-day-of-month, skip missing" policy with a documented alternative (last-day clamp) as a setting. Unit-tested against a golden corpus of 200 rules with expected occurrence lists, plus DST-crossing cases (ADR-08 offsets).

13.5 Media: Music + Recorder

Music: Media3 `ExoPlayer` with `DefaultMediaSourceFactory` queueing playlists via ConcatenatingMediaSource2-style factory; artwork via a BitmapFactory-based decoder with a 64 MB LRU disk cache; library scanner walks SAF tree or MediaStore cursor in a cancellable coroutine, diffing into Room (no duplicates by (path,size,mtime)). Recorder: `AudioRecord` (48 kHz mono) → AAC via `MediaCodec` + `MediaMuxer` (WAV path writes PCM directly); pause = stop feeding muxer, resume with new sync sample; markers = DB rows at flag time; waveform = decimated RMS ring buffer drawn by Compose Canvas at 20 fps max. Media session: `MediaSessionService` for Music (headset/bluetooth controls, media notification); Recorder uses a plain FGS notification (pause/resume/stop actions) — it is not a media-session consumer.

13.6 Files & SAF

Two abstractions: `LocalRepo` (app-private: list/copy/move/rename/delete/trash via `java.io`+`AtomicFile`) and `SafRepo` (DocumentFile tree from persisted grants, with a SAF-can't-do-X map: no true rename-move across roots → copy+delete with progress + undo impossible on SAF deletes (documented in UI as "deleted permanently" for SAF)). Open-with hands off `ACTION_VIEW` + `ClipData` with grant flags. The analyzer walks with a 200 ms-yielding coroutine so the UI never janks; results cached with mtime invalidation.

13.7 Hub search federation

`SearchAggregator` fans out to per-module `SearchSource` implementations (NotesFts, Scans, Events, FilesNames, Tracks, Recordings+Transcripts, CalcHistory) with per-source budgets (25 ms first page, cancellable), then groups results by module in fixed order. Semantic search (pack on) runs post-FTS for note sources only (§12.5). Debounce 120 ms; the aggregator is unit-tested with fake sources for ordering, budget and cancellation (TC-HUB-*).

13.8 Notifications and alarms inventory

Channel	Type	When
`reminders`	High-importance, sound+vibration	Calendar alarms (user-created)
`recording`	Low-importance, ongoing FGS	During recording only
`playback`	Media style	Music playback
`diagnostics`	Default, silent	Only on self-test failure the user asked to run

Nothing else ever posts (P4). Exact alarms via `AlarmManager.setExactAndAllowWhileIdle`; `canScheduleExactAlarms()==false` → setInexactWindow fallback + one-time silent notice in Calendar settings (never a blocking dialog).

13.9 Public API surfaces per module (executor contract tables)

Calculator (`core/domain/calc` + `feature/calculator`):

Symbol	Signature	Notes
`CalcEngine.evaluate`	`(expr: String, cfg: EngineConfig): EvalResult`	pure, thread-safe
`CalcEngine.format`	`(r: EvalResult, cfg: EngineConfig): String`	grouping/fraction display
`EvalResult`	`value: ExactValue, exact: Boolean, error: CalcError?`	sealed
`EngineConfig`	`angleMode, fraction, localeTag`	value class

Symbol	Signature	Notes
`ExpressionViewModel`	`onKey/onCaret/selection ops/history/mem`	single source of UI state
`SelectionController`	`enter/resize/commit/cancel`	gesture-driven state machine

Notes:

Symbol	Signature	Notes
`NoteDocCodec.encode/decode`	`(NoteDoc): String` / `(String): NoteDoc`	round-trip identity
`NoteRepo.upsert/get/trash/restore/purgeExpired`	coroutines	Room txn + FTS
`NoteSearch.query`	`(q: String, budget): List<Hit>`	FTS + snippets

Scanner:

Symbol	Signature	Notes
`CaptureController.start/torch/focus`	CameraX wiring	permission-aware
`DocDetector.corners`	`(Bitmap): Quad?`	pure Kotlin
`OcrRunner.recognize`	`(Bitmap, budget): OcrPayload`	orchestrator client
`ScanRepo.savePage/exportPdf`	page store + PDF mux	app-private

Calendar:

Symbol	Signature	Notes
`RecurrenceEngine.occurrences`	`(CalEvent, Window): List<Instant>`	pure
`CalRepo.upsert/delete/window`	CRUD	index-backed
`ReminderScheduler.arm/cancel/rearmAll`	AlarmManager	boot-aware

Files:

Symbol	Signature	Notes
`LocalRepo.list/copy/move/rename/trash`	path ops	queue writes
`SafRepo.*`	DocumentFile mirrors	can't-do map honored
`Analyzer.walk/cancel`	sizes by category	yielding
`BackupManager.create/restore`	`.msone`	transactional

Music:

Symbol	Signature	Notes
`LibraryImporter.scan`	SAF/MediaStore diff	cancellable
`PlaybackController.play/pause/seek/speed/queue`	Media3	session-backed
`PlaylistRepo`	CRUD + M3U export	—

Recorder:

Symbol	Signature	Notes
`RecorderService.startPause/stop/flag`	FGS intents	≤ 200 ms start
`RecordingRepo`	rows + markers	—
`Transcriber.run`	pack client	segments out

Diagnostics:

Symbol	Signature	Notes
`SelfTest.run`	6 checks → report	§17.8
`LogRing.append/scrub/export`	rolling store	§28.3

Symbol	Signature	Notes
`PackRegistry`	import/enable/disable/verify	§16.8

13.10 Recorder end-to-end sequence (numbered, for the executor)

1. User taps record → `RecorderController.start()` requests mic permission if needed (§14.7 copy) → `startForegroundService` (≤ 200 ms, type `microphone`).
2. Service acquires `AudioRecord` (48 kHz mono), starts encoder thread, opens `MediaMuxer` on the final file path, inserts `Recording` row (state=recording).
3. Waveform: ring buffer of decimated RMS pushed via SharedFlow; Compose Canvas renders ≤ 20 fps.
4. Pause: stop feeding muxer; freeze presentation time; update notification (Resume/Stop); state=paused.
5. Flag: validate not paused → insert `RecordingMarker(offsetMs=timeline)` → haptic.
6. Done: stop/muxer stop/finalize → row state=saved, durationMs=timeline → notification dismissed.
7. Call interruption: audio focus loss → auto-pause (§7.14) with notification note.
8. Storage threshold: pre-write check each 5 s chunk → below floor → graceful finalize, row state=incomplete, user card (not toast-from-worker — the historical crash class stays dead).
9. Process death of service (OEM kill): file finalized best-effort on `onTaskRemoved`/`onDestroy`; row honest about interruption.

13.11 Scanner end-to-end sequence (numbered)

1. Module opens → CameraX preview binds (permission flow §14.7); guides overlay.
2. Shutter → `ImageCapture.takePicture` → freeze overlay ≤ 250 ms.
3. Corner detection on 480p frame → quad found → perspective warp to page.
4. Filter applied (user default) → JPEG q85 → `Scans/page-<n>.jpg` → row update.
5. Review opens (auto); OCR auto-runs when Essentials pack enabled (§16.6 tier rules).
6. OCR: bitmap normalized (≤ 960 px) → det → rec per box → merge → card editable; conf badges.
7. User edits text; actions Copy/→Notes/Re-OCR/Delete per §7.4.
8. Export: PDF mux (pages in order) or image copy or text → destination (Files/Notes/clipboard).
9. Batch chip counts pages; duplicate-detection warns (§7.14); page manager reorders/deletes.

Chapter 14 — Security and privacy engineering

14.1 Threat model (STRIDE-lite, offline)

Threat	Vector	Mitigation
Data theft from device	Lost phone, backup file	App-private storage; optional app lock; encrypted `.msone` (PBKDF2 210k, AES-GCM)
Malicious file import	Crafted image/audio/ICS/pack	Decode via platform codecs only; no native parsers beyond vetted runtimes; pack SHA-256 + zip-bomb guards (max ratio 100×, max entries 10k); ICS parser with depth/size caps
Intent abuse	Another app invokes components	All activities `exported=false` except launcher; receivers not exported (except media-button receiver pattern per API level); `review` grants not persisted
Clipboard leakage	Sensitive calc values on clipboard	Clipboard auto-clears 60 s after calculator copy (setting, default on)
Model-pack tampering	Swapped pack file	SHA-256 manifest verified at import AND at load; mismatch → pack disabled + Diagnostics card
SQLite injection	Search strings	Room bound parameters only; FTS query sanitized (quotes escaped)
Path traversal in backup	Crafted `.msone` entries	Entry names validated (`../` rejected, only `data.json` + whitelisted dirs extracted)

14.2 Privacy guarantees (structural)

The merged manifest contains **no** `INTERNET`, `ACCESS_NETWORK_STATE`, `READ_CONTACTS`, `LOCATION*`, `READ_PHONE_STATE`, `BLUETOOTH_*` (classic), `QUERY_ALL_PACKAGES`, or ad-class permissions (C1). No WebView, no JavaScript engine, no `URL`/`Socket`/`HttpURLConnection` classes in release bytecode — CI scans with `apkanalyzer` + a dex-name grep (§24.4). Unique identifiers: none generated; `Settings.Secure.ANDROID_ID` never read. Crash logs live only in the local ring; `Share log` shows the exact content in a preview sheet before any system share target appears.

14.3 Data lifecycle

Create → live tables/files; Delete → trash (30 d) → hard delete with file shred (overwrite + delete for private files; SAF deletes are OS-level); Backup → user-initiated only; Restore → transactional (§12.4). Telemetry: **there is none** — the only counters are in the local diagnostics ring (§28.3), PII-scrubbed (paths reduced to basenames, expressions kept verbatim because they are the product's content — the log preview makes this visible before sharing).

14.4 Permissions matrix (full app)

Permission	Type	Why	Request moment
`CAMERA`	Dangerous	Scanner viewfinder	First scanner open
`RECORD_AUDIO`	Dangerous	Recorder	First record tap
`READ_MEDIA_AUDIO` (API 33+) / `READ_EXTERNAL_STORAGE` (26–32, maxSdk 32)	Dangerous	Music import	First music open
`POST_NOTIFICATIONS` (API 33+)	Dangerous	Reminders, recording, playback	First reminder creation
`SCHEDULE_EXACT_ALARM`	Normal (26–32) / declared with user grant path 33+	Reminder precision	First reminder (settings fallback per §13.8)
`FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_MICROPHONE`, `FOREGROUND_SERVICE_MEDIA_PLAYBACK`	Normal	Recorder / Music FGS types	—
`RECEIVE_BOOT_COMPLETED`	Normal	Re-arm reminder alarms	—
`VIBRATE`	Normal	Haptics	—

`mycalc` flavor: **none of the above** — the flavor's manifest is empty of permissions; calculator needs nothing (F-038). The one historical wart — an auto-generated `DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION`-class custom permission — is explicitly checked in CI and must not appear (the owner's v3.12.39 QA flagged it; §24.4 gate `no-custom-permissions`).

14.5 Data-in-transit and crypto inventory

There is no transit. Crypto inventory: Java `Cipher` `AES/GCM/NoPadding`, `SecretKeyFactory` `PBKDF2WithHmacSHA256`, `KeyStore` for the app-lock integrity attestation blob only (never for data keys — those derive from the passphrase so backups are portable). No third-party crypto libraries (smaller APK, smaller audit).

14.6 Security review checklist (per release, automated + human-walkable)

#	Check	Method
SC-01	Merged manifest permission set == allow-list (full + flavor separately)	CI manifest-audit diff
SC-02	Exported components == launcher activities only	manifest parse test
SC-03	No network-class symbols in dex (URL/Socket/okhttp/WebView/JS engines)	apkanalyzer + grep gate
SC-04	Backup zip guards reject traversal/bomb fixtures	L2 tests on fixture set
SC-05	Wrong-passphrase restore fails with no partial state	L4 J-03 negative case
SC-06	Pack hash mismatch disables pack and surfaces card	L2/L4 PACK cases
SC-07	Clipboard auto-clear works and is discoverable (settings row)	L4
SC-08	Log ring scrubber passes PII fixture battery	L1
SC-09	App-lock locks after background threshold; credential fallback works	L4 (biometric emulators)
SC-10	No custom permissions declared (esp. the historical `*_NOT_EXPORTED_PERMISSION` pattern)	manifest-audit hard gate
SC-11	Debuggable=false, cleartextTrafficPermitted=false, backup agent absent or agentless-allowlist	build output scan
SC-12	Mapping.txt + APK SHA-256 archived in evidence bundle	release lane artifact check

14.7 Permission request copy (exact strings, normative)

Permission	Dialog title	Dialog body
CAMERA	Allow camera?	Scanner uses the camera to capture pages. Images stay on this phone. You can also import images instead.
RECORD_AUDIO	Allow microphone?	Recorder stores audio on this phone only. Nothing is uploaded — MySpace One has no internet permission.
READ_MEDIA_AUDIO	Allow music access?	Music lists audio files to build your library. Files are read, never changed or uploaded.
POST_NOTIFICATIONS	Allow notifications?	Reminders, recordings and playback need notifications. No marketing, ever.

Denied-state copy (in-feature cards) states the manual alternative (§7.11) and never nags — one card, dismissable, persistent entry in Settings.

14.8 Data classification register

Data	Class	Storage	Leaves device?
Note bodies/titles	Sensitive	Room (private)	only via explicit share/export
Calculator history	Sensitive	Room	only via explicit share/export
Recordings audio	Sensitive	private files	never auto
Transcripts	Sensitive	Room	only via share/export
Scan images + OCR	Sensitive	private files/Room	only via share/export
Events	Sensitive	Room	only via ICS export
Music library metadata	Personal	Room	never
Settings/preferences	Personal	DataStore	never
Backups (.msone)	Sensitive (encrypted)	user-chosen location	user-controlled
Diagnostics ring	Operational (scrubbed)	private files	only via user share
AI pack files	Public asset	private files	never
Crash breadcrumbs	Operational (scrubbed)	private files	only via user share

14.9 Android component inventory (merged manifest, normative)

Component	Flavor	Exported	Purpose
`MainActivity`	both	yes (launcher)	single-activity host
`RecorderService` (FGS)	full	no	recording
`PlaybackService` (MediaSession)	full	no	music
`ReminderReceiver`	full	no	alarm firings
`BootReceiver`	full	no	re-arm alarms
`MediaButtonReceiver`	full	no	headset controls
`ShareLogActivity`	full	no	share-sheet source
`CalcShortcutActivity`	both	no	app shortcuts trampoline

No content providers; no broadcast senders to third parties; every receiver `exported=false` with explicit intent actions internal to the app (media-button receiver pattern per API level documented in code).

14.10 Local incident response runbook (worst-case playbook)

Incident	Immediate steps	Recovery
DB corruption detected	app switches read-only mode; banner; backup prompt	restore latest `.msone`; FTS rebuild; defect S1
Recording file damaged	row marked; player shows reason; raw file kept	partial playback attempt (WAV header resync tool in Diagnostics v1.1)
Pack loads but produces garbage	disable pack automatically; provenance row flags model hash pair	re-import correct version; eval re-run
Restore half-applied (crash)	staging swap design prevents; if bootstrapped mid-restore, startup detects `incoming-restore/` and rolls back	re-run restore
App-lock locks out (no biometrics enrolled)	device-credential fallback per BiometricPrompt contract	OS-level recovery only (by design; no backdoor)

14.11 In-app privacy statement (exact text, About → Privacy)

> **MySpace One is offline by design.**
> This app has no internet permission. It cannot connect to anything, so nothing you write, scan, record, calculate or play can be transmitted — by the app or by anyone using it.
> Your notes, scans, recordings, events, files and history live in the app's private storage on this phone. Backups are files you create and keep (encrypted with your passphrase).
> Optional AI features run entirely on this phone using model packs you install yourself.
> The app collects nothing, phones nothing home, and has no accounts. Diagnostics logs stay on the phone until you choose to share them, and you see exactly what will be shared.
> Delete the app and the private data goes with it — export a backup first if you want to keep anything.

This statement is asserted in tests (strings key `privacy.statement.body`) so it can never silently drift from the app's actual behavior — the statement is a spec artifact as much as a legal one.

14.12 Crypto test vectors (fixed answers the tests pin)

The crypto suite pins known-answer tests: PBKDF2-HMAC-SHA256("passphrase", salt=0x00..0f, 210000, 32B) → recorded hex; AES-256-GCM round-trip with AAD=`msone-v1`; tampered-ciphertext must fail GCM tag verification. Vectors live in `core/data/test/crypto-vectors.json` (generated once, reviewed) — upgrades to iteration counts are ledger decisions (they break old backups only if the *reading* app changes; the format records its own parameters so old backups stay readable).

PART V

METHOD AND INTELLIGENCE

How an autonomous AI agent runs this program without drift (Chapter 15) and how on-device AI is integrated into the product without violating its promises (Chapter 16). The two chapters share a philosophy: capability must be provable, bounded, and reversible.

Chapter 15 — Agent methodology: building an app without a human in the loop

15.1 The mandate

The agent's mandate is: *produce the release defined by S1–S8 (front matter) by executing phases G0–G8, spending at most 30 CI jobs per full pipeline run and 3 parallel runners (C8), and leaving behind evidence, not assertions.* The agent may not: weaken a constraint; ship with a red P0; invent scope; or claim success without an artifact path a human can open.

15.2 Operating loop

Every unit of work — from "add a token" to "integrate OCR" — runs the same micro-cycle:

1. **Read** the binding sections (this document) for the task.
2. **Plan** the change: files, tests, acceptance mapping (task graph entry, §15.3).
3. **Implement** with tests written against acceptance criteria first where practical (the agent writes its own tests; it is both author and reviewer, so the tests are the reviewer).
4. **Verify locally**: compile, unit tests, lint; only then commit.
5. **Evidence**: test output paths, screenshots (for UI), updated traceability entries.
6. **Re-plan**: if acceptance cannot be met, the task splits or a defect/decision is filed — never silently skipped.

15.3 Task graph

The program is a DAG of tasks. Each task record contains: `id`, `phase` (G0–G8), `title`, `binding-spec` (§ refs), `acceptance` (TC/UX/F ids), `depends-on`, `estimate` (agent-hours proxy), `artifacts`. The graph lives at `docs/plan/task-graph.yaml` and is the agent's single source of scheduling truth; a task without binding-spec refs or acceptance ids is invalid. Sample slice:

```
- id: T-CALC-014
  phase: G3
  title: Percent semantics (context-sensitive)
  binding-spec: [§9.2 F-003, §17.1.5]
  acceptance: [TC-ONE-C-004, TC-CALC-law-percent]
  depends-on: [T-CALC-003]
  artifacts: [core/domain/calc/Percent.kt, core/domain/test/calc/PercentTest.kt]
```

15.4 Context strategy (how the agent stays on-spec)

- **Load-on-phase**: at phase start the agent re-reads the phase's chapters; it does not rely on memory across long gaps.
- **Spec-to-code links**: every source file header carries `Spec: §…` comments; the traceability script (§27) flags code touched without spec refs during review.
- **Decision ledger**: `docs/plan/decisions.md` records every choice not dictated by this document (D-nnn: date, question, options, decision, consequence). The ledger is append-only; revisiting a decision requires a new entry, never an edit.
- **No silent context loss**: if a session boundary interrupts work, the agent resumes from the task graph + ledger, never from recollection.

15.5 Phase gates

Agent phase-gate lifecycle — a gate opens only when its exit criteria are demonstrated by committed artifacts (§15).

Iteration inside a phase follows: plan → implement → test → review → re-plan, with the task graph (§15.3) as the unit of work.

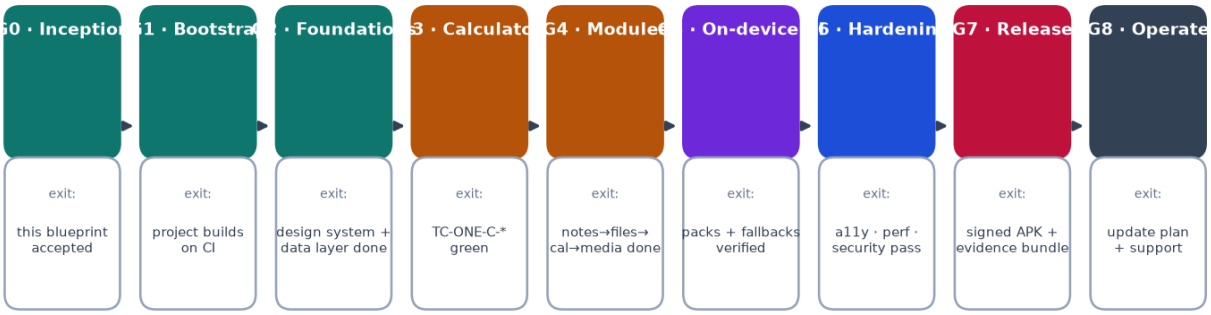


Figure 15. *Nine gates from inception to operation; each opens only on demonstrated exit criteria.*

Gate	Exit criteria (all machine-checkable unless noted)
G0 Inception	This blueprint accepted; task graph G1 slice populated
G1 Bootstrap	`app` builds; both flavors assemble; empty hub renders on emulator; CI green end-to-end
G2 Foundations	Design-system components + tokens done with screenshots; Room v1 + migrations tested; CI architecture tests active
G3 Calculator	Catalogue-0 CALC set green (incl. F-001…F-035 P0s); flavor parity J-09 green; property laws (§17.1.7) green
G4 Modules	Notes, Scanner, Calendar, Files, Music, Recorder, Hub P0 sets green; journeys J-01…J-10 green (AI-dependent steps mocked)
G5 On-device AI	Pack manager F-082; OCR/transcribe/summarize/semantic-search acceptance green on Tier A/B emulators; fallback ladder (§16.7) proven — features green with packs absent
G6 Hardening	A11y audit (TC-A11Y-*) green; perf budgets (§23.6) met on Tier B/C profiles; security checklist §14 verified; string/copy audit UX-015 green
G7 Release	S1–S8 all true; release APK + evidence bundle + regenerated docs published
G8 Operate	v1.1 backlog filed; defect ledger triaged; update-runbook (§29) committed

A gate closes with a **gate report**: one page per gate listing criteria, artifact paths, and open defects, committed under `reports/gates/Gn.md`. The owner can stop the program at any gate; the graph's later tasks are all downstream of closed gates.

15.6 Handling blocked work

Blocked ≠ failed. The agent records: what was attempted (commands, inputs), the observable outcome, why fair assertion was impossible, and the unblock condition. Blocked items never masquerade as passes; they appear in gate reports as `BLOCKED` with owners (agent-fixable vs owner-input-needed). Example class: a gesture the emulator input API cannot reproduce (the owner's past QA showed `adb input swipe` cannot do a stationary hold-then-drag); the honest state is BLOCKED with a device-test recommendation, not a FAIL invention or a silent skip.

15.7 Decision ledger discipline

Entries are cheap and frequent. A change is ledger-worthy when: the document is silent; two spec sections tension; a dependency forces a deviation; or a budget cannot be met. Each entry names the binding sections it *could* have affected. The G7 review walks the ledger top-to-bottom — this is how the owner audits an autonomous build.

15.8 Field lessons baked into this methodology

The owner's previous QA campaigns surfaced failure modes this methodology explicitly prevents: (1) harness navigation drift — probes wandering to launchers and other modules — now prevented by module-scoped sessions and state-reset steps (§22.3); (2) toast crashes on background instrumentation threads — now a harness rule: UI assertions only from the instrumented main path, service work posted to the app's own handlers (§22.4); (3) publisher-success confusion — a green publish job is not a passing result, so gate reports and the PDF report always carry per-case results, never job status alone (§22.6); (4) cross-release guide mismatch — every run pins APK SHAs and guide hash (§22.2, C10).

15.9 Gate report template (normative shape)

```
# Gate G3 report — Calculator
Date / agent session ids: …
## Criteria table
| Criterion | Status | Artifact |
| TC-ONE-C-001..026 | 26/26 pass | reports/evidence/g3/raw/*/user-journey-results.json |
| Law suite L1–L10 | pass (12,000 cases/law, seed 3311) | reports/evidence/g3/laws.json |
| Flavor parity J-09 | pass | … |
## Blocked register
| TC-ONE-C-B2 hold-then-drag | BLOCKED | emulator input cannot reproduce stationary hold; need
    s physical device |
## Defects opened/closed
D-014 trailing-op repeat chain (closed), …
## Decision ledger entries this phase
D-009 …
## Owner actions requested
None / device session request …
```

15.10 Worked micro-task (illustrative agent trace)

Task T-CALC-014 (percent semantics): read §9.2 F-003 + §17.1.5 → write table-driven tests for the seven normative rows + 200 randomized law-L4 cases (seed pinned) → implement `PercentNode` fold in evaluator → run module tests (fail: `200+10%+10%` row) → fix accumulation semantics (percent of accumulated left value) → re-run (green) → detekt/kfmt → commit `feat(calc): context-sensitive percent semantics; Refs: F-003` → update task graph + traceability row → screenshot unaffected (no UI delta). Total: one iteration; evidence = test output file path recorded in task entry.

15.11 Agent tool inventory and use policies

The executor is an AI agent with a fixed toolkit; misuse of tools is a defect class, so use policies are normative:

Tool	Use for	Policy
Local shell (build/test/git)	compile, run tests, commits	never push to protected branches; never force-push; commits carry Refs
Emulator (managed)	L4/L5 runs	3-runner cap; snapshot reset per suite; airplane-mode toggles scripted

Tool	Use for	Policy
File editor	code/docs edits	edits only with a task id; no edits to `qa/testcases.csv` by hand (generated)
Web/docs access	dependency docs, API references	read-only; no downloads outside pinned dependency managers
Issue/ledger files	defects, decisions	append-only patterns (§15.7, §20.9)
Report builder	gate reports, PDF of record	only from manifest-verified evidence

Guardrails: no secrets in the repo (keystore via CI secret store only); no network egress attempts from app code ever; no dependency added without ledger + audit pass; no CI job count growth beyond caps (C8); any tool failure on a critical step (install, build, harness) is recorded with raw output in the evidence tree — silent retries without evidence are forbidden (the owner's honesty standard).

15.12 Anti-drift reviews (weekly cadence)

Once per agent-week (or 40 tasks, whichever first): re-read §1.2–§1.4 (promises/constraints), diff `task-graph.yaml` against the actual commit log (detect tasks done without graph entries), sample three merged PRs and re-review against their binding sections, and re-run the full fast lane from clean. Drift findings go to the ledger with corrective tasks. The G-gates remain the hard checkpoints; anti-drift reviews are the soft Cadence that keeps gates boring.

15.13 Context-recovery worked example (session resume)

Session S-019 ends mid-G4 after `fix(calendar): DST clamp` compiles but tests hang. Next session S-020: (1) reads task graph (T-CAL-014 in-progress, blocked-on flaky timer test), ledger (D-0031 fake-clock injection for alarm tests), last session log; (2) runs module tests with a 120 s timeout — reproduces the hang with stack: `TestDispatcher` idle-wait; (3) applies ledger decision D-0031 (inject `TestClock`), test green, commit, task closed; (4) logs session and proceeds to next task. Nothing was remembered — everything was read. If the recovery had failed twice, protocol says: quarantine the flaky test (§20.10), open defect with stack, continue with the next independent task.

Chapter 16 — On-device AI strategy

16.1 Principles

1. **AI is optional by architecture**: every AI affordance has a manual path of equal correctness (slower or less convenient); absence of packs changes speed, not capability (X7).
2. **Nothing downloads**: packs are side-loaded files imported by the user, SHA-256-verified (C5).
3. **Deterministic degradation**: each capability has an explicit fallback ladder (§16.7); the app never guesses whether AI ran — provenance is recorded with every AI output (`model`, `pack version`, `seed`, `timestamps`).
4. **Bounded resources**: one model hot at a time (except embedder, small); hard memory caps per tier; eviction on background; thermal/battery gates.

16.2 Capability × module map

Capability	Used by	Manual equivalent	Pack	Priority
OCR (det+rec, Latin)	Scanner, hub search over scans	Type text manually / import text files	Essentials	P0 (F-057)
Image labels	Files analyzer auto-albums (v1.1), scanner suggestions	Manual naming	Essentials	P2
Embeddings (MiniLM-L6 int8)	Semantic note search, "find related"	FTS search	Essentials	P1 (F-042, F-054)
ASR (whisper-tiny/base int8)	Recorder transcribe; voice search (v1.1)	Manual notes	Speech	P1 (F-076)
Small LLM (Gemma-2B-it, 4-bit, MediaPipe)	Summarize, rewrite, checklist-ify, smart event parse ("lunch with Ravi tomorrow 1 pm" → fields)	Manual editing	Assistant	P1 (F-054)
TTS	Read note aloud, read transcript	Reading	System TTS (no pack)	P2

16.3 Interaction contract for AI features

Every AI action follows one UX contract: (1) entry is a clearly-marked secondary action (✦ prefix, `AI` tag); (2) a visible running state with cancel; (3) results appear as a **preview** (diff or draft card) — the user applies, edits, or discards; (4) undo survives apply (trash-backed original block); (5) if the pack is missing the row shows the install path ("Diagnostics → Model packs"), never a dead button; (6) outputs carry a provenance footnote (model + pack version) in the note's front-matter (hidden by default, visible in export info).

16.4 The AI orchestrator (core/ai)

Single entry: `AiOrchestrator.run(capability, input, budget): AiResult`. Responsibilities: capability gate (pack present? device tier OK? battery/thermal OK? concurrency slot free?); runtime selection; budget enforcement (wall-clock, memory, tokens); cancellation; provenance capture; result validation (schema checks on LLM JSON, confidence floors on OCR); fallback triggering. The orchestrator is feature-agnostic and unit-tested with fake runtimes — features never touch a runtime directly (ADR leaf rule).

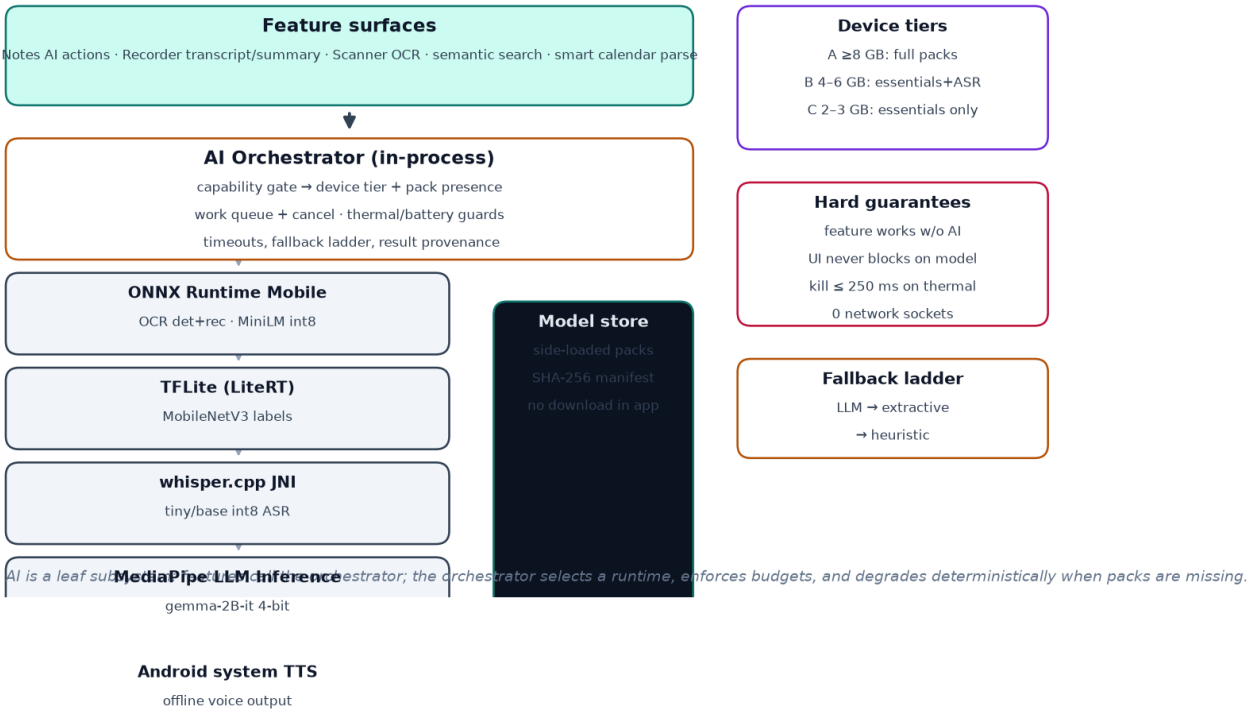


Figure 16. *AI stack: features → orchestrator → runtimes; model store side-loaded; device tiers and guarantees.*

16.5 Models and runtimes (pinned catalogue)

Pack	Contents	Runtime	Size (approx)	Notes
Essentials	DBNet-lite det + CRNN rec (OCR, Latin); MobileNetV3 labels; MiniLM-L6 int8 embeddings	ONNX Runtime Mobile (NNAPI/CPU)	18–25 MB	Tier C usable
Speech	whisper-tiny int8 (multi-core) with optional whisper-base int8 on Tier A	whisper.cpp JNI	45–80 MB	1× realtime on Tier B
Assistant	gemma-2B-it 4-bit (or Qwen2.5-1.5B-it 4-bit alternative, one pack format)	MediaPipe LLM Inference	1.3–1.7 GB	Tier A only; warm-idle ≤ 40 s then evict

Pack format: zip with `pack.json` (id, version, models[], sha256 per file, min-tier, license text), models under `models/`. Import: hash-verify → atomic move into `files/packs/<id>/<version>/` → registry row → enabled state. Pinned model versions are recorded in `core/ai/catalog.csv`; the pack manager displays them (§7.9).

16.6 Resource governance

Tier	Max concurrent jobs	LLM context	Eviction	Battery gate
A	2 (1 LLM)	2 048 tok	LRU 40 s idle	none beyond Doze
B	2 (no LLM)	—	LRU 20 s idle	defer > 30 s jobs on saver
C	1	—	immediate on background	defer all AI on saver except on-demand OCR tap

Memory caps: embedder 80 MB, ASR 220 MB, OCR 150 MB, LLM 1.6 GB (A-only). Runtime `Debug.MemoryInfo` sampling asserts caps in instrumented tests (TC-AI-perf-*).

16.7 Fallback ladder (per capability)

- **OCR**: det+rec → rec-only line strips (det fail) → "crop manually" hint → manual text (module stays usable: F-059).
- **Semantic search**: cosine → FTS-only (silent downgrade, provenance row in diagnostics).
- **Summarize/rewrite (LLM)**: LLM → extractive (TextRank-ish, embeddings) → heuristic (first sentences/checklist regex) with a visible "basic result" tag.
- **Event parse**: LLM → regex/date-parser → manual editor pre-filled from text.
- **Transcribe**: whisper → (nothing; manual notes) — transcription is all-or-nothing but never blocks recording.

Every fallback is a tested path (TC-AI-fallback-*), not an aspiration.

16.8 Pack security and evaluation

Security: SHA-256 from `pack.json` re-hashed at every load; signature pinning optional (pack may carry an ed25519 signature verified against a key baked into the app — recommended for owner-distributed packs); zip guards (§14.1). Evaluation: before a pack version is marked *supported*, it must pass the offline eval kit (`tools/ai-eval/`): OCR ≥ 92% char-accuracy on the 40-image fixture set; ASR ≤ 12% WER on 20 fixture clips; LLM summarization ≥ 4/5 rubric on 20 fixture notes (rubric in-repo); embedding retrieval recall@10 ≥ 0.8 on the fixture corpus. Results are committed as JSON evidence — the same artifact style the owner's QA reports already use.

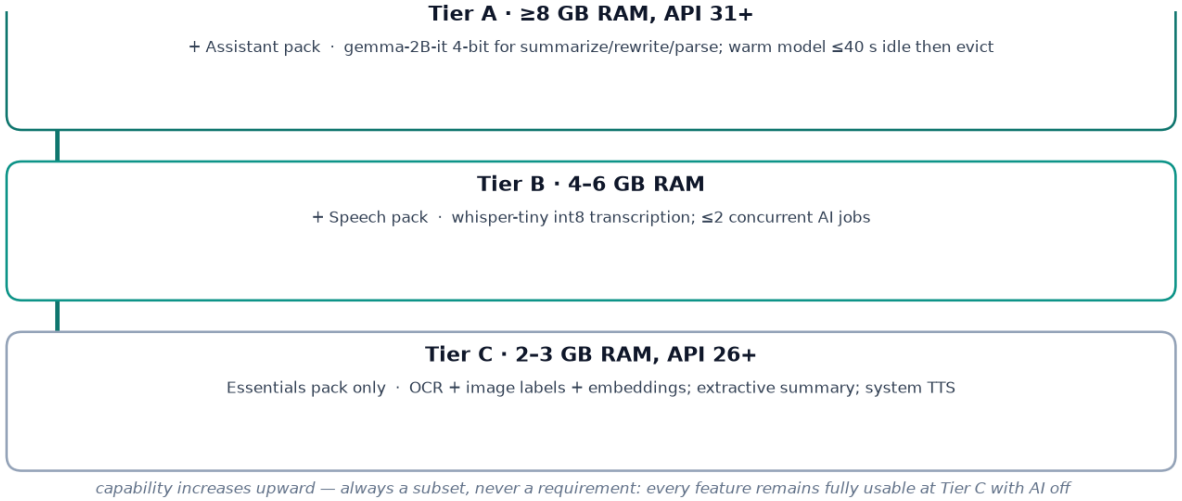


Figure 17. *Device capability ladder: always a subset, never a requirement.*

16.9 What on-device AI will not do here

No generation of note content unprompted; no auto-sending, auto-filing, or auto-deleting; no model-initiated notifications; no personalization profiles; no fine-tuning on device (v1); no voice printing or speaker ID; no cloud fallback "just this once". These are promise-level negatives (P1/P2) and are enforced by review checklist §18.4.

16.10 Orchestrator API reference (core/ai)

Signature	Purpose
`run(cap: Capability, input: AiInput, budget: Budget): AiResult`	single entry; never throws (Result)
`Availability.check(cap): Avail(packInstalled, tierOk, thermalOk, batteryOk, slotFree)`	UI gate for enabling rows + reason strings
`warm(cap, ttl)` / `evict(cap)`	Tier A LLM lifecycle
`register(packs: List<PackMeta>)`	called by pack manager after import/enable/disable
`provenance(result): Provenance`	model id, pack id/version, timings, fallback rung
Budget	wallMs, memCapMb, maxConcurrent, cancelToken
`AiResult` (sealed)	`Ok(payload, prov)` · `Degraded(payload, rung, prov)` · `Unavailable(reason)` · `Cancelled`

Feature modules depend only on this interface + `Availability` for UI state — fake implementations in `core/testing` make features fully testable without packs (G4 runs this way by design).

16.11 Per-capability integration specs (concise)

- **OCR**: input = page bitmap (≤ 960 px long edge normalized) → det boxes (conf ≥ 0.4) → rec per box (conf ≥ 0.55 floor, else line marked low-confidence) → reading-order merge (column-aware simple: sort by y then x with 0.6 line-height clustering) → `OcrPayload(lines, meanConf)`. UI: editable card; provenance footnote.
- **Embeddings**: input = note plainText first 2 000 chars → MiniLM int8 (seq 128, mean-pool, L2 norm) → f32[384] stored. Batched 16/batch on idle coroutine; rebuild tool in Diagnostics.
- **ASR**: input = 16 kHz mono PCM stream → whisper-tiny int8, language hint `en` (user-settable later), vad-trimmed segments; output segments `{startMs,endMs,text}` + mean log-prob; monotonic timestamps asserted.
- **LLM (Assistant pack)**: tasks = summarize (map-reduce over 1 500-token chunks, 5 bullets, JSON schema), rewrite (tone presets: shorter/formal/bullets), checklist-ify (extract actionable items JSON), event-parse (fields JSON). All prompts stored in `core/ai/prompts.yaml` with versioning; temperature 0.2; max output 512 tokens; JSON schema validated, retry once on schema fail, then fallback rung.
- **TTS**: system engine only; queue per note reading; speed follows music speed? no — separate setting default 1.0; graceful no-engine state (card, not error).

16.12 `core/ai/prompts.yaml` (initial content, normative)

```
version: 1
defaults: {temperature: 0.2, max_output_tokens: 512, lang: en}
summarize:
  system: >
    You summarize personal notes for their author. Output 3-5 short bullets,
    each <= 15 words, preserving names, numbers and dates verbatim.
    Never invent facts. Output JSON: {"bullets": ["..."]}.
  user: "Summarize this note:
---
{body}"
rewrite:
  system: >
    You rewrite the user's note text in the requested style. Keep meaning,
    names and numbers. Do not add new facts. Keep language = input language.
    Output JSON: {"text": "..."}.
  user: "Style: {style}
Text:
---
{text}"
checklistify:
  system: >
    Extract actionable checklist items from the text. Keep the original
    wording where possible; one item per action; <= 20 items.
    Output JSON: {"items": [{"text": "...", "done": false}]}.
  user: "Text:
---
{body}"
event_parse:
  system: >
    Extract one calendar event from the text. The reference time is
    {now_local} ({tz}). Resolve relative dates ("tomorrow", "next Friday").
    If a field is unknown, omit it. Output JSON:
    {"title": "...", "start": "YYYY-MM-DDTHH:MM", "duration_min": n,
     "all_day": bool, "repeat": null}.
  user: "Text: {text}"
json_retry: 1      # on schema failure, re-ask once with the error, then fall back
```

Prompt changes are versioned with eval re-runs (§16.8); a prompt change without a green eval is revertible debt.

16.13 Capability × tier × power behavior matrix (normative)

Capability	Tier C	Tier B	Tier A	Battery saver (any tier)
OCR scan tap	allowed, ≤ 6 s budget	≤ 2.5 s	≤ 2 s	allowed (user-initiated)
OCR auto-batch	deferred to charge	allowed	allowed	deferred with chip
Embeddings write-time	off (batch on idle)	batched	batched	off until charge
Semantic search	off silently	on	on + LLM rerank v1.1	off silently (provenance row)
ASR transcribe tap	allowed ≤ 2× RT	≤ 1× RT	≤ 1× RT	allowed (user-initiated)
LLM summarize/rewrite/parse	unavailable row	unavailable row	allowed, warm ≤ 40 s	one-shot allowed, warm disabled
TTS read aloud	system engine	system	system	allowed
Warm eviction	immediate on background	20 s idle	40 s idle	immediate

The matrix is encoded in `core/ai/TierPolicy.kt` as data (not scattered ifs) and unit-tested as a table — the orchestrator's `Availability.check` reads exactly this policy.

16.14 Pack distribution README (ships beside packs)

```
MySpace One — AI model packs (side-loaded, offline)
1. Download the .mspack file (this repo's releases) — check the SHA-256 beside it.
2. In the app: Diagnostics → Model packs → Import → pick the file.
3. Status shows "installed · SHA ✓". Enable per pack. Delete anytime.
Requirements per pack: Essentials any device; Speech 4 GB+ RAM; Assistant 8 GB+ RAM.
Never download packs from other sources: the app verifies hashes, and signed packs verify sign
    atures too.
```

16.15 Sample AI outputs (calibration fixtures, abbreviated)

Capability	Input	Output (verified shape)
OCR	fixture `inv-007.png`	`{"lines": [{"t": "INVOICE #118", "conf": 0.97}, {"t": "2 x USB-C cable 1,350", "conf": 0.93}], "meanConf": 0.94}`
ASR	fixture `clip-011.wav`	segments `[{s:0,e:4120,t:"please review the invoice"}, …]`, lang `en`
Summarize	fixture note `field-survey`	`{"bullets": ["Survey covered plots 12-18", "Soil samples due Friday", "Budget approved 45k"]}`
Checklist-ify	groceries paragraph	`{"items": [{"text": "Milk", "done": false}, …]}`
Event parse	"lunch with Ravi tomorrow 1 pm" (now=27 Oct 13:36 IST)	`{"title": "Lunch with Ravi", "start": "2026-10-28T13:00", "duration_min": 60, "all_day": false, "repeat": null}`

Every sample is also a fixed eval case — drift in output shape/quality fails the eval kit, not a user's afternoon.

16.16 Pack-of-record provenance block (About → packs detail)

```
essentials 1.2.0 · installed 2026-10-28 · sha256 5c1f…
  models: det.onnx (DBNet-lite) · rec.onnx (CRNN en) · minilm.int8.onnx
  eval: char-acc 94.1% (threshold 92) · retrieval recall@10 0.86 (0.8) · PASS
speech 1.1.0 · eval WER 9.8% (12) · PASS
assistant 1.0.2 · eval rubric 4.4/5 (4) · schema discipline 100% · PASS
```

16.17 Summarize flow — numbered orchestrator sequence

1. User taps `Summarize` on note → feature calls `Availability.check(SUMMARIZE)`.
2. Gate result drives UI: enabled / reason ("Assistant pack needs an 8 GB device").
3. Enabled → `run(SUMMARIZE, {body}, budget(wall=20s, mem=1.6GB))`; UI shows running + cancel.
4. Orchestrator: tier/battery/concurrency checks → load LLM (warm or cold, evict timer reset).
5. Chunk body ≤ 1 500 tokens → per-chunk summarize → reduce to 5 bullets (2 LLM calls max).
6. Validate JSON schema; on fail retry once with error; on second fail → extractive fallback (rung 2, visible tag).
7. Provenance captured (model, pack, rungs, ms, tokens) → payload to feature.
8. UI renders preview card (Apply / Edit / Discard); Apply writes block + provenance footnote + undo anchor.
9. Every step's outcome appended to the diag ring AI record (§28.2).

PART VI

DETAILED TECHNICAL SPECIFICATION

The single source of truth for behavior-level engineering: the calculator language with its exact grammar and laws, history and memory contracts, the note document format, recurrence rules, the recorder service, file handling, and every settings key. When Chapter 9 says what, this chapter says exactly how it must behave.

Chapter 17 — Detailed specifications

17.1 The calculator language

17.1.1 Lexical structure

Characters: digits `0-9`, `.` (decimal point; locale comma input is accepted and normalized), operators `+ − × ÷ ^`, brackets `( )`, `%`, constants `π e`, function names `sin cos tan ln log` and inverse/`√`/`!` via pad, and `⌫ AC` as commands (not tokens). Unicode minus `−` and multiplication sign `×` are first-class; ASCII `- * /` are accepted on paste and normalized. Numbers: one dot max; digits beyond 15 significant are preserved in token form but evaluated in Decimal context (§17.1.2). Max expression length 512 chars (enforced at input).

17.1.2 Number model — exactness ladder

1. **Rational path** (default for `+ − × ÷` and integer `^`): every operand that originates from a literal, or from `+ − × ÷` over exact values, is a `BigRational` (numerator/denominator as BigInteger). `0.1+0.2` → `3/10` → displays `0.3`. `1÷3×3` → `1` exactly (the owner's documented `0.9999999999` bug class cannot exist on this path).
2. **Transcendental path** (`sin cos tan ln log √ non-integer-^ π e`): operands convert to `BigDecimal(34 significant digits, HALF_EVEN)`; results carry an `inexact` flag. `sin(π)` is handled by identity reduction (`π` argument → 0 exactly) so the classic `3e-17` artifact cannot appear.
3. **Rendering**: exact values render from the rational (up to 15 decimal places, trailing zeros trimmed; denominator powers of 2·5 only — otherwise longest-repeating detection shows up to 10 places with ellipsis option). Inexact values render to 10 significant digits by default (`show as fraction` renders rationals as `a/b`, F-030). The famous failure `0.9999999999` is structurally impossible because display never re-parses a rounded string; it renders the underlying exact/inexact value.
4. **Overflow/domain**: `÷0` → `Error` card "Cannot divide by zero"; `ln(−1)` etc. → `Error` card with the domain reason; factorial of non-neg integers > 169 → `Error` (result beyond display); exponent overflow → scientific rendering `1.234567891e+15` *with the exact value retained internally* so continued computation does not lose precision (the display-string-reuse bug of `1e+15+1 → Error` is prohibited: continuation always uses the internal exact value, F-008 note).

17.1.3 Canonical input form

The stored/compared form of any expression (history, clipboard-on-copy, logs) is canonical ASCII: `2+3*(4+1)`. The UI renders pretty glyphs (× − ÷ π) from tokens; copy/history/export use canonical form with one exception — copy of a *number* copies the display digits without grouping (F-007/F-014).

17.1.4 Grammar and parsing (EBNF)

```
expr    := term { ("+" | "-") term };
term    := power { ("*" | "/") power };
power   := unary [ "^" power ];            (* right associative *)
unary   := "-" unary | postfix;
postfix := primary [ "!" | "%" ];          (* postfix % §17.1.5 *)
primary := NUMBER | CONST | FUNC "(" expr ")" | "(" expr ")" | primary "(" expr ")";  (* impli
    cit × *)
```

- **Implicit multiplication** (F-002) arises when a `primary` is followed by `(`, a constant, or a function: `2(5+2)`, `(2)(3)`, `2π`. The parser inserts a `MUL` node with the same precedence as `×` (left-assoc): `2(5+2)=28`, `1/2π` = `(1/2)π` = `0.5π` — matching Google Calculator's documented behavior.
- **Percent as postfix** (F-003): parsed as `Percent(expr)`; semantics in §17.1.5.
- **Auto-balance** (F-006): the `=` key appends missing `)` before evaluation; display shows the balance preview via live answer; the `( )` smart key inserts `()` pair when the parser expects a primary, else inserts `)`.
- **Operator replacement** (guide C4/F-005): typing an operator when the caret follows an operator replaces it (except unary `−` after `(`, `×`, or start: `10×−` legal); typing `+` right after `−`-as-unary is a replace.
- **Errors** surface as typed results: `SyntaxErr` (with caret position hint in the error card), `DomainErr`, `DivZero`, `TooLong`, `TooBig`.

17.1.5 Operator semantics — normative table

Input	Result	Rule
`200+10%`	`220`	percent = 10% of 200 (context: additive)
`200−10%`	`180`	subtract 10% of 200
`200×10%`	`20`	percent = plain ×0.1
`200÷10%`	`2000`	divide by 0.1
`50%`	`0.5`	standalone: value/100
`200+10%+10%`	`242`	each percent of the accumulated left value
`5+` then `=`	`10`	**trailing-operator rule**: equals with a dangling operator repeats the left operand as the right (`5+5`) — Google parity F-004
`5+` then `=` then `=`	`15`	repeat semantics chain (each equals adds 5 again)
`10×−`	`-10` (on `=`)	unary minus folds into power/term per EBNF
`2^3^2`	`512`	right-associative power
`(2+3 =`	`25`	auto-balance

17.1.6 Functions and constants

`sin cos tan` + INV (`asin acos atan`), `ln log` (natural, base-10), `√`, `^`, `!` (integer factorial, `0! = 1`), constants `π`, `e`. Angle mode: degree default with a `DEG/RAD` toggle on the scientific pad (persisted; INV sin in degree mode returns degrees — the owner's guide D6 case `Inv sin 0.5 → 30°`). Accuracy contract: transcendental results within 1 ulp of 34-digit computation; live-answer latency budget ≤ 1 ms for ≤ 64-char expressions on Tier B (measured by `CalcEngineBenchmark` in CI, §23.6).

17.1.7 The laws (property-based test suite)

The engine must satisfy all laws below over randomized expression corpora (≥ 10 000 cases per law in CI; seeds recorded in test output):

Law	Statement
L1 Round-trip	`format(evaluate(s))` re-parses and re-evaluates to the same value for all renderable outputs
L2 Exact identity	`(a+b)−b = a` exactly on the rational path (no epsilon)
L3 Distribute-display	`0.1+0.2` renders `0.3`; `1/3*3` renders `1` (rendering never invents `0.999…`)

Law	Statement
L4 Percent consistency	`x+p% ≡ x+(x*p/100)`; `x*p% ≡ x*p/100` for all sampled x,p
L5 Implicit equivalence	`2(5+2) ≡ 2*(5+2)` AST-equal after implicit-mul insertion
L6 Balance completeness	any prefix-valid expression with open brackets evaluates identically to its balanced form
L7 Replace determinism	the operator-replacement state machine is a pure function of (current token, typed token)
L8 Continuation fidelity	after `=`, operating on the displayed result equals operating on the internal exact value
L9 Locale independence	evaluation is identical across en-IN/en-US/de-DE inputs modulo punctuation normalization
L10 History fidelity	stored canonical expression re-evaluates to the stored exact result

17.1.8 Rendering and grouping

Grouping: Indian style `1,23,456` for `en-IN`, western `123,456` otherwise; applied to the integer part only; never inside fraction display; never in canonical form. Decimal separator follows locale for display; input accepts both. Very large/small magnitudes switch to scientific display at ≥ 1e15 or < 1e−9 (display-only; §17.1.2 continuation rule).

17.2 History and memory contracts

- **History row** written on every successful `=` (and on `Error` — errors are history too, Google parity), fields per §12.2; write is synchronous with result display (≤ 16 ms, same frame).
- **Replay**: card tap → expression painted into the field (with caret at end) → sheet closes after paint (≥ 1 frame later) → double-tap race guard (debounce 350 ms) (F-022/F-023).
- **Persistence across restart** (F-024): tested by TC-ONE-C-013 — insert 5 entries, kill process (`adb shell am kill` + force-stop), relaunch, assert all 5 + exact order.
- **Memory slots**: slot 1 is the MAdd target; overflow menu `Memory` opens the sheet; `M+`/`M−`-style accumulation is *not* a separate key (not in the pad) — accumulation happens by repeated MAdd into the same slot? No: MAdd **adds** the selected value into slot 1 (guide semantics: `MAdd stored 123456`); `MC` clears the selected slot; up/down moves reorder (F-026). Values stored as exact strings; MUse inserts/replaces verbatim.

17.3 Notes document format (`NoteDoc`)

```
{"v":1,
 "blocks":[
  {"t":"h","lvl":1,"runs":[{"x":"Field survey"}]},
  {"t":"p","runs":[{"x":"Plain "},{"x":"bold","b":true},{"x":" text"}]},
  {"t":"check","items":[{"x":"Milk","c":true},{"x":"Rice","c":true},{"x":"Turmeric","c":false}
    ]},
  {"t":"bullet","items":[{"x":"First"}]},
  {"t":"div"}]}
```

Rules: unknown block types on import are preserved verbatim under `{"t":"raw"}` and rendered as plain text (forward compatibility); export to Markdown maps h→`#`, p→text, check→`- [x]`, bullet→`-`, div→`---`, bold `**`, italic `*`; import reverse-maps (lossless round-trip property TC-NOTE-rt over the fixture corpus). `plainText` projection concatenates run text with `\n` per block. Title = first h/first line, max 80 display chars.

17.4 Recurrence (supported RRULE subset)

```
FREQ = DAILY | WEEKLY | MONTHLY | YEARLY
INTERVAL = 1..999          BYDAY = MO,TU,WE,TH,FR,SA,SU (WEEKLY; multi-value allowed)
BYMONTHDAY = 1..31,-1 (MONTHLY)   COUNT = 1..9999   UNTIL = date
EXDATE = comma list of dates (stored separately)
```

- Weekly with BYDAY: occurrences on listed weekdays at INTERVAL-week steps anchored on `startUtc` local date.
- Monthly BYMONTHDAY: day exists in month else skipped (setting: `clamp to last day` alternative); `-1` = last day.
- Yearly: same month/day; Feb 29 skipped in non-leap years (setting: `Mar 1 alternative`).
- Expansion: lazily generated per visible window (month grid) + next-occurrence index for reminders; generation is pure and unit-golden-tested (§13.4).
- ICS mapping: exported RRULE strings limited to this subset; import of unsupported RRULE (e.g., BYSETPOS) → single occurrence on DTSTART + `note` suffix "recurrence simplified from ICS" (F-063).

17.5 Recorder service

- Lifecycle: `startForeground` within 200 ms of record tap (API 26+ `startForegroundService` contract); type `microphone`; notification actions Pause/Resume/Stop; process-death of the *activity* never stops recording; device reboot stops it (documented; no auto-resume).
- Audio: 48 kHz mono PCM 16-bit → AAC-LC 96 kbps (`MediaCodec`+`MediaMuxer`, `.m4a`); WAV alternative 48 kHz 16-bit PCM. Files: `Recordings/REC-yyyyMMdd-HHmmss.ext` + DB row.
- Pause: encoder stops, muxer kept open, `PresentationTimeUs` frozen (resumed timeline continuous); markers during pause are rejected.
- Marker: DB insert with `offsetMs = currentTimelinePosition`; max 200/recording; labels ≤ 40 chars.
- Failure: storage-full mid-record → finalize file gracefully, mark row `incomplete`, user toast with path (no data loss of already-written frames); mic loss (unplug) → auto-pause with notification update.
- Transcription (pack): segments = `{startMs,endMs,text}` list stored in `transcriptSegmentsJson`; searchable via `plainText` mirror; provenance block (model id, pack version, language guess, WER n/a) stored in DB `transcript` header line (§16.3).

17.6 Files handling rules

- SAF duplicate-name policy: numeric suffix `name (2).ext`; private-repo same. Case-insensitive collision check on FAT-style roots.
- Move across SAF roots = copy+delete with progress; cancellation leaves source intact (copy first, delete after verify).
- Trash: private files move to `.trash/` with restore manifest; SAF deletes are immediate (OS limitation, disclosed in UI confirm text, §13.6).
- Analyzer cache: sizes cached per directory with mtime/children-count invalidation; deep walks yield every 200 ms (jank budget §23.6).
- Backup/restore per §12.4/§14.1 (zip guards, transactional restore).

17.7 Settings key registry (normative)

Key	Type	Default	Values / notes
`calc.haptics`	bool	true	key + selection haptics
`calc.fraction`	bool	false	show rationals as a/b
`calc.keyTheme`	enum	`mySpaceSettings`	mySpaceSettings, graphite, ocean, sunset, mint, rose, lemon, iris
`calc.keyShape`	enum	`roundedSquare`	circle, squircle, pill, roundedSquare, none
`calc.effects`	enum	`none`	none, bubbles, ripple, confetti
`calc.angleMode`	enum	`deg`	deg, rad
`calc.clipboardClearSec`	int	60	0 disables
`app.theme`	enum	`system`	system, light, dark
`app.dynamicColor`	bool	true (A12+)	Material You
`app.lockEnabled`	bool	false	biometric+credential
`cal.defaultReminderMin`	int	15	−1 = none
`cal.exactAlarmFallbackNoticed`	bool	false	one-time notice flag
`rec.format`	enum	`aac`	aac, wav
`music.speed`	float	1.0	0.5..3.0
`files.sort`	enum	`name`	name, date, size, type
`ai.packs.<id>.enabled`	bool	true (when installed)	registry mirror
`hub.firstRunCardsDone`	bool	false	privacy card once

Registry is a single Kotlin object with typed accessors; Settings UI is generated from it in tests (a missing row = TC-SET-* failure), preventing key drift.

17.8 Diagnostics self-test script (normative steps)

1. Engine: 100 seeded law-smoke cases (L1–L5) → pass/fail counts.
2. DB: `PRAGMA quick_check` + insert/read of one row per table in a transaction (rolled back).
3. Packs: re-hash each installed pack file vs registry.
4. Alarms: `AlarmManager.canScheduleExactAlarms()` report.
5. Notifications: channel existence + importance.
6. Storage: free bytes; warn < 500 MB.

Output: JSON + rendered card; nothing leaves the device unless the user shares it (F-080/F-081).

17.9 Fraction rendering rule (exact path)

When `calc.fraction` is ON: rationals with denominator ≤ 999 render as `a/b` (mixed form `3 1/2` when |a|>b, with thin-space grouping); denominators > 999 fall back to decimal (10 sig digits); inexact-path values never render as fractions (no fake precision). The `=` result and live answer both honor the setting; history stores canonical form regardless (rendering setting is display-only).

17.10 Worked evaluation traces (normative examples)

Traces for 30 expressions — including every row of §17.1.5 and the owner's historical failure cases (`5+ =`, `1÷3×3`, `1e15+1`, `0.1+0.2`) — are tabulated in **Appendix B** with token stream, path (exact/inexact), internal value, and rendered output. The law-suite corpus generator must include every Appendix B row as a fixed prefix so regressions on historical bugs are impossible to miss.

17.11 Notification text templates (normative)

Notification	Title	Body	Actions
Reminder	`<event title>`	`<time> · <note-or-empty>`	Open
Recording	`Recording — MySpace One`	`<duration> · tap to return`	Pause / Stop
Paused recording	`Recording paused`	`<duration> so far`	Resume / Stop
Playback (media style)	`<title>`	`<artist> · <album>`	Prev / Play-pause / Next
Self-test failure (user-ran)	`Self-test found a problem`	`<n> of 6 checks failed`	Open Diagnostics

Wording rules: no exclamation marks, no emoji, no "please"; numbers humanized (`1 hr 12 min`).

17.12 Per-module settings keys (registry extension)

Module	Key	Type	Default
Notes	`notes.sort`	enum	edited
Notes	`notes.view`	enum	list
Notes	`notes.confirmTrash`	bool	false
Scanner	`scan.defaultFilter`	enum	original
Scanner	`scan.autoOcr`	bool	true (pack on)
Calendar	`cal.weekStart`	enum	locale
Calendar	`cal.monthlyOverflow`	enum	skip
Calendar	`cal.sound`	enum	system
Files	`files.showHidden`	bool	false
Files	`files.sortDir`	enum	asc
Music	`music.repeat`	enum	off
Music	`music.shuffle`	bool	false
Music	`music.crossfadeSec`	int	0
Recorder	`rec.markerHaptic`	bool	true
Recorder	`rec.keepScreenOn`	bool	true
Diagnostics	`diag.ringDays`	int	7
Hub	`hub.searchRecents`	bool	true
Global	`app.pureBlackDark`	bool	false (A10+)

Rules: keys are never read from raw strings in feature code (registry accessors only — TC-SET-registry reflects); removals migrate per §29.6.

17.13 Worked parse trees (executor calibration)

`2(5+2)` →

```
BIN(+, BIN(×, NUM(2), GROUP(BIN(+, NUM(5), NUM(2))))   # implicit × inserted, left-assoc
```

`200+10%+10%` →

```
BIN(+, BIN(+, NUM(200), PCT(NUM(10), ctx=ADD, lhs=200)),
       PCT(NUM(10), ctx=ADD, lhs=220))                 # lhs accumulates → 242
```

`10×−3^2` →

```
BIN(×, NUM(10), NEG(BIN(^, NUM(3), NUM(2))))           # unary binds under ^: −9 ×10 → −90
```

Property test oracles compare against these trees structurally (AST equality after implicit-mul insertion, law L5).

17.14 Rendering edge table (display rules digest)

Case	Rule	Output
15+ digits integer	group by locale, no exponent	`12,34,56,78,90,12,345`
≥ 1e15 magnitude	sci display, exact kept	`1.000000001e+15`
≤ 1e−9 magnitude	sci display	`3.333333333e−10`
repeating decimal	10 places then … (fraction setting offers `a/b`)	`0.3333333333…`
exact half	shortest form	`2.5`
negative zero	normalized	`0`
`+0.5` leading ops	leading `+` dropped, `−` kept	`−0.5`

17.15 Additional worked cases (Appendix B extension, B-31…B-45)

#	Input	Path	Rendered
B-31	`0.5+0.25`	exact	`0.75`
B-32	`1÷3+1÷6`	exact	`0.5`
B-33	`(1÷3)×3+0.2`	exact	`1.2`
B-34	`2^0.5`	inexact	`1.414213562`
B-35	`8^1÷3`	exact-fold	`2` (`8^(1/3)` rational exactness note: cube-root identity table)
B-36	`10!÷8!`	exact	`90`
B-37	`√(−4)`	domain	error card
B-38	`log(0)`	domain	error card
B-39	`e^0`	exact	`1`
B-40	`π×2`	inexact	`6.283185307`
B-41	`sin(sin(30°))` DEG	identity+inexact	`0.5` (inner exact-by-table, outer identity)
B-42	`100−10%−10%`	exact	`81`
B-43	`50+50%`	exact	`75`
B-44	`−5+10`	exact	`5`
B-45	`1.5!`	domain	error card (integer factorial only)

PART VII

ENGINEERING STANDARDS AND THE AGENT LIFECYCLE

The rules the code itself must follow (Chapter 18) and the full G0–G8 lifecycle the agent executes (Chapter 19), including per-phase task slices, review checklists, and definitions of done. Together they make the build auditable end to end.

Chapter 18 — Engineering standards

18.1 Language and style

- **Kotlin 2.x**, JVM target 17, Compose compiler aligned; coroutines + Flow for asynchrony; no `GlobalScope`; explicit `Dispatchers` injection (no hardcoded `Dispatchers.IO` in classes — injected `DispatcherProvider` for testability).
- Formatting: ktfmt (standard style), enforced by spotless; detekt default suite + custom rules: `NoHardcodedColor` (use tokens), `NoDirectToastFromWorker`, `NoNetworkApis` (blocks `java.net`, `okhttp`, `WebView` symbols), `SpecHeaderPresent` (file header carries `Spec: §…`).
- Commits: conventional prefixes (`feat(calc):`, `fix(calendar):`, `test:`, `docs:`, `chore:`); every commit message ends with `Refs: F-xxx…` where applicable — the traceability script consumes this (§27).

18.2 Definition of done (per task)

1. Implementation + tests committed; `Refs:` present.
2. Local: compile, module tests, detekt/lint clean, `git diff --check` clean.
3. New UI → screenshot tests added/updated; new schema → migration test; new engine behavior → property or table test.
4. Task graph entry updated (status, artifacts); acceptance IDs linked.
5. No new `TODO` without a linked task id.

18.3 Branching and review

Trunk-based with short-lived feature branches (`t-<task-id>`), squash-merged by the agent after self-review. Self-review is a distinct act: the agent re-reads the diff against the binding spec sections and records a one-line review note in the merge commit (`Reviewed against §17.1.5: percent table rows 1–7 covered`). CI protects `main`: no direct pushes, all checks required (§24.3).

18.4 Review checklists (condensed)

Privacy checklist (every PR): no new permission; no network-class API; no new identifier; no logging of content beyond PII-scrubbed ring; new user data classified (§14.3). **AI checklist (AI-touching PRs):** fallback path exists + tested; provenance captured; budgets enforced; cancel works; pack-absent UX correct. **Calc checklist (engine PRs):** laws re-run; §17.1.5 table rows touched by the change re-tested; canonical-form tests updated. **UX checklist (UI PRs):** token-only colors; touch targets ≥ 48 dp; state triple (empty/error/content) present; TalkBack labels; dark mode screenshot.

18.5 Documentation standards

Code documents *how*, this blueprint documents *what/why*: source comments stay local (no essay comments), `docs/` mirrors decisions via the ledger, and every module ships a `README.md` ≤ 40 lines (purpose, spec refs, test entry points). The release regenerates: this blueprint PDF (from `docs/blueprint/`), the traceability matrix, the license list, and the OpenAPI-less API surface summary (`tools/api-surface.py` output) — all archived in the release evidence (S8).

18.6 Annotated review example (quality bar for self-reviews)

Diff under review (task T-CALC-014, percent accumulation fix):

```
-    val left = eval(node.left)
-    val pct = eval(node.right) / Decimal(100)
-    return when (node.parent) {         // BUG: used syntax parent, not semantic lhs
+    val left = eval(node.left)          // §17.1.5 rows 1-3,6: context = left value
+    val pct = eval(node.right) * left / Decimal(100)  // additive rows: % of accumulated left
+    return when (node.kind) {           // ADD vs MUL path per normative table
```

Review note (merge commit): `Reviewed against §17.1.5: all 7 table rows re-tested incl. row 6 chains; L4 law corpus re-run seed 3311 green; found and fixed wrong-parent bug via row 6 (200+10%+10%=242). Refs: F-003`. The bar: the reviewer-voice note cites the exact spec rows touched and names the test that caught the bug — a reader can audit without re-deriving.

Chapter 19 — The agent lifecycle G0–G8 in full

19.1 Phase table

Phase	Goal	Key tasks (slice)	Gate	Budget (agent iterations)
G0	Alignment	Blueprint acceptance; task graph seed	G0 report	1–2
G1	Skeleton	Gradle workspace, flavors, CI scaffold, empty hub, smoke UI test	Builds on CI, both APKs assemble	6–10
G2	Foundations	Design system (tokens→components), Room v1, repositories, arch tests, RestoreKeys	G2 report; screenshot suite green	15–25
G3	Calculator	Engine (laws) → UI (pads/caret/selection) → history/memory/themes → flavor parity; Catalogue-0 calc green	G3 report	25–40
G4	Modules	Notes → Scanner → Calendar → Files → Music → Recorder → Hub; P0 sets + journeys (AI mocked)	G4 report	60–90
G5	AI	Pack manager, OCR, ASR, embeddings, LLM features, fallbacks, eval kit	G5 report; fallback ladder proven	25–40
G6	Hardening	A11y, perf budgets, security checklist, copy audit, battery/thermal tests, i18n externalization	G6 report	15–25
G7	Release	Versioning, signing, shrink, release QA (black-box harness), evidence bundle, docs regen, distribution builds	S1–S8	8–12
G8	Operate	Backlog, defect triage, update runbook, post-release monitoring-by-diagnostics	G8 report	3–5

Budgets are iteration guides, not deadlines; gates, not calendars, control progression.

19.2 Per-phase verification depth

- G1–G2: JVM tests + Robolectric + first emulator smoke (one runner).
- G3–G4: full pyramid per §20.2; emulator matrix API 26/33/35 sharded across the 3 runners; black-box harness (§22) starts at G3 (calculator) because it is the owner's proof instrument.
- G5: adds device-tier profiles (2 GB / 4 GB / 8 GB emulator skins) and pack fixtures.
- G6–G7: release-type builds only; black-box harness runs against the *unsigned-then-signed release* APK with the same evidence style as the owner's prior campaigns (results JSON + screenshots + manifest, §22.6).

19.3 Metrics that steer the agent

Tracked per phase, committed as `reports/metrics.json`: task burn (done/total per phase), defect discovery rate (§20.9), test flake rate (< 1% hard limit — a flaky test is quarantined within one day), build time p50 (< 6 min), APK size trend, coverage on core modules (≥ 85% line on `core/domain`, §20.8), and law-suite runtime (< 90 s CI budget). Metrics feed the gate reports; the agent re-plans any metric trending red.

19.4 Interruption and resumption protocol

If a session ends mid-phase: the next session (a) reads task graph + ledger + last gate report, (b) runs the phase's fast verification (compile + module tests) to re-earn trust in the working tree, (c) resumes the first open task in dependency order. No work is redone on guesswork; no state is trusted without verification — the same evidence discipline the owner's QA harnesses already follow.

19.5 Owner interaction points

Only four moments require the owner: blueprint acceptance (G0), any P2 deferral or cutline decision (§10.3), provision of anything genuinely unobtainable (e.g., a physical device for B2-class gestures), and release acceptance (G7). Everything else is recorded, not asked.

19.6 Phase task slices (representative task graph rows)

G1 (skeleton, 8 tasks): T-BOOT-001 workspace+wrappers; T-BOOT-002 flavors assemble; T-BOOT-003 CI fast lane; T-BOOT-004 empty hub on emulator; T-BOOT-005 provenance BuildConfig; T-BOOT-006 keystore ceremony + owner backup rehearsal; T-BOOT-007 screenshot harness bootstrap; T-BOOT-008 G1 gate report.

G2 (foundations, 22 tasks): tokens table+tests; theme light/dark; AppTopBar/BottomSheet/Fab/EmptyState/ErrorState/SearchField/ConfirmSheet screenshots; Room v1 schema + export; DAOs + repo tests; SAF repo + fakes; DataStore settings registry; RestoreKeys; arch tests active; hub shell + continue-row plumbing; G2 report.

G3 (calculator, 34 tasks): tokenizer; parser; exact evaluator; Decimal path; renderer+grouping; percent; implicit-mul; trailing-op; balance; errors; law suite; keypad UI basic; sci pad; ExpressionField caret; live answer; selection detector+pill; B-actions (cut/copy/paste/MAdd/MUse); empty-caret menu; history sheet+persistence+replay+race guard; memory sheet; themes×shapes; effects; haptics; overflow exactness; flavor swap+lint; mycalc zero-permission gate; L5 calculator campaign; perf budgets; a11y pass; G3 report.

G4 (modules, 58 tasks): notes (12), scanner (9), calendar (9), files (9), music (7), recorder (7), hub (5) — each task carries its §9 feature IDs; journeys J-01…J-10 with AI mocked; G4 report.

G5 (AI, 26 tasks): pack format+manager; registry+hash gates; orchestrator+budgets; ONNX bindings; det/rec OCR pipeline; embeddings+semantic merge; whisper JNI; MediaPipe LLM + prompts.yaml; per-feature integration (notes actions, recorder transcribe, scanner OCR, hub semantic, calendar parse); fallback ladder tests; eval kit runs; tier profiles; G5 report.

G6 (hardening, 18 tasks): a11y audit battery; TalkBack journeys; contrast tests; font-scale sweeps; Macrobenchmarks; memory caps; battery/Doze tests; security checklist SC-01..12; string audits; dynamic color; predictive back; tablet layouts; locale sweeps; flake quarantine sweep; G6 report.

G7 (release, 10 tasks): versioning; R8 tuning; reproducible double-build; release L5 campaign on artifacts; docs regen (this blueprint, traceability, licenses, api-surface); evidence bundle assembly; RELEASE-NOTES; distribution staging; sign-off pack for owner; publish.

G8 (operate, 4 tasks): defect triage; backlog v1.1; runbook; G8 report.

19.7 Filled example — G1 gate report (target quality bar)

```
# Gate G1 report — Bootstrap
Date: (execution date) · Sessions: S-001…S-006
## Criteria
| Criterion | Status | Artifact |
| Workspace builds (both flavors, debug) | PASS | ci-run-1207/log.txt |
| Empty hub renders on E33B emulator | PASS | evidence/g1/hub-cold-start.png |
| CI fast lane green (build+lint+unit) | PASS | ci-run-1208/summary.json |
| Keystore ceremony + owner backup rehearsal | PASS | evidence/g1/keystore-rehearsal.md |
| Screenshot harness bootstrap (hub golden) | PASS | shots/hub-golden.png |
## Defects
D-001 (S3) flavor res merge order → fixed, verified (ci-run-1209)
## Ledger
D-0001 Koin over Hilt (ADR-04 confirmed at build time; build 38% faster)
## Owner actions
None. (Keystore backup download receipt requested at G2.)
```

Quality bar: every row has an artifact path that opens; every open item has an owner and a next check date; nothing is asserted without a path.

19.8 metrics.json example (per-phase artifact)

```
{"phase": "G3", "generated": "2026-11-14T18:02Z",
 "tasks": {"done": 31, "open": 3, "blocked": 1},
 "tests": {"total": 712, "flake_rate": 0.004, "quarantined": 1},
 "build": {"p50_min": 4.6, "p95_min": 7.1},
 "apk_mb": {"full": 21.4, "mycalc": 5.1},
 "coverage": {"core.domain": 0.91, "core.data": 0.84, "features": 0.63},
 "laws": {"cases_per_law": 12000, "runtime_s": 71},
 "defects": {"open_s1": 0, "open_s2": 0, "opened_this_phase": 14, "fixed": 14},
 "ci": {"jobs_last_full_lane": 26, "runner_cap_ok": true}}
```

19.9 Phase-level definitions of ready and done

Phase	Ready when	Done when
G1	blueprint accepted; graph seeded	gate report committed
G2	G1 closed; tokens table drafted	design-system screenshots + Room v1 tests green
G3	G2 closed	Catalogue-0 CALC + laws + flavor parity green on CI
G4	G3 closed	module P0 sets + journeys green (AI mocked)
G5	G4 closed	packs + fallback ladder proven on tier profiles
G6	G5 closed	a11y/perf/security/copy audits green on release-type builds
G7	G6 closed	S1–S8 artifacts published; owner walkthrough accepted
G8	G7 published	backlog + runbook + triaged ledger committed

PART VIII

QUALITY: STRATEGY, CATALOGUE, ENVIRONMENTS, HARNESS

How the product is proven: the test pyramid and CI pipeline, the complete test-case catalogue bound to feature IDs, test data and emulator environments, and the black-box harness contract that produces the owner-style evidence bundles. Quality here is measured in opened artifacts, not in assertions made.

Chapter 20 — Testing strategy and catalogue

20.1 Philosophy

Tests exist to produce **evidence for the owner**, not comfort for the agent. Three consequences: (1) every test writes an artifact (results JSON, screenshot, or log) into the evidence tree; (2) the black-box layer runs against *distributed APKs*, mirroring how the owner actually receives software — a green source CI says nothing about a released APK; (3) honesty classes are explicit — PASS, FAIL, and **BLOCKED** (environment prevented a fair assertion; never counted as pass, never inferred as a product defect, §15.6).

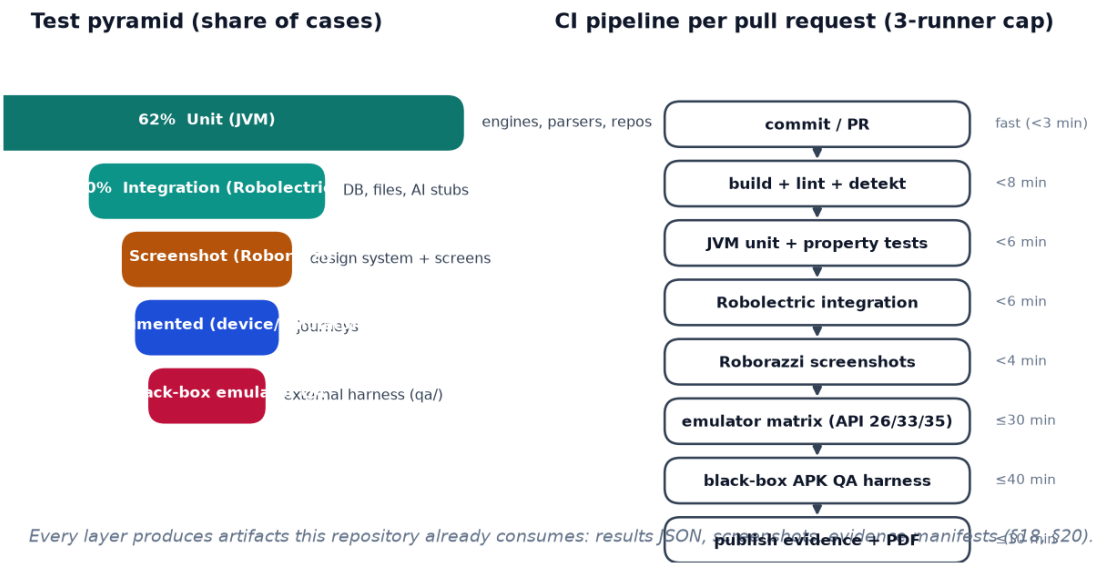


Figure 18. *Test pyramid and per-PR CI pipeline with budgets.*

20.2 The five layers

Layer	Tech	Scope	Budget
L1 Unit (JVM)	JUnit4 + Truth + Koroutines-test	engines, models, repos with fakes; property laws	< 3 min
L2 Integration	Robolectric	Room/FTS round-trips, SAF fakes, orchestrator with fake runtimes	< 6 min
L3 Screenshot	Roborazzi	design-system components + module screens × (light/dark × font 1.0/2.0)	< 4 min
L4 Instrumented	Espresso/UIAutomator on Gradle-managed emulators	journeys J-01…J-10, TC-A11Y, TC-PERF	≤ 30 min sharded
L5 Black-box APK QA	External Python harness (`qa/` pattern: OCR screenshot reader, deterministic taps, results JSON + screenshots)	distributed-APK behaviors: Catalogue-0, TC-ONE-*, parity, provenance	≤ 40 min sharded

L5 is deliberately external to the app build — it tests what shipped, in the style of the owner's existing campaigns (pinned APK SHAs, guide hashes, run IDs, evidence manifests).

20.3 Architecture and lint tests

Beyond behavior: Konsist architecture rules (§11.1 arrows), detekt custom rules (§18.1), manifest assertion tests (permission deny-list, C1), dependency deny-list audit (ad/tracking SDK prefixes), and the string audit (UX-015 network-vocabulary ban). These run as fast L1-class tests so violations fail in under a minute.

20.4 Screenshot discipline

Every design-system component and every module screen renders in a `ScreenshotSuite` activity; Roborazzi records golden PNGs at 1080×1920; PR diffs produce side-by-side evidence images; intentional changes re-record with a ledger note. Calculator key themes × key shapes generate a matrix of key-region shots consumed by the theme-diff check (UX-009: ≥ 10% pixel-diff on theme switch).

20.5 Naming and binding

`TC-<AREA>-<nnn>`: areas CALC, ONE (guide-parity set), HUB, NOTE, SCAN, CAL, FILE, MUSIC, REC, DIAG, SET, SEC, PERF, A11Y, STATE, AI, PACK, NAV, SYS, LAW (property laws). Every TC lists `binding: F-xxx/UX-xxx` and every F/UX lists its TCs — bidirectional, enforced by the traceability script (§27).

20.6 Catalogue-0 (release-blocking set)

The minimum demonstration a release build must pass (L4/L5): the full calculator parity set below plus one core scenario per other module (the module smoke set). This is the direct descendant of the owner's guide-driven cases; numbering intentionally echoes them.

Case	Verifies	Binding
TC-ONE-C-001	`2+3×4 = 14`; `(2+3)×4 = 20`	F-001
TC-ONE-C-002	implicit multiplication `2(5+2) = 28`	F-002
TC-ONE-C-003	percent `200+10% = 220`; `200×10% = 20`; `50% = 0.5`	F-003
TC-ONE-C-004	pending operator: `10+` then `−` → `10−`; `5+ =` → `10`	F-004
TC-ONE-C-005	`10×−` evaluates `-10`	F-005
TC-ONE-C-006	auto-balance `(2+3 =` → `25`; smart `( )` key	F-006
TC-ONE-C-007	`123456` displays `1,23,456`; copy strips commas	F-007
TC-ONE-C-008	exactness: `0.1+0.2 = 0.3`; `1÷3×3 = 1`	F-008
TC-ONE-C-009	functions: INV sin 0.5 → `30` (deg); domain error card for `ln(−1)`	F-009
TC-ONE-C-010	live answer appears/stays consistent; no stale answer on incomplete input	F-010
TC-ONE-C-011	caret: tap mid-number inserts digit; ⌫ deletes one char	F-011
TC-ONE-C-012	selection pill: long-press `123456` → highlight + `Cut Copy MAdd MUse Paste`	F-012
TC-ONE-C-013	history: 5 entries, force-stop, relaunch → 5 entries exact	F-024
TC-ONE-C-014	replay: tap card paints expression before close; race guard	F-022/23
TC-ONE-C-015	MAdd/MUse: select `4` in `3+4`, MUse stored `123456` → `3+123456 =` `1,23,459`	F-019/20
TC-ONE-C-016	AC clears selection then expression	F-017
TC-ONE-C-017	tap-away exits selection, places caret	F-016
TC-ONE-C-018	cut & retype; copy/paste sanitize	F-014/15
TC-ONE-C-019	empty-caret menu (`MUse Paste`); insertion at caret	F-018

Case	Verifies	Binding
TC-ONE-C-020	memory sheet: slots, MUse/MAdd/MC, reorder	F-026
TC-ONE-C-021	key shape picker: 5 shapes, instant change, persists	F-028
TC-ONE-C-022	overflow list exactness (six items)	F-032
TC-ONE-C-023	key theme: 8 themes; Rose switch pixel-diff ≥ 10%; persists	F-027
TC-ONE-C-024	effects: 4 styles, = animation, persists	F-031
TC-ONE-C-025	no IME on calculator screen	F-033
TC-ONE-C-026	light/dark: expression/answer/caret colors per H1–H4	F-035

Module smoke set (Catalogue-0, one each): note create-edit-search survives restart; scan import→export PDF; event+reminder fires in exact-alarm-permitted emulator; backup→wipe→restore (J-03); music folder import→play 5 s; record 10 s with marker→playback; hub search finds an item from 3 modules; MyCalc flavor runs TC-ONE-C-001…008 with zero module remnants (J-09); zero-permission manifest on flavor (F-038); airplane-mode cold start smoke (J-05).

20.7 Beyond Catalogue-0 (catalogue digest)

Full catalogue lives in `qa/testcases.csv` (generated-checked); digest:

- **SEC** (14 cases): manifest deny-list; dex network-class scan; backup zip-bomb/pTraversal rejections; wrong-passphrase restore; clipboard auto-clear; pack tamper → disabled; app-lock round-trip; SAF grant review; trash hard-delete shred; log PII scrub; intent export surface scan; custom-permission absence (flavor); WebView absence; receiver export scan.
- **PERF** (10): cold start budgets (UX-002); jank budgets: calculator 60 fps p50 keystroke→frame ≤ 16 ms (≤ 64 chars), lists 55 fps p50 scroll on Tier C profile; recorder FGS start ≤ 200 ms; hub search first page ≤ 120 ms @10k notes; DB write frame-safety; memory caps per AI tier (§16.6); APK size budgets (§23.6); battery: 10-min idle music paused = 0 wakelocks; analyzer walk no-jank; OCR job ≤ budget on Tier B fixture.
- **A11Y** (12): TalkBack core journeys; contrast token test per theme (programmatic); touch-target scan; focus order; font 2.0 no-clip; reduced-motion removal; content descriptions; live-region on live answer; dialog focus trap; heading structure; braille-display-friendly labels (no emoji in a11y text); audio-independent recorder UI.
- **STATE** (8): process-death storm J-06; rotation mid-record/capture/draft; restore-keys registry completeness; continue-row exactness; calculator expression+caret+selection restore; trash 30-day expiry sweep (local clock advance test); migration v1→v2 dry-run fixture; backup schema drift rejection (newer backup on older app → clear error).
- **AI/PACK** (16): orchestrator gate matrix (tier×battery×concurrency); each fallback rung (§16.7) forced and asserted; provenance fields present; pack import/reject/enable/disable/delete; hash mismatch disable; eval-kit thresholds (§16.8); LLM JSON schema validation; streaming cancel; warm-evict on Tier A; embedder batching budget; transcribe segment timestamps monotonic; semantic merge ordering; OCR confidence floor; label pack absent UX; assistant absent UX; TTS absence (engines without TTS) degrade silently.

20.8 Coverage gates

Line coverage ≥ 85% on `core/domain`, ≥ 75% on `core/data`, ≥ 60% on features (measured by Kover, reported per module in CI artifacts). Coverage is a floor, not a goal: a fully covered wrong expectation is still a bug — the binding to F/UX IDs (§20.5) is the real oracle.

20.9 Defect ledger

All FAILs (and any suspicious PASS) land in `reports/defects.md` with: id, TC, severity S1 (crash/data loss)…S4 (polish), repro, evidence path, status (New→Triaged→In-progress→Fixed→Verified→Closed / Blocked / Rejected-with-reason). Blocked items record the exact prevention (e.g., "emulator `input swipe` cannot reproduce stationary hold-then-drag; requires physical device — owner input"). Verified requires a green re-run on the pinned release APK (C10). Severity S1 blocks release unconditionally; S2 needs a gate-report waiver with rationale.

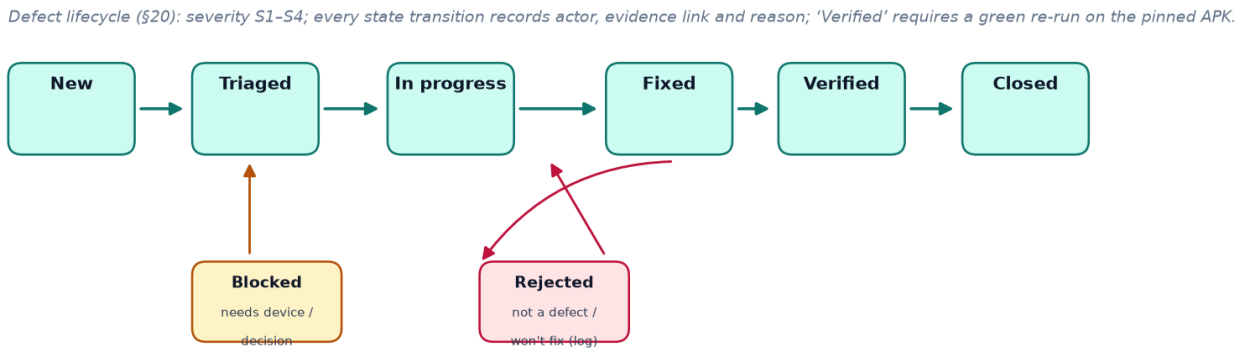


Figure 19. *Defect lifecycle with blocked and rejected paths.*

20.10 Severity/priority interaction and flake policy (normative)

Defect priority = severity × feature-priority matrix (`qa/severity.yaml`): an S2 on a P0 feature is release-blocking; an S2 on a P2 feature is fix-in-backlog. Reopened defects count double in the metrics (flaky fixes are the classic hidden cost).

Flake policy: a test that fails without product change is quarantined within one working day (moved to `quarantine/` with an owner and a one-week clock); two consecutive green full-lanes are required to release from quarantine. A flake root-cause note is mandatory (timing? ordering? resource?) — the owner's earlier campaigns showed ANR-dialog interference classes; the harness now records system-dialog encounters per case (§22.4) so flake analysis starts with evidence, not guesses.

20.11 Evidence retention

Gate + release evidence bundles are kept for the life of the repository (history is the audit trail); per-PR artifacts 30 days (CI-managed); local diagnostics ring 7 days by design. Nothing is ever deleted from `reports/` without a ledger entry (the owner's preservation standard).

20.12 One exemplar test per layer (house style reference)

```
// L1 (JVM, property): TC-LAW-003 — rendering never invents 0.999…
class LawL3RenderingTest {
  private val engine = CalcEngine()
  @Test fun `third-times-three renders one`() {
    val r = engine.evaluate("1÷3×3", EngineConfig.deg())
    assertEquals("1", engine.format(r, EngineConfig.deg()))
  }
  @Test fun `law holds on random decimals`() {
    Random(3311).decimalTriples(10_000).forEach { (a, b, c) ->
      val expr = "($a÷$b)×$b×$c÷$c"  // cancel back to a
      val r = engine.evaluate(expr, EngineConfig.deg())
      assertTrue(r.exact)                       // rational path all the way
      assertEquals(a, engine.format(r, EngineConfig.deg()))
    }
  }
}

// L2 (Robolectric): TC-NOTE-fts-01
@Test fun `search finds and ranks with snippets`() {
  db.noteDao().upsert(fixture("groceries"), fixture("laptop advice"))
  val hits = NoteSearch.query("laptop", budget = 25)
  assertEquals(1, hits.size); assertTrue(hits[0].snippet.contains("laptop"))
}

// L3 (Roborazzi): theme switch diff (UX-009)
@Test fun `rose theme repaints keys`() {
  val before = capturePad(theme = mySpaceSettings)
  setKeyTheme(Rose); val after = capturePad(theme = Rose)
  assertTrue(pixelDiff(before, after, region = KEYS) >= 0.10)
}

// L4 (instrumented): TC-ONE-C-004 trailing operator
onCalc { tapSequence("5", "+"); tap("=") }
onView(withText("10")).check(matches(isDisplayed()))

// L5 (black-box harness): Appendix Q script C-008 executes the same case
// from outside the app on the distributed APK, asserting via OCR + evidence shots.
```

House style rules visible above: tests cite spec rows in names/comments; seeds recorded; budgets explicit; no sleeps; assertions on user-visible state.

20.13 Coverage report example (Kover CI artifact, abridged)

```
{"module": "core/domain", "line": 0.91, "branch": 0.87, "budget": 0.85, "ok": true},
{"module": "core/data",   "line": 0.84, "branch": 0.79, "budget": 0.75, "ok": true},
{"module": "feature/calculator", "line": 0.71, "budget": 0.60, "ok": true},
{"module": "feature/scanner",    "line": 0.63, "budget": 0.60, "ok": true}
```

Uncovered-code review happens at gates: any uncovered class in `core/*` needs either a test or a ledger reason (generated code, delegate-only, JNI shim).

Chapter 21 — Test environments and data

21.1 Emulator matrix

Profile	API	RAM/skin	Purpose
E26C	26	2 GB, 1080p	floor: compatibility, Tier C
E33B	33	4 GB	median: notifications model, exact alarms, media APIs
E35A	35/36	8 GB	Tier A: LLM pack, predictive back, edge-to-edge

Managed devices in CI (Gradle managed emulator or equivalent), 3 parallel runners max (C8), airplane-mode cold-start runs as a system-state step (J-05) on each profile.

21.2 Physical-device delta

Some behaviors cannot be honestly tested via emulator input injection (documented example: stationary hold-then-drag selection, B2/F-013 — `adb input swipe` starts a move immediately; the app's 300 ms stationary detector rejects it). Policy: emulator runs mark such cases **BLOCKED** with the prevention recorded; the release evidence lists them as "requires physical-device verification"; the owner's device run can execute the same harness (the L5 harness is plain adb+Python, runnable on any USB device).

21.3 Test data fixtures (in-repo, versioned)

`qa/fixtures/`: 40 OCR images (printed/handprint Latin, multi-size, incl. invoice-style), 20 audio clips (clean/noisy speech, 5–30 s) with reference transcripts, 200-note corpus (Markdown+NoteDoc) for search/eval, 200 RRULE golden corpus, 12 `.msone` backups (v1; incl. 2 malicious-pattern archives for rejection tests), 6 music fixtures (generated tones with metadata, 10–60 s), 3 model-pack dummies (correct-shape, bad-hash, oversized) for PACK tests, 40-image eval kit + 20-note summarization rubric set (§16.8). All synthetic; no personal data.

21.4 Determinism

Every randomized test takes a seed recorded in its output; law-suite corpora are seeded per CI run id so failures reproduce; screenshot tests pin font rendering (bundled fonts, fixed density). Time-dependent tests use an injected `Clock` (trade-off noted: system-clock paths get one dedicated smoke per module to catch real-world skew).

21.5 Fixture generation and provenance

All synthetic fixtures are generated by `tools/fixtures.py` (seeded) so they are reproducible: OCR images rendered from bundled fonts with affine jitter; audio clips from phoneme-balanced synthetic speech corpus + noise mixes; music = generated sine/chirp tones with ID3/Vorbis metadata; backups = built from the Room fixture seeds via the real backup manager (never hand-crafted zips), except the malicious-pattern set which is hand-built to exercise guards. Every fixture file carries a `fixtures-manifest.json` (sha256, generator seed, purpose) verified by a fast CI test — the same provenance habit the release evidence uses.

21.6 Fixture manifest example (checked by CI)

```
{"ocr/inv-007.png": {"sha256": "3f2c…", "seed": "7", "purpose": "invoice printed 10pt"},
 "asr/clip-011.wav": {"sha256": "91aa…", "seed": "11", "ref": "asr/clip-011.txt"},
 "backups/v1-ok-03.msone": {"sha256": "bb0d…", "generator": "BackupManager@9c4e"},
 "packs/dummy-essentials.mspack": {"sha256": "5c1f…", "models": 3}}
```

A missing fixture or hash drift fails the environment test before any suite runs — QA never starts on a drifted tree.

21.7 Emulator provisioning script (runner bootstrap, normative)

```
sdkmanager "emulator" "platform-tools" "system-images;android-26;google_apis;x86_64"   "system
    -images;android-33;google_apis;x86_64" "system-images;android-35;google_apis;x86_64"
avdmanager create avd -n e26c -k "system-images;android-26;google_apis;x86_64" -d pixel_4   &&
     adb emu avd snapshot reset qm  # cold each suite
# profile caps: e26c → 2048 MB heap, e33b → 4096, e35a → 8192 (tier parity with §3.2)
```

Snapshots are reset per suite (never reused across suites) to keep J-05/J-06 honest; airplane mode is toggled via `svc wifi disable && svc data disable` + `cmd connectivity airplane-mode enable`, and the harness *asserts* radio-down state before airplane-labeled cases (an asserted environment, not an assumed one).

21.8 Physical-device lab protocol (when a device is attached)

```
1. adb device fingerprint recorded (model, Android version, build) into evidence
2. Harness runs the same YAML cases; gestures B2-class attempted for real:
   long-press then continuous move with pressure/stationarity the emulator lacks
3. OEM quirks probed: alarm policy, FGS kill aggressiveness, SAF provider quirks
   (results appended to the OEM quirk register in the ledger)
4. Screenshots pulled per case; manifest verified; results merged with emulator runs
   (dedupe rule: device results annotate, never silently overwrite emulator ones)
```

21.9 CI schedule summary

Lane	When	Contents
Fast	every PR	L1+L2+L3 + audits (≤ 12 min)
Full	trunk merge + nightly 02:00	+ L4 matrix + L5 nightly profile
Release	tags	reproducible builds + release L5 + docs + bundle
Owner-device	on demand	§21.8 protocol

Chapter 22 — Black-box APK QA harness contract

This chapter formalizes the L5 layer so its reports match the evidence style the owner already trusts (their calculator QA campaigns): pinned inputs, honest states, screenshots, manifests, and a PDF report of record.

22.1 Scope rule

The harness tests **distributed APKs** (release or dist builds), never debug builds, and never infers released-binary behavior from source CI. MySpace-module focus discipline: probes target the app's calculator (and the module under test); anything else recorded by a probe is context, never counted as calculator coverage.

22.2 Pinned-input manifest

Every run embeds: APK path, package, versionName/versionCode, SHA-256 (asserted against the manifest before install — a mismatch aborts the run as INVALID, not FAIL); the spec hash (this blueprint's section set, or the owner guide hash when the owner supplies one); source commit + workflow run id; emulator profile; host runner image. The report builder re-validates every row (stronger than trusting labels) — the owner's reports already set this standard.

22.3 Session discipline (per probe)

1. Fresh install (or `pm clear`) per suite unless the case tests persistence.
2. Deterministic route to the module (explicit activity/component when exported to self; else UI path) with a **navigation assert** — if the harness finds itself off-module (the historical "Pixel launcher" drift), it records BLOCKED-with-context and resets; it does not keep probing unrelated screens.
3. State-reset between cases (AC, close sheets, dismiss dialogs); popup vocabulary handled explicitly (e.g., a persistent history button is not a popup — the harness must not "back out" of the app hunting for it, a recorded past lesson).
4. Input honesty: taps/long-presses via instrumented UIAutomator when in-process (L4) or adb `input` (L5); gestures the input API cannot fairly reproduce are BLOCKED by rule (§21.2), never retried into false FAILs.
5. OCR-based assertions (screenshot → text) tolerate rendering variance with normalized matching; ambiguous reads capture the raw crop as evidence.

22.4 Stability rules from field lessons

No toast calls from worker threads in the app (the harness *tests* this via SYS cases; the historical crash class stays dead); harness waits are condition-based (idle-resource + idle-instrumentation), never fixed sleeps beyond 2 s; ANR dialog handling: the harness detects system ANR dialogs, selects *Wait* up to 3 times (recording each), then reports BLOCKED — mirroring the owner's harness behavior; every screenshot embeds the step name in its filename for mechanical manifest pairing.

22.5 Outputs

Per run, under `reports/evidence/<run-slug>/`:

```
raw/<shard>/user-journey-results.json     # per-case: id, status(PASS/FAIL/BLOCKED), actual, c
    omment, shots[]
raw/<shard>/calculator-evidence-*.png     # step screenshots
raw/<shard>/logcat.txt                    # filtered, scrubbed
evidence-manifest.json                    # every file: sha256 + size (verified post-write)
qa-summary.json                           # counts, pinned inputs, run ids
```

The PDF builder consumes only manifest-verified files (any missing/mismatched file fails the build), then emits the report of record with: provenance page, version-alignment notes, per-case tables, embedded screenshots, blocked-items register, and source-CI context clearly labeled as *not* direct-APK proof. Publisher success is never conflated with passing results — the report shows failing cases even when the publishing job is green (field lesson §15.8(3)).

22.6 Acceptance style

Reports state: what was proven, what was blocked and why, what was not tested at all. No third-party substitutes for named comparisons (e.g., official Google Calculator parity is compared only when the genuine package is installable on the test device; absence is recorded, never substituted — the owner's explicit standard).

22.7 Harness case schema (normative example)

```
- id: TC-ONE-C-004
  binding: [F-004]
  suite: calculator-parity
  pre: install(app=full, pin_sha=…), start_module(calculator), state_reset()
  steps:
    - tap_sequence: [5, +]
    - tap: "="
    - assert_ocr: {region: display, contains: "10", normalize: strip_grouping}
    - screenshot: user-tc-one-c-004-after-equals.png
  on_blocked: record(context="…"), continue_suite
  evidence: [results-json, screenshots, logcat-slice]
```

Rules: every step has an evidence artifact or a named assertion; `on_blocked` must record the prevention, not silently pass; regions are symbolic (resolved per layout) so the harness survives minor UI moves; OCR normalization rules (`strip_grouping`, `unicode_ops`) are shared code with the app's renderer tests to avoid double standards.

22.8 Harness run profiles (normative)

Profile	Trigger	Suites	Artifacts
PR-smoke	every PR (L5-capable runner free)	calculator parity subset (C-001…008) on E33B debug-as-dist build	results JSON, 8 screenshots
nightly	schedule	full ONE set + module smokes on E26C/E33B/E35A	3 × results JSON + manifest
release-candidate	tag build	full set against signed release APKs (both flavors), airplane-mode profile included	qa-summary + PDF of record inputs
owner-device	on demand (USB device attached)	full set on physical device, B2-class gestures attempted for real	results + device screenshots

Profiles share the case YAML (§22.7); only environment blocks differ. A release-candidate profile red blocks the release regardless of other lanes (Catalogue-0 rule).

22.9 Evidence naming conventions (normative)

- Screenshots: `calculator-evidence-<case-or-step-id>-<slug>.png` (the owner's existing pattern), lowercase, hyphenated.
- Results: `user-journey-results.json` per shard (compatibility with the owner's downstream tooling).
- Manifest: `evidence-manifest.json` at bundle root; every file listed with sha256 + bytes; verified post-write by the publisher before any report build.
- Logs: `logcat-<shard>.txt` filtered to the app pid + harness tags, scrubbed per §28.3.

22.10 Journey YAML sample (J-02, trimmed)

```
- id: J-02
  title: Record to minutes
  pre: [install(pinned), grant(mic), pack(speech=installed)]
  steps:
    - {do: start_module, arg: recorder, shot: j02-01-open}
    - {do: tap, arg: record, wait: {notification: "Recording"}}
    - {do: sleep, arg: 30}
    - {do: tap, arg: flag, shot: j02-04-flag}
    - {do: tap, arg: done}
    - {do: assert_db, arg: "recording row exists, duration>=29000"}
    - {do: tap, arg: transcribe, budget: {s: 120}, shot: j02-07-transcribed}
    - {do: assert_ocr, region: transcript, contains_any: ["meeting", "agenda"]}
    - {do: tap, arg: to_notes}
    - {do: assert_db, arg: "note created with sourceRecordingId"}
  blocked_if: [mic_denied_unexpected, fgs_start_slow]
```

22.11 OCR assertion normalization rules (shared with renderer tests)

Rule	Input sample	Normalized
strip_grouping	`1,23,459`	`123459`
unicode_ops	`3+4×2` (with U+2212/00D7)	`3+4x2` ASCII-fold
collapse_ws	multi-space OCR jitter	single space
casefold_alpha	`History` vs `HISTORY`	lower
superscript_fold	`10³`	`10^3`
ligature_fold	`ﬁ`	`fi`
minus_unify	`− — –`	`-`

Both the app's renderer tests and the harness import the same normalization module (`qa/normalize.py` ↔ `core/designsystem/testutil`) — one definition of "equal" everywhere (the owner's harness philosophy).

22.12 `user-journey-results.json` sample (schema-compatible with owner tooling)

```
[{"id": "TC-ONE-C-004", "binding": ["F-004"], "status": "PASS",
  "actual": "display=10; second equals=15", "comment": "trailing-operator rule proven",
  "shots": ["calculator-evidence-tc-one-c-004-after-equals.png"], "ms": 4120},
 {"id": "TC-ONE-C-B2", "binding": ["F-013"], "status": "BLOCKED",
  "actual": "hold-then-drag not reproducible via adb input",
  "comment": "no product failure inferred; requires physical device"}]
```

22.13 Harness code skeleton (Python, `qa/` layout contract)

```
# qa/harness.py — entrypoint: run(profile) → evidence dir
class Device:            # adb wrapper: tap/swipe/longpress/key/shell/screenshot
    def tap(self, x, y): ...
    def longpress(self, x, y, ms=300): ...
class OcrReader:         # screenshot → normalized text (§22.11 rules)
    def text(self, img, region=None) -> str: ...
class Case:              # loaded from YAML (§22.7)
    id: str; steps: list[Step]; on_blocked: str
class Runner:
    def run_case(self, case, device, ocr) -> CaseResult:   # PASS/FAIL/BLOCKED + evidence
class Publisher:
    def write_results(self, results): ...                  # user-journey-results.json
    def write_manifest(self, files): ...                   # sha256 + verify pass
# CLI: python3 qa/harness.py --profile nightly --apk artifacts/MySpace-…-dist.apk --pin-sha …
```

The harness stays independent of the app build (own venv, own deps) — it must be runnable by the owner against any APK file without the repo's toolchain.

PART IX

BUILD, RELEASE AND OPERATIONS

The toolchain, variants, budgets, CI/CD lanes, audit gates, signing, release bundles, distribution and post-release operations. This part turns a green codebase into a verifiable artifact an owner can install and trust.

Chapter 23 — Build system and configuration

23.1 Toolchain (pinned)

Tool	Version policy
JDK	17 (Temurin), pinned via wrapper + CI matrix
AGP/Kotlin	latest stable at G1, frozen for the release; upgrades post-G7 go through the ledger
Gradle	wrapper, configuration cache ON, build cache shared per runner
SDK	compileSdk 36, target 35, min 26
R8	full mode, `optimization.json` diff attached to release evidence

23.2 Variants

Variant	Purpose	Signing
`fullDebug`	dev	debug key
`fullRelease`	the product APK	release keystore (§25.2)
`mycalcDebug`	flavor dev	debug key
`mycalcRelease`	standalone calculator APK	release keystore

Flavor config: `applicationId` per §9.3; `resValue` app names; source-set swaps (hub absent in mycalc; launcher icon per flavor; version-name suffix none). BuildConfig flags: `PROVENANCE_SHA` (git rev-parse injected), `BUILD_TIME` — surfaced in About (F-084).

23.3 Reproducibility

Deterministic flag set (`zipflinger` stable ordering, fixed file timestamps where supported, locale-independent resource merging); the CI builds twice on tag builds and compares SHA-256 (V1–V2 signatures stable) — a mismatch fails the release. Local-vs-CI drift is checked once per phase via a "release candidate" build on the runner and recorded in metrics.

23.4 Flavor parity mechanics

`feature/calculator` is flavor-agnostic; `app/mycalc` contributes only: launcher, app name, nav graph root = calculator, and a compile-time `BuildConfig.IS_STANDALONE` consumed by the calculator module to hide cross-module actions (vs. runtime ifs — the R8 tree in mycalc proves no hub classes remain: lint check `mycalcNoCrossModule` scans the flavor's dex for `feature/hub` + `feature/*` class references and fails on any, enforcing F-040/F-039 structurally).

23.5 Dependency policy

Curated, small: Compose BOM, Room, DataStore, Koin, Media3, CameraX, Biometric, ONNX Runtime Mobile, LiteRT (pack-gated code path), MediaPipe LLM task lib, whisper.cpp JNI (self-built, prebuilt .so per ABI), kotlinx libs, Robolectric/Espresso/Roborazzi (test). Rules: no transitive network stacks even transitively present (a `dependencies` audit fails if `okhttp/retrofit` appears in the release graph — the AI libs are verified network-free configurations); new dependency requires a ledger entry + size delta note; licenses re-generated per release (F-085).

23.6 Size and performance budgets (release, per ABI set)

Budget	Limit	Measured by
fullRelease APK	≤ 30 MB	build output + size report
mycalcRelease APK	≤ 6 MB	build output
cold start hub (Tier B)	≤ 1.2 s p50	Macrobenchmark in CI
cold start mycalc	≤ 0.8 s p50	Macrobenchmark
keystroke→frame (≤ 64 chars)	≤ 16 ms p50	engine benchmark + compose tracing
hub first search page @10k notes	≤ 120 ms p50	JUnit perf test on fixture corpus
recorder FGS start	≤ 200 ms p95	instrumented timing
DB write on main path	never (queue)	architecture test + strict-mode detector in debug

Budgets are asserted in CI (fail the build, not a dashboard); deliberate budget raises require a ledger entry + owner-visible note at the next gate.

23.7 Variants DSL (trimmed, normative shape)

```
android {
  flavorDimensions += "dist"
  productFlavors {
    create("full") {
      dimension = "dist"
      applicationId = "app.space.one"
      resValue("string", "app_name", "MySpace One")
    }
    create("mycalc") {
      dimension = "dist"
      applicationId = "app.space.calc"
      resValue("string", "app_name", "MyCalc")
      versionNameSuffix = ""
    }
  }
  buildTypes {
    release {
      isMinifyEnabled = true; isShrinkResources = true
      proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"),
                    "proguard-rules.pro")
      signingConfig = signingConfigs.getByName("release")
      buildConfigField("String", "PROVENANCE_SHA", "\"${ext["gitSha"]}\"")
    }
  }
  androidResources { localeFilters += listOf("en", "en-IN") }
  packaging { jniLibs.useLegacyPackaging = false }
}
```

23.8 R8 keep rules (release, near-complete file)

```
# AI JNI bridges (reflection from native)
-keep class space.one.core.ai.jni.** { *; }
# Compose + Room generated are handled by their rules; add:
-dontwarn org.slf4j.**
# DataStore serializers via kotlinx
-keepclassmembers class * extends com.google.protobuf.GeneratedMessageLite { <fields>; }
# ONNX Runtime
-keep class ai.onnxruntime.** { *; }
# MediaPipe tasks
-keep class com.google.mediapipe.tasks.** { *; }
# Keep line numbers for local symbolication
-keepattributes SourceFile,LineNumberTable
-renamesourcefileattribute SourceFile
```

`optimization.json` + `mapping.txt` archived per release (§25.3); a CI check fails if rules grow beyond ~40 lines without a ledger entry (rule bloat = design smell).

23.9 Build performance techniques (why CI fits the budget)

Technique	Effect
Gradle configuration cache + build cache	30–50% faster re-builds
Module-parallel test tasks (3 runners)	device lane fits 90 min cap
Test sharding by module (§11.2)	no single runner bottleneck
Robolectric over emulator for L2	minutes cheaper per case
Roborazzi on JVM (not device)	screenshot lane < 4 min
`androidResources.localeFilters`	smaller resources, faster AAPT
Baseline profiles generated at G6	cold-start budget without runtime JIT cost

23.10 Version catalog (gradle/libs.versions.toml, excerpt)

```
[versions]
kotlin = "2.0.21"
agp = "8.7.3"
compose-bom = "2024.12.01"
room = "2.6.1"
media3 = "1.5.1"
camerax = "1.4.1"
onnxruntime = "1.20.0"
mediapipe-tasks = "0.10.21"
[libraries]
compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "compose-bom"
    }
room-runtime = { group = "androidx.room", name = "room-runtime", version.ref = "room" }
# (single source of dependency truth; dependabot-style bumps are ledger-gated)
```

Chapter 24 — CI/CD

24.1 Pipeline shape

One workflow per PR (fast lane: L1+L2+L3, architecture/lint, ~12 min) and one per trunk-merge or nightly (full lane: + L4 matrix + L5 harness, ≤ 90 min wall with 3 runners sharding; ≤ 30 jobs per run, C8). Release lane (tags): build, reproducibility double-build, L5 against release APK, docs regen, evidence bundle, GitHub Release draft.

24.2 Runner discipline

Concurrency groups cancel superseded runs; emulator jobs (the scarce resource) shard: E26C+E33B on runner 1, E35A on runner 2, L5 on runner 3; module test tasks shard by module; artifacts (results JSON, screenshots, logs) upload even on failure — a red run with no evidence is itself a pipeline defect.

24.3 Required checks (branch protection)

`build+lint`, `unit+property`, `integration`, `screenshots`, `instrumented-matrix`, `blackbox-apk-qa`, `size-budgets`, `manifest-audit`, `dependency-audit`, `traceability`. All green = mergeable. The historical publisher pattern (report commit by CI) is retained for release lane only; PR lanes never write to the repo.

24.4 Audit gates (config as law)

- `manifest-audit`: deny-list permissions (C1 set), exported-component allow-list, flavor zero-permission assert (F-038).
- `dependency-audit`: deny-list package prefixes (ads/tracking: `com.google.android.gms.*` ad classes, `com.facebook.*`, `com.applovin*`, `com.adcolony*`, `com.unity3d.ads*`, `io.branch*`, `com.mixpanel*`, `com.amplitude*`, analytics/crash reporters), plus network-stack presence scan (§23.5).
- `traceability`: every F/UX has ≥ 1 TC; every TC has binding; every commit in the release range carries `Refs:` or maps to chore/docs (warn).
- `docs-drift`: blueprint § refs in code must exist in the document set (regex parse), preventing stale pointers.

24.5 CI workflow sketch (structure, trimmed)

```
name: full-lane
on: { push: { branches: [main] }, schedule: [{cron: "0 2 * * *"}] }
concurrency: { group: full-lane, cancel-in-progress: true }
jobs:
  fast:        # runner 1
    steps: [checkout, setup-jdk-17, gradle: spotlessCheck detekt lint testAll]
  integration: # runner 1 (after fast)
    steps: [robolectric suite, manifest/dependency/traceability/docs-drift audits]
  shots:       # runner 2
    steps: [roborazzi verify, upload diff artifacts on failure]
  device:      # runner 3, matrix [E26C, E33B, E35A]
    steps: [managed-emulator up, connectedShadedAndroidTests (journeys, a11y, perf)]
  blackbox:    # runner 3 (after device)
    steps: [assembleRelease both flavors, pin SHAs, run qa harness, upload evidence]
  report:
    needs: [blackbox]
    steps: [build PDF of record from evidence, attach to run summary]
```

Job count per full lane stays ≤ 30 (matrix expansion counted); the fast PR lane is a strict subset (≤ 8 jobs).

24.6 Fast PR lane YAML (trimmed)

```
name: fast-lane
on: {pull_request: {}}
concurrency: {group: pr-${{ github.ref }}, cancel-in-progress: true}
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: {distribution: temurin, java-version: "17"}
      - run: ./gradlew spotlessCheck detekt :app:lintFullDebug
      - run: ./gradlew testDebugUnitTest robolectricSuite
      - run: ./gradlew roborazziVerify
      - run: python3 tools/traceability.py --check
      - run: python3 tools/audits.py manifest dependencies docs-drift
      - uses: actions/upload-artifact@v4
        if: always()
        with: {name: reports, path: build/reports/}
```

24.7 Nightly lane additions (beyond PR fast lane)

```
  nightly:   # after fast jobs pass on main
    steps:
      - run: ./gradlew connectedShadedDebugAndroidTest  # journeys J-01..J-10 (3 runners)
      - run: python3 qa/run_blackbox.py --profile nightly --emulators E26C,E33B,E35A
      - run: ./gradlew macrobenchmarkHub macrobenchmarkCalc
      - run: python3 tools/traceability.py --emit reports/traceability.csv
      - uses: actions/upload-artifact@v4
        with: {name: nightly-evidence, path: reports/evidence/nightly-*/}
```

24.8 CI artifact retention

Artifact	Where	Retention
Test reports (fast lane)	CI artifacts	30 d
Screenshot diffs	CI artifacts	30 d
Nightly evidence	CI artifacts	90 d
Release bundles	Git release assets + repo `releases/`	forever

Chapter 25 — Release engineering

25.1 Versioning

`versionName = MAJ.MIN.PAT` (user-visible), `versionCode = MAJ*10000+MIN*100+PAT*10+flavorBit` (full=0, mycalc=1, keeping the owner's monotonic scheme familiarity). Release candidates: `3.13.0-rc1` locally, stripped on tag.

25.2 Signing

Keystore generated once in CI secrets flow (owner-held backup is an owner action item at G1); PKCS12, RSA-3072, 25-year validity; `sigv1+sigv2+sigv3` enabled for sideload compatibility. Signing happens only in the release lane; the keystore never touches the workspace outside the secret mount; public SHA-256 of the signing cert is published in the About screen + reports (owners can verify install provenance).

25.3 Shrink and hardening

R8 full + resource shrink; keep rules minimal (reflection only in AI JNI bridges — pinned rules); `-allowaccessmodification`; debug metadata stripped except line info (crash-free local symbolication via committed `mapping.txt` per release into the evidence bundle).

25.4 Pre-flight checklist (automated)

S1–S8 re-run on the exact release artifacts; zero-permission flavor assert; size budgets; reproducibility double-build; L5 release run green (blocked items enumerated); docs regenerated; licenses regenerated; About provenance matches. Any red → no release, defect filed (§20.9).

25.5 Release artifacts (the bundle)

```
releases/v3.13.0/
  myspace-one-3.13.0-<code>-release.apk
  mycalc-1.1.0-<code>-release.apk
  qa/qa-summary.json + report.pdf (L5 report of record)
  evidence/ (manifest-verified screenshots, logs, law-suite results)
  docs/ (blueprint PDF, traceability.csv, api-surface.txt, LICENSES.md)
  RELEASE-NOTES.md (human, short, honest: features, fixes, blocked register)
```

25.6 Distribution (owner model)

Direct sideload via the owner's existing pattern (GitHub-hosted artifacts on the repo/branch the owner controls) + pack files distributed alongside with their SHA-256 manifest. In-app updater: none (no network, by design) — update = install-over, tested with signature-continuity check (TC-SEC-upgrade: v(n)→v(n+1) install-over preserves data through Room migrations).

25.7 RELEASE-NOTES.md template

```
## MySpace One 3.13.0 (code 31300) · MyCalc 1.1.0 (code 11100)
Date / provenance: commit <sha>, CI run <id>, APK SHA-256 <…>

### Highlights
- Calculator: [user-visible changes only]
- Notes / Scanner / …: …

### Fixes
- …

### Honest register
- Blocked during QA: [list with reasons and device-test recommendations]
- Known limitations: [e.g., ICS RRULE subset]

### Upgrade notes
- Install over the previous version; data migrates automatically (tested: v(n)→v(n+1)).
- Backups made by newer versions cannot be restored by older versions.
```

25.8 Signing ceremony (normative steps)

```
# One-time (G1): generate in CI secret store
keytool -genkeypair -v -keystore msone-release.p12 -alias msone   -keyalg RSA -keysize 3072 -v
    alidity 9125   -dname "CN=MySpace One, O=Owner"
# Owner action: download + verify passphrase + store offline (two copies) → rehearse restore
# Release lane: mount secret → gradle signReleaseBundle → verify
apksigner verify --print-certs myspace-one-release.apk   # cert SHA-256 recorded in evidence +
     About
# Tag the keystore fingerprint into reports (provenance continuity across versions)
```

Loss of the key after first public release = cannot install-over (new key = uninstall). The G1 rehearsal (owner re-imports keystore from backup into a clean runner and signs a test APK) is the cheapest insurance in the whole program (R-010).

25.9 Release QA session script (the G7 instrument)

```
1. gradle assembleFullRelease assembleMycalcRelease        → record SHAs
2. Reproducibility: rebuild both → SHA equal? (fail = stop)
3. Install both on E33B (airplane ON):
   a. run L5 release profile (Appendix Q scripts verbatim)
   b. J-05, J-06, J-10 on the *release* artifacts
4. Owner-device optional pass for B2-class gestures
5. Diagnostics self-test on device → JSON into evidence
6. Assemble bundle (§25.5), verify manifest, build PDF of record
7. Pre-flight checklist §25.4 → G7 sign-off pack
```

Every step emits into `releases/v<ver>/qa/…`; the session is re-runnable byte-for-byte from the script (deterministic session — the owner can watch any step).

25.10 Filled RELEASE-NOTES example (illustrative v3.13.0)

```
## MySpace One 3.13.0 (code 31300) · MyCalc 1.1.0 (code 11100)
2026-12-18 · commit 9c4ef21 · CI run 4123 · full APK sha256 8f31… · mycalc sha256 2b7d…

### Highlights
- Calculator: exact engine — `1÷3×3 = 1`, percent chains, Google-style pending operators.
- Selection memory: hold any number → Cut/Copy/MAdd/MUse/Paste.
- Notes: checklists, full-text search, optional on-device AI actions (preview + undo).
- Scanner: documents to clean PDFs with on-device OCR.
- Calendar: recurrence + exact reminders, ICS import/export.
- Files: private + SAF browsing, storage analyzer, encrypted backup/restore.
- Music and Recorder: gapless playback, speed, sleep timer; recordings with markers.
- Diagnostics: self-test, model-pack manager, share-with-preview logs.

### Fixes
- First release.

### Honest register
- Blocked during QA: stationary hold-then-drag selection needs a physical device
  (emulator input limitation). Everything else in Catalogue-0: PASS.
- Known limitations: OCR v1 is Latin-script; ICS import supports the documented
  RRULE subset; Assistant pack needs an 8 GB-class device.

### Upgrade notes
- First release: install fresh. Backups from this version restore here and in later versions.
```

25.11 Bundle verification commands (owner-runnable)

```
sha256sum myspace-one-3.13.0-31300-release.apk   # == About screen + release notes
apkanalyzer manifest print …apk | grep uses-permission   # expected list only
python3 tools/verify_manifest.py releases/v3.13.0/        # checks evidence-manifest.json
```

Chapter 26 — Post-release operations (G8)

26.1 Support model

The Diagnostics module is the support surface: self-test card, log share (user-initiated), pack manager, and the About provenance block — an owner receiving a friend's bug report asks for exactly these three things. Support runbook (`docs/ops/runbook.md`): triage by severity template, evidence request templates, known-issue list from the defect ledger.

26.2 Update cadence and backports

Patch releases for S1/S2 defects (engine/data correctness first); minor for P1 completions; the blueprint version-bumps with each minor. Backport policy: only to the latest minor. Every update release re-runs the full release lane (no "quick builds" — C10 applies forever).

26.3 EOL and data exit

If the program ends: last release ships with a built-in "Export everything" deep action (Settings → Privacy → Export all data → `.msone` unencrypted option) and the docs note key escrow-free backup reality (user passphrases are the only keys; lost passphrase = lost backup — disclosed at backup creation, §14.3).

26.4 Support playbook (triage decision tree)

```
Report arrives → ask for Diagnostics self-test JSON + shared log (user-initiated)
├─ self-test fails engine/DB  → S1/S2 defect, reproduce via fixture seed, fix + law re-run
├─ self-test fails pack hash  → guide re-import of pack (corrupt transfer), no code change
├─ alarm missed               → check canScheduleExactAlarms in log slice → OEM policy doc + fa
   llback notice
├─ SAF oddity                 → capture root URI pattern → §13.6 map → workaround task
└─ UX complaint               → ledger entry; batch into next minor; never silent-fix without t
   raceability
```

26.5 Support FAQ (owner-facing, from the runbook)

Symptom	First response
"Reminder didn't ring"	Settings → Calendar shows exact-alarm status? Share self-test JSON (step 5). OEM battery settings guide.
"Backup won't open"	Passphrase is case-sensitive; version note: newer backups need the newer app.
"Music files missing after cleanup"	Files → Media folders re-grant; library rescan button in Music → ⋮
"Recording stopped by itself"	Check diag ring: call interruption vs storage event; file kept to pause point (by design).
"AI button greyed"	Diagnostics → Model packs → install/import the named pack; check device tier note.
"App is slow on old phone"	Tier C expectations §3.2; effects default off; try battery-saver-aware mode.

26.6 Owner feedback loop

Feedback enters exactly like defects: the owner's observation → ledger/defect entry (with the owner's own evidence style — screenshots welcome) → triage per Appendix U → fix in the normal loop with `Refs:` traceability. Feature wishes compete in the v1.1 backlog by the same priority rules as everything else — the owner's word is the spec, and the spec is versioned.

26.7 Update announcement template (owner channel)

```
MySpace One v3.13.1 is out (MyCalc unchanged at 1.1.0).
Install-over: open the APK, allow install, done — your data stays.
Highlights: <2-3 bullets from RELEASE-NOTES>
Verify: Diagnostics → About shows build 31301 / commit <sha7>.
Full report: <link to the release bundle PDF>
```

PART XI

APPENDICES

Reference material an executor needs at arm's length: the complete test-case catalogue, worked calculator traces, the recurrence golden corpus, guide-coverage maps, microcopy dictionary, error catalogue, pack format, gesture map, runbooks, constraint register, fixture inventory and agent templates.

Appendix A — Complete test-case catalogue

Machine-readable source of truth: `qa/testcases.csv` (generated from this table by `tools/traceability.py`; drift fails CI). Status column = latest full-lane result (populated from G3 onward). Binding = feature/UX ids.

A.1 Calculator parity (ONE) and engine (CALC/LAW)

Case	Binding	Verifies	Layer
TC-ONE-C-001…026	§20.6 table	guide-parity set (26 cases)	L4/L5
TC-LAW-001	F-008	L1 round-trip, 10k seeded cases	L1
TC-LAW-002	F-001/8	L2 exact identity	L1
TC-LAW-003	F-008	L3 rendering never invents 0.999…	L1
TC-LAW-004	F-003	L4 percent consistency	L1
TC-LAW-005	F-002	L5 implicit-mul AST equivalence	L1
TC-LAW-006	F-006	L6 balance completeness	L1
TC-LAW-007	F-005	L7 replacement determinism	L1
TC-LAW-008	F-008	L8 continuation fidelity (1e15+1 class)	L1
TC-LAW-009	F-007	L9 locale independence	L1
TC-LAW-010	F-024	L10 history fidelity	L1
TC-CALC-edge-001…020	§7.2.8 register	edge inputs (20 rows)	L1/L2
TC-CALC-frac-01	F-030	fraction rendering + fallback >999 denom	L1
TC-CALC-deg-01	F-009	DEG/RAD toggle + persistence + INV arc	L4
TC-CALC-perf-01	§23.6	keystroke→frame ≤ 16 ms p50 ≤ 64 chars	L1 bench
TC-PERF-key	§23.6	30 keys/s stress, no drops, no >24 ms frames	L4

A.2 Calculator UX / state / system

Case	Binding	Verifies	Layer
TC-STATE-calc-01	F-043	expression+caret+selection survive death	L4
TC-STATE-calc-02	UX-017	rotation mid-selection	L4
TC-SYS-theme-01	F-035	light/dark token resolution live	L3/L4
TC-SYS-reduced-motion-01	X8	effects/motion removed under setting	L4
TC-A11Y-calc-01	§6.6	TalkBack key announcements + caret context	L4
TC-A11Y-contrast-01	§6.6	all 8 key themes × light/dark contrast programmatic	L1
TC-ONE-haptics	F-029	on/off honored	L4
TC-ONE-clipclear	F-014/SEC-07	clipboard 60 s auto-clear	L4

A.3 Hub / search

Case	Binding	Verifies	Layer
TC-HUB-001	F-041	grouped results across 4+ sources, deep links	L4
TC-HUB-002	F-042	semantic merge + "related" tag (pack)	L4
TC-HUB-003	F-043	continue-row exact restoration	L4
TC-HUB-004	F-044	status lines freshness ≤ 1 s	L2
TC-HUB-005	F-046	first-run cards exactly once	L4
TC-HUB-006	§5.5	budgets: 25 ms/source, 120 ms paint @10k	L1 perf
TC-HUB-007	§5.5	FTS sanitization (operators as literals)	L2

A.4 Notes

Case	Binding	Verifies	Layer
TC-NOTE-rt	F-047/52	NoteDoc↔Markdown lossless round-trip (200 corpus)	L1
TC-NOTE-save-01	F-047	autosave 500 ms + death restore	L4
TC-NOTE-fts-01	F-048	ranking + snippet + budget	L2
TC-NOTE-trash-01	F-050	undo snackbar + 30-day sweep	L4
TC-NOTE-check-01	F-051	toggles + progress card	L2
TC-NOTE-edge-01…09	§7.14	register rows	L1/L2/L4
TC-AI-note-01	F-054	summarize preview/undo/provenance	L4
TC-AI-note-02	F-054	rewrite tones + schema retry/fallback	L2

A.5 Scanner

Case	Binding	Verifies	Layer
TC-SCAN-cap-01	F-055	batch capture + torch + focus	L4
TC-SCAN-imp-01	F-055	import path with camera denied	L4
TC-SCAN-crop-01	F-056	corner detect on fixtures ≥ 95% IoU	L1
TC-SCAN-ocr-01	F-057	OCR char accuracy ≥ 92% fixture set	L4
TC-SCAN-exp-01	F-058	PDF export + page order editor	L4
TC-SCAN-abs-01	F-059	graceful absence UX	L4
TC-SCAN-edge-01…07	§7.14	register rows	L1/L4

A.6 Calendar

Case	Binding	Verifies	Layer
TC-CAL-rr-01	F-061	golden corpus 200 rules exact occurrences	L1
TC-CAL-dst-01	F-061	DST-crossing series (ADR-08)	L1
TC-CAL-alarm-01	F-062	exact fire in policy window (E33/E35)	L4
TC-CAL-alarm-02	F-062	inexact fallback + one-time notice	L4
TC-CAL-ics-01	F-063	import subset + unsupported→one-off+note	L2
TC-CAL-ics-02	F-063	export range re-import identity	L2
TC-CAL-edge-01…06	§7.14	register rows	L1/L4

A.7 Files / backup

Case	Binding	Verifies	Layer
TC-FILE-browse-01	F-065	private+SAF ops incl. rename/move/search	L4
TC-FILE-saf-01	F-065	revoked grant card + re-grant	L4
TC-FILE-anlz-01	F-066	donut correctness on fixture tree + yields	L2
TC-SEC-backup-01	F-067	round-trip restore identity	L4
TC-SEC-backup-02	F-067	wrong passphrase: clean fail, no partial state	L4
TC-SEC-backup-03	§14.1	traversal/bomb fixtures rejected	L2
TC-FILE-trash-01	F-068	trash lifecycle + shred	L2
TC-FILE-edge-01…05	§7.14	register rows	L2/L4

A.8 Music / Recorder

Case	Binding	Verifies	Layer
TC-MUSIC-imp-01	F-069	SAF/MediaStore import diff, no dupes	L4
TC-MUSIC-play-01	F-070	queue/shuffle/repeat/speed/gapless	L4
TC-MUSIC-foc-01	F-070	audio focus + headset route loss	L4
TC-MUSIC-pl-01	F-071	playlist CRUD + M3U export/import	L2
TC-MUSIC-mini-01	F-072	miniplayer persists across modules	L4
TC-MUSIC-edge-01…06	§7.14	register rows	L2/L4
TC-REC-svc-01	F-073	FGS ≤ 200 ms + notification actions	L4
TC-REC-mark-01	F-074	markers insert/seek; pause rejects flags	L4
TC-REC-fmt-01	F-075	AAC default + WAV option; files in Files	L2
TC-AI-asr-01	F-076	fixture WER ≤ 12%; segments monotonic	L4
TC-REC-note-01	F-077	→ Notes linked note content	L4
TC-REC-edge-01…06	§7.14	register rows	L1/L4

A.9 Diagnostics / Settings / Security / System

Case	Binding	Verifies	Layer
TC-DIAG-self-01	F-079	six-step script all green on clean build	L4
TC-DIAG-log-01	F-080	ring caps + scrub + share preview	L2
TC-PACK-mgr-01	F-081	import/enable/disable/delete lifecycle	L4
TC-PACK-sec-01	F-081/SC-06	hash mismatch disables + card	L2
TC-PACK-eval-01	§16.8	eval-kit thresholds met for pinned packs	L4
TC-SET-lock-01	F-082	app lock threshold + credential fallback	L4
TC-SET-about-01	F-083	provenance matches release bundle	L2
TC-SET-lic-01	F-084	licenses regenerated, complete	L1
TC-SEC-perm-full	F-034/SC-01	full-app manifest allow-list exact	L1
TC-SEC-perm-flavor	F-038	flavor manifest zero permissions	L1
TC-SEC-dex	SC-03	no network-class symbols	L1
TC-SEC-clip-01	SC-07	clipboard clear	L4
TC-SEC-scrub-01	SC-08	PII scrubber battery	L1
TC-SEC-lock-01	SC-09	app-lock round trip	L4
TC-SYS-tokens	§6.7	Kotlin tokens == tokens.csv	L1
TC-ENV-01…16	§3.5	environment matrix rows	L4/L5
TC-NAV-001	UX-001	two-tap traversal audit	L2
TC-STATE-storm	J-06	process death restoration	L4
TC-STATE-mig-01	§29.2	migration fixture round trip	L2
TC-STATE-bakdrift	§29.2	newer backup rejected cleanly	L2

A.10 Journeys (L4/L5)

J-01…J-10 as specified in §9.12, each with step tables in the harness YAML and per-step screenshots; J-09 pins flavor parity; J-10 uses the low-storage image.

Appendix B — Worked calculator traces (normative examples)

Fixed prefix of every law-suite corpus (TC-LAW-*). "Path" = arithmetic route (§17.1.2). Internal value shown exactly (rational) or to 12 significant digits (inexact flag).

#	Input	Tokens	Path	Internal value	Rendered
B-01	`2+3×4`	2 + 3 × 4	exact	14	`14`
B-02	`(2+3)×4`	(2 + 3) × 4	exact	20	`20`
B-03	`2(5+2)`	2 (5 + 2)	exact (implicit ×)	28	`28`
B-04	`(2)(3)`	(2)(3)	exact	6	`6`
B-05	`200+10%`	200 + 10 %	exact	220	`220`
B-06	`200−10%`	200 − 10 %	exact	180	`180`
B-07	`200×10%`	200 × 10 %	exact	20	`20`
B-08	`200÷10%`	200 ÷ 10 %	exact	2000	`2,000`
B-09	`50%`	50 %	exact	1/2	`0.5`
B-10	`200+10%+10%`	200 + 10% + 10%	exact	242	`242`
B-11	`5+ =`	5 + ⇒ repeat	exact	10	`10` (owner bug class killed)
B-12	`5+ = =`	repeat chain	exact	15	`15`
B-13	`10×−`	10 × −(…)	exact	−10	`-10`
B-14	`2^3^2`	2 ^ 3 ^ 2	exact	512	`512`
B-15	`(2+3 =`	balance	exact	25	`25`
B-16	`0.1+0.2`	exact rationals	exact	3/10	`0.3`
B-17	`1÷3×3`	exact	exact	1	`1` (owner bug class killed)
B-18	`1e15` then `+1 =`	continuation uses internal exact	exact	1000000000000001	`1,000,000,000,000,001` (display may sci-format; continuation exact — owner bug class killed)
B-19	`123456`	literal	exact	123456	`1,23,456` (en-IN)
B-20	copy of B-19 result	clipboard	—	`123456`	raw digits, no commas
B-21	`sin(90)` DEG	identity reduction	exact	1	`1`

#	Input	Tokens	Path	Internal value	Rendered
B-22	`cos(0)`	identity	exact	1	`1`
B-23	`tan(90)` DEG	domain	—	—	`Error` card: undefined
B-24	`asin(0.5)` DEG	exact-by-table (special angle)	exact	30	`30`
B-25	`sin(π÷6)` RAD	inexact	inexact≈0.4999999999999994 → reduction	1/2 by identity table	`0.5`
B-26	`√2`	inexact	inexact	1.414213562373…	`1.414213562`
B-27	`ln(−1)`	domain	—	—	`Error` card: outside domain
B-28	`5÷0`	domain	—	—	`Error` card: cannot divide by zero
B-29	`2+3×(4+1 =`	balance	exact	25	`25`
B-30	`1/2π`	1 ÷ 2 π implicit	exact	1/2 π → inexact on π	`1.570796327`

Appendix C — Recurrence golden corpus (representative rows)

Full corpus: `core/domain/test/fixtures/rrules.json` (200 entries; these 24 are the pattern anchors). Each entry: rule, start (local), expected occurrence list (first 6), notes.

#	Rule (relative to start)	Start	First occurrences	Note
C-01	FREQ=DAILY	01-Jan-26	01,02,03,04,05,06 Jan	base
C-02	FREQ=DAILY;INTERVAL=3	01-Jan-26	01,04,07,10,13,16	interval
C-03	FREQ=DAILY;COUNT=4	15-Mar-26	15,16,17,18 Mar	count cap
C-04	FREQ=DAILY;UNTIL=20260110	01-Jan-26	01..10 Jan	until inclusive
C-05	FREQ=WEEKLY	Mon 05-Jan-26	05,12,19,26 Jan…	anchor weekday
C-06	FREQ=WEEKLY;BYDAY=MO,WE,FR	05-Jan-26	05,07,09,12,14,16	multi-day
C-07	FREQ=WEEKLY;INTERVAL=2;BYDAY=TU	06-Jan-26	06,20 Jan,03 Feb	biweekly
C-08	FREQ=MONTHLY	31-Jan-26	31 Jan,31 Mar,31 May	31st skips short months
C-09	FREQ=MONTHLY (clamp setting)	31-Jan-26	31 Jan,28 Feb,31 Mar	clamp variant
C-10	FREQ=MONTHLY;BYMONTHDAY=-1	31-Jan-26	31 Jan,28 Feb,31 Mar	last-day
C-11	FREQ=MONTHLY;BYMONTHDAY=15	15-Feb-26	15 Feb,15 Mar…	fixed day
C-12	FREQ=YEARLY	29-Feb-26*	(*2028 test year) 29 Feb,29 Feb	leap anchor
C-13	FREQ=YEARLY;INTERVAL=2	10-Jun-26	10 Jun 26,28	biennial
C-14	FREQ=DAILY;EXDATE=20260103	01-Jan-26	01,02,04,05…	exdate
C-15	FREQ=WEEKLY;BYDAY=MO;EXDATE 3 rows	05-Jan-26	skips listed Mondays	multi-exdate
C-16	FREQ=MONTHLY;COUNT=12 over year boundary	15-Aug-26	…12 total	count over years
C-17	DST spring gap US 2026-03-08 02:15 daily	07-Mar-26	02:15 → 03:15 local post-gap	offset policy
C-18	DST fall overlap 2026-11-01 01:30 daily	31-Oct-26	disambiguated to first instance	offset policy
C-19	all-day weekly	05-Jan-26	dates only, no time shift	all-day + tz-free
C-20	FREQ=DAILY in Kolkata tz over DST-irrelevant zone	01-Jan-26	fixed 09:00 IST	non-DST zone sanity
C-21	BYDAY with INTERVAL misaligned anchor	06-Jan-26 (Tue, interval 2)	06,20 Jan	anchor=first occurrence
C-22	UNTIL before start	01-Jan-26; UNTIL=20251231	empty series	empty legal
C-23	COUNT=1	04-Jul-26	04 Jul only	single
C-24	FREQ=YEARLY;BYMONTHDAY=29;BYMONTH=2	29-Feb-28	leap years only	explicit leap

Appendix D — Owner-guide coverage map (B/C/D/E → requirements)

Each historical guide section maps to requirements and tests — this is how the blueprint guarantees the owner's past acceptance language is preserved, extended, and never silently dropped.

Guide item	Intent	Requirement(s)	Test(s)
B1 hold `123456` → highlight + pill	selection entry	F-012	TC-ONE-C-012
B2 hold-then-slide resize	gesture	F-013	TC-ONE-C-B2 (device-class)
B3 boundary handles	(old-guide)	superseded by B2 char-wise resize; documented in ledger D-002	—
B4 copy/paste no commas	clipboard sanitize	F-014	TC-ONE-C-018
B5 cut + type into gap	edit flow	F-015	TC-ONE-C-018
B6 tap clears menu, places caret	selection exit	F-016	TC-ONE-C-017
B7 AC clears selection	staged clear	F-017	TC-ONE-C-016
B8 selection vs scroll	gesture priority	F-012 (detector spec §7.2.2)	TC-CALC-edge
B11 empty caret menu	insertion	F-018	TC-ONE-C-019
B12 MAdd/MUse mid-expression	memory	F-019/20	TC-ONE-C-015
C1 long expr + live answer	field	F-010	TC-ONE-C-010
C4 `10×−` unary	parser	F-005	TC-ONE-C-005
C5 `200+10%`	percent	F-003	TC-ONE-C-003
C6 `5+3×2=11`, auto-balance	engine	F-001/6	TC-ONE-C-001/006
C8 caret visible + autoscroll	field	F-011	TC-ONE-C-011
C9 no device keyboard	input mode	F-033	TC-ONE-C-025
D1 `2(5+2)=28`	parity	F-002	TC-ONE-C-002
D2 `10+` then `−`	operator replace	F-004	TC-ONE-C-004
D3 `5+ =` → 10	trailing-op	F-004	TC-ONE-C-004
D4 `1,23,456` grouping	rendering	F-007	TC-ONE-C-007
D6 INV sin degrees	functions	F-009	TC-CALC-deg-01
D8 backspace one char	edit	F-011	TC-ONE-C-011
D9 history replay paint-then-close	history	F-022	TC-ONE-C-014
D10 replay race guard	robustness	F-023	TC-ONE-C-014
E1 history newest-on-top	history	F-021	TC-ONE-C-013
E2 swipe-down opens history	entry points	F-021	TC-ONE-C-013 (entry matrix)
E3 3-dot: five items + Effects	menu exactness	F-032/F-026	TC-ONE-C-022/020
E4 effects ≥ 4 styles	effects	F-031	TC-ONE-C-024
H1–H4 themes/caret	theming	F-035/F-027	TC-SYS-theme-01/TC-ONE-C-023

Guide item	Intent	Requirement(s)	Test(s)
F MyCalc zero permissions	flavor	F-036…040	TC-SEC-perm-flavor/J-09

PART X

TRACEABILITY, RISK AND ACCEPTANCE

The maps that keep the program honest: requirement-to-test-to-commit traceability, local observability, the maintenance policy, the live risk register, the glossary, and the final acceptance script the owner walks at G7.

Chapter 27 — Traceability matrix

27.1 Mechanism

`tools/traceability.py` scans three sources and emits `reports/traceability.csv`:

1. This blueprint (F/UX/TC/§ ids by regex).
2. Source headers and commits (`Refs:` trailers).
3. Test results JSON (latest full-lane run).

Invariants enforced (CI `traceability` check):

- Every F-xxx has ≥ 1 TC and ≥ 1 commit.
- Every TC has ≥ 1 F/UX binding and a latest status.
- Every § cited in code exists in the blueprint.
- No TC references a nonexistent F (and vice versa).
- P0 features cannot hold status `not-implemented` at G7.

27.2 Matrix shape (excerpt)

```
feature,area,priority,spec,tests,latest_status,commits,notes
F-001,CALC,P0,§17.1,TC-ONE-C-001;TC-LAW-002,pass,ab12ef0;ff9012a,
F-013,CALC,P0,§7.2.2,TC-ONE-C-B2,blocked,ab12ef0,physical-device input
F-038,NCALC,P0,§14.4,TC-SEC-perm-flavor,pass,1c23d40,
```

The matrix is regenerated per gate and embedded in gate reports; the release bundle ships the final one (S5).

27.3 Traceability script outline (tools/traceability.py)

```
1. parse blueprint md  → ids {F,UX,TC,§} with locations
2. parse git log range → commits + Refs trailers
3. parse qa/testcases.csv → case → bindings
4. parse latest results JSONs → case → status
5. join; assert invariants (§27.1); emit CSV + markdown summary
6. exit nonzero on any violation (CI traceability check)
```

Chapter 28 — Observability and diagnostics (local-only)

28.1 Principles

Observe without transmitting: all telemetry-equivalents are local, scrubbed, size-capped, user-visible, and user-exportable. If a data point cannot be shown to the user in Diagnostics, it is not collected.

28.2 What is recorded (the local ring)

Signal	Where	Retention
Crash/uncaught-exception records (stack, thread, recent breadcrumbs)	`logs/crash/`	5 latest
ANR watches (main-thread block > 5 s during user action)	ring	7 days
AI job records (capability, pack, duration, memory, fallback rung used, cancel source)	ring + per-note provenance	7 days
Repository timings p50/p95 (per operation class)	ring (aggregates only)	7 days
Feature usage counters (module opens, actions) — coarse, per-install	DataStore	until cleared
Alarm scheduler outcomes	ring	7 days

`adb logcat` during a QA run is captured by the harness (§22.5) — the app itself never streams anywhere.

28.3 Scrubbing rules

Ring scrubber reduces absolute paths to basenames, strips phone-number/email patterns (regex), keeps calculator expressions verbatim (product content), and replaces account names (`never present anyway`). The share preview shows the exact final text (F-081). Unit tests pin scrubber behavior (TC-SEC-scrub).

28.4 Self-test and health computation

§17.8 defines the script; health cards derive from the same primitives (storage %, quick_check, pack hashes, alarm capability). Self-test is synchronous (≤ 10 s) with per-step progress; results are artifacts (`diagnostics/selftest-<ts>.json`), one tap to share.

28.5 Diagnostics ring schema (normative)

```
CREATE TABLE diag_ring (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  ts INTEGER NOT NULL,            -- epoch millis
  kind TEXT NOT NULL,             -- crash|anr|ai|repo|alarm|feature
  severity TEXT NOT NULL,         -- info|warn|error
  module TEXT NOT NULL,
  message TEXT NOT NULL,          -- scrubbed
  detailJson TEXT                 -- structured extras (timings, rung, ids)
);
CREATE INDEX idx_diag_ring_ts ON diag_ring(ts DESC);
```

Retention sweep: on startup keep `severity=error` rows 7 days, `info/warn` 3 days, hard cap 2 MB (oldest evicted first). Export = JSON Lines, previewed verbatim before sharing (F-081). Feature counters live in DataStore under `stats.*` keys, viewable in Diagnostics → Usage

(module opens, actions per module) — coarse, local, clearable.

28.6 Self-test JSON example (artifact format)

```
{"ts": 1769900000123, "app": "3.13.0", "commit": "9c4e…",
 "checks": [
  {"id": "engine",  "ok": true,  "detail": "100 law-smoke cases, seed 7"},
  {"id": "db",      "ok": true,  "detail": "quick_check ok; 10 tables round-trip"},
  {"id": "packs",   "ok": true,  "detail": "essentials@1.2.0 sha ok; speech@1.1.0 sha ok"},
  {"id": "alarms",  "ok": true,  "detail": "canScheduleExactAlarms=true"},
  {"id": "notif",   "ok": true,  "detail": "4 channels present"},
  {"id": "storage", "ok": false, "detail": "412 MB free (<500 warn)"}],
 "result": "WARN"}
```

Result classes: PASS (all green), WARN (warn-level fails only), FAIL (any error-level). The Diagnostics card mirrors this JSON verbatim — what you see is what would be shared.

28.7 Feature counters (Diagnostics → Usage)

Counter	Meaning
`opens.<module>`	module entries
`calc.solves`	successful = presses
`notes.created/edited`	notes lifecycle
`scan.captures/ocrs`	scanner activity
`cal.events/alarms_fired`	calendar activity
`music.plays_seconds`	listening time
`rec.recordings/transcribed`	recorder activity
`ai.jobs.<cap>.<rung>`	AI usage incl. fallback rung histogram

Counters are additive integers, clearable, exported in the log bundle header — enough to see what's used, too coarse to be sensitive.

Chapter 29 — Maintenance and update policy

29.1 Code health practices

No orphan code: detekt + unused-module scan per gate; dependency refresh window quarterly (post-G7) with ledger entries; blueprint drift check at each minor release (§ refs validated); the task graph closes at G8 and a v1.1 graph seeds from the defect ledger + P2 backlog.

29.2 Data evolution

Room migrations only forward; every migration gets a fixture test (§21.3 backups as inputs); FTS rebuild command for catastrophic index cases; `indexes/vectors.bin` versioned with a rebuild path; backup format versioned — older-app-rejects-newer-backup with a clear message (STATE case).

29.3 Model pack evolution

Packs version independently of the app; `catalog.csv` records supported pack versions per app version; a pack too new is rejected with "update the app" guidance; a pack too old runs with an eval-waiver note in Diagnostics. Eval kit (§16.8) re-runs per accepted pack version — results versioned in-repo.

29.4 Documentation maintenance

This blueprint is versioned with the code; every merged feature PR must land its spec delta in the same merge (docs-drift check §24.4); the changelog accumulates in `RELEASE-NOTES.md`; the defect ledger is never pruned (history is evidence — the owner's standing practice).

29.5 Model/pack update decision tree (operational)

```
New pack version published by owner
├─ Eval kit green (§16.8 thresholds)
│    ├─ App catalog.csv supports it → mark supported; note in ledger; users import at will
│    └─ App catalog too old → app update release adding catalog row (minor bump)
└─ Eval red → do not distribute; record eval JSON; fix or pin previous version
App update with behavior-affecting prompts (§16.12)
└─ Eval re-run mandatory before merge (docs-drift check enforces version bump)
```

29.6 Deprecation policy (in-app)

Removed settings keys migrate once with documented defaults (registry version row); removed features surface a one-line "moved/removed" card in Diagnostics → Changelog for one minor release. Nothing silently disappears — small product, high trust.

29.7 Changelog discipline example (RELEASE-NOTES entry quality)

```
### Fixes
- Calculator: `5 + =` now yields 10 (pending-operator repeat, Google parity). [D-014]
- Recorder: pauses cleanly on incoming calls; file kept to the pause point. [D-021]

### Honest register
- Blocked during QA: stationary hold-then-drag selection needs a physical device
  (emulator input API limitation). Reported as BLOCKED, not failed.
- Known limitation: ICS import supports the documented RRULE subset; exotic rules
  import as one-off events with a note.
```

Every bullet carries a ledger id and the honest register is never empty — if nothing is blocked, the line says "None this release" (the absence is the claim).

29.8 Maintenance calendar (steady state)

Cadence	Items
Per release	pre-flight §25.4; notes; ledger triage
Monthly	defect sweep; dependency advisories read (no auto-bumps)
Quarterly	dependency refresh window; blueprint drift audit; eval kit re-run if packs changed
Yearly	targetSdk review; minSdk decision; keystore health check

Chapter 30 — Risk register (live)

ID	Risk	Likelihood	Impact	Mitigation	Trigger/response
R-001	Calculator parity divergence on a subtle rule (percent chains, trailing-op)	M	High (owner trust)	§17.1 normative tables + laws L1–L10 + Catalogue-0	Any Catalogue-0 red = release stop
R-002	Emulator-only input limits leave B2-class gestures unverified	High	Medium	BLOCKED-by-rule policy (§21.2); harness runs on physical devices on demand	Owner device session at G6/G7
R-003	AI pack distribution friction (sizes, sideload confusion)	M	Medium	Pack manager UX with sizes + SHA display; owner-pack README	G5 usability check
R-004	LLM output quality below rubric on low-end tiers	M	Medium	Fallback ladder (§16.7) keeps features useful; assistant pack is Tier A only	Eval kit thresholds
R-005	Size budget breach from media/AI libs	M	Medium	Budgets assert in CI (§23.6); ABI filtering; pack-off builds	CI fail → trim task
R-006	Room migration data loss in the field	Low	Critical	Fixture migration tests; transactional restore; backup-before-migrate on schema jumps	Any fixture red = migration rework
R-007	Exact-alarm policy drift across OEMs	M	Medium	Inexact fallback + one-time notice (§13.8); reminder smoke on E33/E35	OEM report → channel guidance
R-008	SAF quirks across OEM file managers	M	Medium	SAF-can't-do map + copy-first semantics (§13.6); device smoke at G6	OEM bug → workaround task
R-009	Agent drift from spec over long program	M	High	This document's load-on-phase + traceability + gate reports (§15.4–15.5)	Gate report audit by owner
R-010	Keystore loss after first release	Low	Critical	Owner backup action item at G1 with verified restore rehearsal	G1 checklist item
R-011	Scope creep via "helpful" additions	M	Medium	§1.4 forever-list; ledger discipline; owner cutline control (§10.3)	Any unplanned feature = rejected PR
R-012	Performance regressions as modules land	M	Medium	Budgets in CI from G2 (fail-fast); Macrobenchmarks per module	Budget red → fix before next task

30.1 Early-warning indicators (watched per phase)

Indicator	Green	Watch	Act at
Test flake rate	< 0.5%	0.5–1%	≥ 1%: quarantine sweep task
Build time p50	< 5 min	5–8 min	> 8 min: module-parallelize
APK size trend/phase	< +1.5 MB	+1.5–3 MB	> 3 MB: budget review task
Defect discovery slope	falling	flat	rising near G6: hardening extends
Blocked register size	shrinking	flat	growing: owner session scheduled
Ledger churn in one area	< 3/week	3–5/week	> 5/week: spec gap — fix blueprint
CI job count/run	≤ 24	25–28	> 28: collapse matrix before cap

R-013	OCR accuracy complaints on real-world handwriting	M	Medium	Scope honestly printed/hand-print Latin v1; confidence badges; roadmap non-Latin	Eval fixtures extended by owner samples
R-014	Recorder FGS killed by aggressive OEM battery managers	M	Medium	FGS + notification contract; OEM whitelist guidance in runbook	OEM report → guidance update
R-015	Backup passphrase forgotten by end users	High	High	Warning at creation (Appendix E copy); optional passphrase hint (stored app-side, clearly weaker)	Support playbook entry
R-016	LLM hallucinated facts in summaries	M	Medium	Preview-only UX; extractive fallback; "never invent facts" prompt + rubric	Rubric red → prompt/pack rework
R-017	Bundle size of docs/evidence grows unbounded	M	Low	Retention §20.11; generated artifacts out of Git except release bundles	Repo size alert
R-018	Timezone bugs in recurrence/reminders	M	High	ADR-08 offsets; DST golden corpus (C-17/18); reminders use UTC alarm + local render	Any corpus red = S2 fix

30.2 Risk heat map (likelihood × impact placement)

	Low impact	Medium impact	High impact
High likelihood	—	R-002 · R-015	—
Medium likelihood	R-017	R-001 · R-003 · R-004 · R-005 · R-007 · R-008 · R-009 · R-011 · R-012 · R-013 · R-014 · R-016 · R-018	—
Low likelihood	—	—	R-006 · R-010

Reading: the register is deliberately crowded in medium/medium — the honest state of any greenfield build; the two high-impact lows (R-006 migration loss, R-010 keystore loss) get disproportionate mitigation (fixture tests; G1 rehearsal) because their cost is catastrophic and their prevention cheap.

30.3 Risk response drills (rehearsed, not just written)

Drill	When	Pass
Restore-from-backup on a clean device	G4, then every release	counts+hash match
Keystore restore rehearsal	G1, then yearly	signed test APK verifies
Storage-full drill (J-10)	G4, every release	graceful refusals
Alarm-miss drill (Doze)	G5, yearly	fires within window or fallback notice
Process-death storm (J-06)	every gate from G3	state restored

Chapter 31 — Glossary

Term	Meaning
App pinning / sideload	Installing an APK directly (no store); the owner's distribution model
BLOCKED	A test state: environment/input prevented a fair assertion; never counted as pass; never a defect by itself
Catalogue-0	The release-blocking test set (§20.6), descendant of the owner's guide cases
Canonical form	ASCII-normalized expression stored/compared/exported (§17.1.3)
Decision ledger	Append-only record of choices not dictated by this document (D-nnn)
Dist APK	A distribution APK actually given to users (what L5 tests)
Evidence manifest	JSON listing every artifact file with SHA-256 + size, verified post-write
Exact path	BigRational arithmetic where +−×÷ and integer ^ stay exact (§17.1.2)
FTS	SQLite full-text search (FTS4 here, ADR-02)
Gate (G0–G8)	Phase boundary that opens only on demonstrated exit criteria (§15.5)
Hub	The combined app's home screen (§5.3)
Live answer	As-you-type evaluation under the expression (§7.2.1)
MAdd / MUse	Memory add/use selection actions storing/replacing numeric values (§7.2.2)
Pack	Side-loaded AI model bundle, SHA-verified, never downloaded (§16.5)
P0/P1/P2	Priority tiers: release blocker / required / deferrable (§9.1)
Provenance	The who-built-what-when chain: commit, run id, APK SHA-256 (C10)
Publisher success ≠ pass	A green publish job says nothing about case results (field lesson)
RRULE	ICS recurrence rule; the supported subset is normative (§17.4)
SAF	Storage Access Framework document model (§13.6)
Shard	A CI runner slice of a test suite (3-runner cap, C8)
Tier A/B/C	Device capability classes (§3.2) governing AI scope
Traceability matrix	F↔TC↔commit mapping generated per gate (§27)
Trash	30-day soft-delete for private items (§14.3)
Keyset pagination	next-page queries by last-seen key (no OFFSET scans)
Predictive back	Android 13+ back-gesture preview
Baseline profile	precompiled code paths shipping in the APK for fast startup
Macrobenchmark	system-level startup/scroll measurement harness
NNAPI	Android's on-device neural acceleration interface
int8 quantization	8-bit model weights — smaller and faster, small accuracy cost
WER	word error rate (ASR quality metric)
Rubric	fixed scoring guide for LLM output quality
RRULE/ICS	iCalendar recurrence rule / calendar exchange format
Property-based test	randomized inputs checked against stated laws
Golden file	recorded-expected artifact diffs must justify changes against

Term	Meaning
Gate report	one-page phase-closure evidence summary (§15.9)
Evidence bundle	manifest-verified artifact directory shipped with releases
Keyset cursor	the `updatedAt < cursor` value carried in list pages

Chapter 32 — Final acceptance and sign-off (G7 script)

The owner-facing acceptance walk, in order, each step against real artifacts:

1. **Identity**: install `myspace-one` release APK on a device/emulator → About shows version + commit + APK SHA-256 matching the release bundle.
2. **Privacy proof**: manifest shows zero internet-class permissions (S2); airplane mode on; full smoke (J-05) passes.
3. **Calculator**: run Catalogue-0 CALC set (or read the L5 PDF of record); spot-check TC-ONE-C-004 (`5+ =` → `10`), C-008 (`1÷3×3` → `1`), C-023 (theme diff) by hand.
4. **Flavor**: install `mycalc` APK → zero permissions (F-038), parity spot-check, no module remnants.
5. **Modules**: one journey each (J-01…J-10) or gate-report review with artifact paths.
6. **AI**: with packs → OCR a fixture, transcribe a fixture, summarize a note; without packs → the same features degrade exactly per §16.7; pack manager rejects the tampered fixture pack.
7. **Data**: backup → wipe → restore (J-03); migration fixture log reviewed.
8. **A11y/UX**: UX-001…UX-020 checklist review with screenshots.
9. **Evidence**: traceability matrix shows all P0 green; defect ledger has no open S1/S2; blocked register reviewed and accepted.
10. **Sign-off**: owner accepts (or files defects); G7 report marked ACCEPTED with date; release published per §25.5.

Closing note to the executing agent. This document asked you to build one honest thing: a private, offline, combined tool that works exactly as specified, provable at every step. When you must choose between shipping more and proving what you shipped, prove it. The evidence bundle is the product's shadow — keep it complete, keep it honest, and the owner never needs to trust you.

32.1 Sign-off record (filled at G7)

Field	Value
Release version / code	____ / ____
Blueprint version against which built	____
APK SHA-256 (full / mycalc)	____ / ____
CI run id (release lane)	____
Catalogue-0 result	____ pass / ____ fail / ____ blocked
Blocked register accepted by owner	yes / no (+notes)
Open S1/S2	must be 0 / ____
Owner signature / date	____

32.2 The G7 walkthrough, narrated

The owner starts with identity (step 1) because every later proof depends on "is this the build I think it is": the About screen SHA must equal the release-notes SHA must equal the CI artifact SHA — three independent views of one truth. Privacy (step 2) is proven, not promised: airplane mode is toggled by hand and the manifest is read on-device. Calculator (step 3) is the emotional core — the owner replays the exact behaviors their old guides named, including the two historical bugs (`5+ =`, `0.999…`), now expected dead. The flavor (step 4) proves the product story ("a calculator, nothing else") in one permission screen. Modules (step 5) ride the journey evidence; the owner may re-run any single journey by hand in minutes. AI (step 6) shows the ladder honestly: capabilities on, capabilities off, same features usable. Data (step 7) is the trust moment: backup, wipe, restore, identical counts. A11y/UX (step 8) rides screenshots plus one TalkBack pass. Evidence (step 9) is where the program's honesty is auditable: traceability green, ledger closed, blocked register — if any — owned and accepted. Step 10 is a date and a signature. If any step surprises the owner, the release stops; surprise is a defect of the process, not of the owner.

32.3 Common acceptance questions (owner asks / spec answers)

"Prove it's not phoning home." Manifest has no INTERNET permission (S2 artifact); dex scan shows no socket classes (SC-03); airplane-mode journey J-05 green on the exact APK; the app cannot — structurally — transmit.

"Why should I trust the green report?" The report builder reads only manifest-verified artifacts; every case row carries screenshots and timings; the harness code is in-repo and re-runnable against any APK by hand (§22.13).

"What if a gesture wasn't tested fairly?" The blocked register names it, the prevention, and the unblock path; BLOCKED never counts as pass (§15.6) — and physical-device runs use the same harness (§21.8).

"How do I know updates won't break my data?" Migration fixtures run old backups/schemas through new code (STATE-mig), install-over is itself a tested case (TC-SEC-upgrade), and backups restore transactionally or not at all (J-03).

"Where do the AI models come from?" You install packs yourself from files with published SHAs; the app verifies hashes at import *and* at load; it never downloads anything (C5, §16.8).

"What's my escape hatch?" Export everything (§26.3), unencrypted option; notes export as Markdown; playlists as M3U; events as ICS — your data leaves in open formats, on your

terms.

Appendix E — UX microcopy dictionary (normative strings v1, en-IN)

Copy rules: sentence case; no exclamation marks; no emoji; "phone" not "device" in user copy; numbers humanized. The strings table below is generated into `core/designsystem/strings.csv`; a lint test asserts feature code references keys, never literals (TC-SYS-strings).

Key	String
hub.firstRun.privacy.title	Your data stays on this phone
hub.firstRun.privacy.body	MySpace One has no internet permission. Nothing you do here leaves this phone.
hub.firstRun.start.body	Everything works offline. Start with Calculator, or import music and files from your storage.
hub.search.hint	Search notes, scans, events, files, music…
hub.recovered	Recovered — tap to reopen
calc.history.empty	Solved calculations appear here.
calc.history.clear.confirm	Clear history? Pinned cards are kept.
calc.memory.empty	Memory is empty. Select a number and tap MAdd.
calc.err.divideByZero	Cannot divide by zero
calc.err.domain	Outside domain (e.g., log of a negative number)
calc.err.syntax	Can't calculate this yet — check the highlighted part
calc.err.tooLong	Expression is too long
calc.err.overflow	Result is too large to display
notes.empty	Notes you write are saved on this phone automatically.
notes.trash.snackbar	Note moved to trash
notes.trash.undo	Undo
notes.trash.empty.confirm	Empty trash now? Items are deleted permanently.
notes.ai.missing	Install the Essentials pack in Diagnostics to use AI actions.
notes.ai.running	Working…
notes.ai.applied	AI result applied — Undo restores your original text
scan.ocr.missing	OCR needs the Essentials pack. Capture and export still work.
scan.import.title	Import image or PDF
scan.dup	Page 2 looks like page 1
cal.alarm.exactNotice	Reminders may be a few minutes late because exact alarms are restricted.
cal.ics.simplified	Recurrence simplified from this calendar file
files.saf.revoked	Access removed — tap to re-grant this folder
files.lowStorage	Storage is low — recordings and scans may fail.
music.unplayable	Can't play this file
music.sleep.set	Sleep timer set — playback will stop gently
rec.permission.body	Audio is stored on this phone only. Nothing is uploaded — MySpace One has no internet permission.
rec.paused.byCall	Paused — phone call took the microphone
rec.incomplete	Recording stopped early — saved up to here
diag.selftest.pass	All 6 checks passed

Key	String
diag.selftest.failBody	%1$d of 6 checks failed
diag.share.preview.title	This is exactly what will be shared
settings.about.provenance	Built from commit %1$s · %2$s
backup.create.passphraseWarn	If you forget this passphrase, the backup cannot be opened — by anyone.
generic.error.storage	Couldn't save — storage is full
generic.done	Done

Appendix F — Error catalogue (AppError → user message + action)

Error class	Trigger examples	User message	Action
Storage.Full	write fails ENOSPC	generic.error.storage	Open Files (Clean up)
Storage.SafRevoked	grant dead	files.saf.revoked	Re-grant
Storage.Corrupt	quick_check fail	Database needs attention	Open Diagnostics
Input.Syntax	parser	calc.err.syntax	Edit expression
Input.Domain	evaluator	calc.err.domain	Edit expression
Input.DivZero	evaluator	calc.err.divideByZero	Edit expression
Input.TooLong	512 cap	calc.err.tooLong	Shorten expression
Policy.AlarmDenied	exact alarms revoked	cal.alarm.exactNotice	Open reminder settings
Policy.MediaDenied	read-media denied	music import card	Grant / pick folder
Model.PackMissing	capability w/o pack	feature-specific "needs pack"	Open Diagnostics
Model.PackCorrupt	hash mismatch	Pack disabled — file damaged	Re-import pack
Model.Budget	wall/mem cap hit	basic result shown	(fallback applied silently, provenance recorded)
Backup.BadPassphrase	GCM tag fail	Passphrase doesn't match this backup	Try again
Backup.BadArchive	schema/guards	This backup can't be opened by this version	Check version / Docs
Internal.Unexpected	catch-all	Something went wrong — details in Diagnostics	Open Diagnostics

Appendix G — Model pack format (normative)

```
<pack-id>-<version>.mspack (zip, deflate)
  pack.json          # manifest, UTF-8
  models/…           # model files exactly as manifested
  LICENSE            # full license text
```

```
{
  "id": "essentials",
  "version": "1.2.0",
  "minApp": "3.13.0",
  "minTier": "C",
  "signature": "ed25519:base64…",
  "models": [
    {"file": "models/det.onnx", "sha256": "…", "runtime": "onnx",
     "inputs": {"image": [1,3,960,960]}, "notes": "DBNet-lite"},
    {"file": "models/rec.onnx", "sha256": "…", "runtime": "onnx"},
    {"file": "models/minilm.int8.onnx", "sha256": "…", "runtime": "onnx"}
  ]
}
```

Rules: unknown manifest fields are ignored (forward compat); `signature` verified against the baked-in public key when present (owner-distributed packs are signed); import = hash-verify all files → atomic move → registry transaction; the pack manager displays id, version, total size, tier, and per-model purpose labels; load-time re-verify per §16.8. Rejection reasons surface verbatim from the guard that fired (size, hash, schema, tier).

Appendix H — Gesture and control-equivalence map

Every gesture (X: no gesture-only features) with its visible control twin:

Gesture	Where	Equivalent control
Swipe down on expression	Calculator	History button
Hold-then-slide on text	Calculator selection	(no twin: long-press + selection handles serve same result; B2 is an enhancement)
Swipe between months	Calendar	‹ › buttons
Drag sheet handle	All sheets	Tap outside / back
Swipe-to-delete rows	Notes list, playlists	Row overflow → Delete
Long-press module icon	Hub	App shortcuts menu
Swipe MiniPlayer down	Global	Player close button
Drag selection handles	Calculator	Cut/Copy boundaries via re-long-press

Keyboard (hardware, tablets): all screens honor Tab order, Enter activates focused control, Esc = back; calculator accepts hardware digits/operators (still no IME popup).

Appendix I — Release runbook (operator steps)

1. Freeze: merge the release branch `release/v3.13.0`; version bump commit.
2. Tag `v3.13.0` → release lane runs: reproducible double-build, L5 campaign, docs regen.
3. Verify pre-flight §25.4 checklist output (all green; blocked register reviewed).
4. Owner walkthrough §32 steps 1–9; collect ACCEPTED (or defects → fix loop).
5. Publish: GitHub Release with bundle §25.5; attach APK SHAs to RELEASE-NOTES.
6. Post: announce per §26 (owner channels); defect ledger triage session; backlog v1.1 seeded from P2 + new findings; G8 report.

Rollback policy: apps are install-over; a broken release is superseded by the next patched build (no server to roll back — another benefit of the model).

Appendix J — Owner constraint register (conversation-derived)

The owner's standing instructions, mapped to normative sections (traceability of *why*):

Owner instruction (paraphrase)	Where enforced
App must work locally, on device, no sign-in, no internet	§1.2 P1/P2, §1.3 C1/C2, §14.2
No ads	§1.3 C3, F-034, §24.4 dependency audit
Combined all features into one app	§1.1, §2.1, Part II
Main priority = easy UX and compatibility	§4 principles, §3.2 tiers, §7.12 responsive
Prepare from scratch to fully tested working APK	S-criteria, G1–G7 lifecycle
Doc must include UI, features, usability, architecture, methodology, model integration, on-device AI	Parts II, III, IV, V, VII
All stages and cycles of the app	Part I→XI structure; G0–G8
200 pages	this document
Calculator only focus in QA (historical)	§22.1 scope rule
Publisher success ≠ passing result (historical lesson)	§15.8(3), §22.5
BLOCKED ≠ FAIL; no defect inference from blocked gestures (historical)	§15.6, §20.1, §21.2
Preserve historical reports; don't rewrite	§26/§29 ledger preservation; Part X maintenance
One evidence-rich link with screenshots/provenance	§22.5, §25.5
Max 30 jobs, ≤ 3 parallel runners	§1.3 C8, §24
Compare with official Google Calculator only (no AOSP substitute)	§22.6

Appendix K — Fixture inventory (generated, seeded)

Fixture set	Count	Generator notes
OCR images	40	printed+handprint Latin, 3 size classes, jitter affines, invoice-style subset
ASR clips	20	synthetic speech 5–30 s, clean/noisy mixes, reference transcripts
Note corpus	200	Markdown+NoteDoc pairs incl. checklists, headings, unicode, long docs
RRULE corpus	200	Appendix C anchors + randomized laws, DST cases
Music files	6	tones with complete metadata incl. edge tags
Backups	12	v1 real-manager outputs + 2 malicious-pattern archives
Model packs	3	well-formed, bad-hash, oversized
Eval kit	40+20	OCR images + notes with rubric answers (§16.8)
Perf corpora	1	10k-note DB seed, 5k-track library seed

Appendix L — Agent working templates

Task (task-graph row) — see §15.3 for fields. Defect entry:

```
### D-<n> — <title>
- Found by: TC-… / manual / review
- Severity: S1..S4 · Status: New
- Repro: steps (exact), environment, seed
- Evidence: reports/evidence/… paths
- Spec delta needed: §… (yes/no)
- Fix commit: <sha> · Verified by: TC-… re-run (date)
```

Decision ledger entry:

```
### D-<n> — <question>
- Date / phase
- Binding sections that could be affected: §…
- Options considered: a) … b) …
- Decision: …
- Consequence / revisit-if: …
```

Daily agent log (per session):

```
## Session <id> — date
- Phase: G… · Tasks touched: …
- Commits: …
- Tests: … (evidence paths)
- Blockers opened/closed: …
- Next: first open task in dependency order
```

Appendix M — Screen inventory (pointer)

The master screen inventory — now extended with Figma frame ids, states-to-build, and build-checklist duties — lives as **§6.14 (Screen inventory with Figma frame registry)** and is mirrored to `docs/blueprint/ui-screens.csv`. It supersedes the table that previously occupied this appendix (preservation note: the pre-1.1 table remains in Git history). Appendix M is retained as this pointer so cross-references from earlier revisions resolve.

Appendix N — Repository and DAO method inventory (executor contract)

Room DAOs (names normative):

DAO	Methods
`CalcHistoryDao`	`insert(row): Long`, `page(cursor, limit)`, `pinnedFirst()`, `count()`, `clearUnpinned()`, `pin(id, on)`, `lruEvictTo(n)`
`MemorySlotDao`	`all()`, `upsert(value, label)`, `delete(id)`, `move(id, up: Boolean)`
`NoteDao`	`upsert(note): Long`, `get(id)`, `pageList(cursor, limit)`, `searchFts(q, limit)`, `setPinned`, `setTrashed(now?)`, `purgeExpired(before)`, `counts()`
`ScanDocDao`	`insert`, `get`, `setOcr(id, text)`, `linkNote(id, noteId)`, `pageList`, `deleteWithFiles(repo)`
`CalEventDao`	`upsert`, `delete`, `window(start, end)`, `nextReminders(from, to)`, `byIcsUid`, `counts()`
`TrackDao`	`upsertAll`, `byFolder`, `search`, `markUnplayable`, `pageSongs/albums/artists`
`PlaylistDao`	`create`, `rename`, `delete`, `entries(id)`, `insert(entry)`, `move(entry, to)`, `remove(entry)`
`RecordingDao`	`insert`, `get`, `setTranscript`, `setTrashed`, `list`, `markers(recId)`, `addMarker`, `deleteMarker`
`DiagRingDao`	`append`, `recent(filter, limit)`, `sweep(now)`, `sizeBytes()`

Repositories add: transactional wrappers, dispatcher routing, and file-side effects (never in DAOs). Every method above appears in L1/L2 tests — the inventory is checked by a unit test that reflects over DAO interfaces (missing test = fail).

Appendix O — Icon inventory (design system)

60 icons max, Material Symbols subset (names normative): `home`, `calculate`, `description`, `scanner`, `calendar_month`, `folder`, `music_note`, `mic`, `monitor_heart`, `settings`, `search`, `close`, `add`, `more_vert`, `back_arrow`, `chevron_left_right`, `pin`, `delete`, `restore_from_trash`, `content_copy`, `content_cut`, `content_paste`, `memory_add`, `memory_use`, `backspace`, `function`, `percent`, `bracket`, `equal`, `swap_vert`, `drag_handle`, `filter`, `crop_rotate`, `torch`, `camera`, `image`, `picture_as_pdf`, `text_snippet`, `event`, `alarm`, `repeat`, `export`, `import`, `share`, `lock`, `key`, `palette`, `shapes`, `bubble_chart`, `waves`, `celebration`, `speed`, `bedtime`, `queue`, `skip_next_previous`, `play_pause`, `stop`, `flag`, `done`, `bug_report`, `verified`. Icons are lint-checked against usage (unused icon = removal PR).

Appendix P — Sample L5 "report of record" page order (normative)

1. Cover (title, versions, run ids, SHAs) · 2. Provenance & pinning table · 3. Version-alignment notes · 4. Executive result summary (counts by suite, honesty classes) · 5. Per-suite detail: case tables (id, status, actual, comment) with inline screenshots per case · 6. Blocked register with preventions · 7. Defect ledger delta · 8. Source-CI context labeled non-proof · 9. Environment matrix · 10. Evidence manifest summary (file counts, verification status) · 11. Changelog of the report itself. This order descends from the owner's accepted report formats — continuity over novelty.

PART XII

EXECUTION APPENDICES

Step-level scripts the executor runs verbatim: Catalogue-0 case scripts, journey step tables, engine reference tables, 30-second feature demos, severity/SLA matrices, an Android glossary for the owner, the executor quick-start, and document control.

Appendix Q — Catalogue-0 step scripts (executor-ready)

Environment stamp for every case: fresh install of the pinned APK, airplane mode ON (unless the case tests alarms), OCR reader enabled, screenshots at every assert. States: PASS / FAIL / BLOCKED per §20.1. Scripts use the harness verbs: `tap k`, `longpress region`, `ocr region`, `shot name`, `assert_ocr`, `adb cmd`, `install apk`, `start screen`, `reset state`.

Case	Script
TC-ONE-C-001	`tap 2 + 3 × 4 =` → `ocr display == "14"` · `AC` · `tap ( 2 + 3 ) × 4 =` → `== "20"`
TC-ONE-C-002	`tap 2 ( 5 + 2 ) =` → `== "28"` · `( 2 )( 3 ) =` → `"6"` · `2 π =` → `"6.283185307"`
TC-ONE-C-003	`200 + 10 % =` → `"220"` · `AC` · `200 × 10 % =` → `"20"` · `50 % =` → `"0.5"`
TC-ONE-C-004	`10 +` → shot · `tap −` → `ocr expr == "10−"` · `AC 5 + =` → `"10"` · `=` again → `"15"`
TC-ONE-C-005	`10 × − =` → `"-10"` · leading `− 5 =` → `"-5"`
TC-ONE-C-006	`( 2 + 3 =` → `"25"` · smart-key: at start `(` inserts pair; after `+` inserts `)` when open
TC-ONE-C-007	`123456` typed → `ocr expr == "1,23,456"` · select all, Copy → `adb clipboard == "123456"`
TC-ONE-C-008	`0.1 + 0.2 =` → `"0.3"` · `1 ÷ 3 × 3 =` → `"1"` (exact-path proof in law logs)
TC-ONE-C-009	`INV sin 0.5 ) =` (DEG) → `"30"` · `ln ( − 1 ) =` → error card text
TC-ONE-C-010	type `2+3` → live `"5"` ≤ 1 frame · type `+` → live disappears (no stale)
TC-ONE-C-011	`123456`, tap caret between 2 and 3 → type `9` → `"1293456"` grouped · `⌫` → `"123456"`
TC-ONE-C-012	longpress `234` (300 ms) → highlight + `ocr pill == "Cut Copy MAdd MUse Paste"`
TC-ONE-C-013	add 5 solves → `adb am force-stop` → relaunch → history count == 5, order exact
TC-ONE-C-014	open history, tap card → expr painted pre-close (frame log) · double-tap → single paint
TC-ONE-C-015	`3+4`, select `4`, memory has `123456` → `MUse` → `3+123456` · `=` → `"1,23,459"`
TC-ONE-C-016	selection on → `AC` → selection gone, expr remains · `AC` → empty
TC-ONE-C-017	selection on → tap elsewhere in expr → pill gone, caret at tap
TC-ONE-C-018	select `456` in `123456` → Cut → shot gap · type `0` → `"1230"` · paste → `"1230456"`
TC-ONE-C-019	clear expr → longpress empty → pill `MUse Paste` · MUse (mem `123456`) → `123456`
TC-ONE-C-020	overflow → Memory → sheet rows + MUse/MAdd/MC + up/down moves persist
TC-ONE-C-021	Settings → Key shape → each of 5 shapes → shot per shape → persists restart
TC-ONE-C-022	overflow → `ocr menu == exactly 6 items in order`
TC-ONE-C-023	shot pad before → Key theme → Rose → shot after → pixel-diff ≥ 10% · restart → Rose
TC-ONE-C-024	Effects → pick Bubbles → `2+2 =` → animation frames captured · persists
TC-ONE-C-025	focus expression, `adb shell ime list` untouched · typing via keys only, no IME window
TC-ONE-C-026	light shot + dark shot → token asserts (expr ink, amber answer, caret color)

Module smoke scripts (one line each): note create→edit→search→restart; scan import→crop→export PDF; event+15-min alarm on test clock; backup→wipe→restore identity; music import→play 5 s→pause; record 10 s→flag→play; hub search hits 3 modules; mycalc parity C-001…008; flavor zero-permission manifest; airplane cold start all modules.

Appendix R — Journey step tables (J-01…J-10)

Journey	Step	Action	Assert
J-01 Scan to note	1	import fixture image	page appears in review
	2	run OCR (pack on)	text card ≥ 92% accuracy
	3	edit one line	text saved
	4	→ Notes	note created, body contains text
	5	open Files → Scans	image present; note link visible in note metadata
J-02 Record to minutes	1	record 30 s	FGS notification ≤ 200 ms
	2	flag at ~10 s	marker row exists
	3	Done	file + row; playable
	4	Transcribe	segments monotonic; text non-empty
	5	→ Notes	linked note with transcript + markers
J-03 Backup/wipe	1	create data (notes/events/recording)	counts baseline
	2	backup (pass `test-1234`)	`.msone` in Backups; manifest hashes ok
	3	`pm clear`	fresh state
	4	restore from backup	counts match; hashes match; history restored
	5	wrong passphrase attempt	clean failure, original data untouched
J-04 Calculator day	1–12	C-001…C-026 compressed set	all pass in one session
J-05 Airplane pass	1	airplane ON, cold start	hub interactive
	2	one action per module	all succeed; logcat: zero net attempts
J-06 Death storm	1	draft note A, calc expr B, start recording C	states active
	2	`am kill` ×3	relaunch each time
	3	reopen all three	A text, B expr+caret, C still recording (FGS) or finalized cleanly
J-07 Fresh low-end	1	E26C 2 GB + battery saver	smoke suite green
	2	full J-04 subset	no ANR (watch checks)
J-08 TalkBack day	1	enable TalkBack	J-04 subset + note create completable
	2	scan a11y tree	no unlabeled controls on visited screens
J-09 MyCalc parity	1	install flavor	home = calculator
	2	C-001…008	identical outcomes
	3	dex scan	zero module classes; zero cross-module strings
J-10 Storage-full	1	fill storage < 50 MB	banner once
	2	try record	graceful refusal, message
	3	try scan	graceful refusal at shutter
	4	verify existing data	no corruption (quick_check ok)

Appendix S — Engine reference tables

Operator precedence (high→low): `! %` (postfix) → unary `−` → `^` (right) → `× ÷` implicit → `+ −`. Parentheses override; functions bind like unary.

Token class	Members	Lexer notes
NUM	`0-9 .`	one dot; _ separators accepted in paste, stripped
OP	`+ − × ÷ ^`	unicode + ASCII aliases normalized
BRACKET	`( )`	depth cap 32 (TooDeep → syntax error)
CONST	`π e`	exact rationals? no — inexact path entry
FUNC	`sin cos tan asin acos atan ln log √`	open-bracket inserted
POSTFIX	`! %`	domain/context rules §17.1.5

Error cards (exact copy): `Cannot divide by zero` · `Outside domain (e.g., log of a negative number)` · `Can't calculate this yet — check the highlighted part` · `Expression is too long` · `Result is too large to display` · `Number too deep in brackets`.

Selection state machine: `IDLE →(long-press 300 ms) PILL_SHOW →(slide) RESIZING →(lift) PILL_SHOW →(tap inside) CARET_MOVE → IDLE`; `AC` from PILL_SHOW → CLEAR_SELECTION → IDLE; history replay/`=` → IDLE (selection cleared).

Live-answer pipeline: keystroke → store update → engine job (cancel previous) ≤ 1 ms → render. Budget breach (> 1 ms measured over rolling 50 ops) → job hops to Default with 50 ms cap, result annotated in diagnostics only.

Appendix T — Thirty-second demo script per feature (owner acceptance aid)

Feature	How to see it in 30 seconds
F-001 precedence	`2+3×4=` → 14
F-002 implicit	`2(5+2)=` → 28
F-003 percent	`200+10%=` → 220
F-004 pending op	`5+=` → 10, again → 15
F-007 grouping	type `123456` → `1,23,456`; copy → bare
F-008 exactness	`1÷3×3=` → 1
F-010 live answer	type `2+3` → amber 5 under field
F-012/13 selection	hold `123456`, slide to grow/shrink
F-019/20 memory	select number → MAdd → select in expr → MUse
F-021/22 history	three solves → history → tap top card
F-027 themes	⋮ → Key theme → Rose
F-028 shapes	⋮ → Key shape → Circle
F-031 effects	⋮ → Effects → Bubbles → `2+2=`
F-035 dark mode	system dark toggle → amber answer, white caret
F-041 hub search	hub → "rent" → grouped hits
F-047 notes	new note → checklist → home → back
F-057 OCR	scan fixture page → text card
F-061 recurrence	event → Repeat → Monthly on 31st → Feb preview skips
F-062 alarm	event in 2 min, alert 1 min → heads-up
F-067 backup	backup → wipe → restore
F-070 playback	import folder → play → speed 1.5×
F-073 recorder	record → pause → resume → flag → done
F-076 transcribe	recording → Transcribe → segments
F-081 packs	Diagnostics → import dummy pack → verified chip

Appendix U — Severity, priority and SLA matrices

Severity	Definition	Examples	Release rule	Fix SLA (agent-iterations)
S1	Crash, data loss, security violation	restore wipes data; engine wrong on money math	blocks release, always	immediate
S2	Core feature broken, no workaround	percent wrong; history lost on restart	blocks without waiver	≤ 2 iterations
S3	Feature degraded, workaround exists	theme not persisting on one OEM path	gate-report waiver	≤ 5
S4	Polish	spacing, copy tweak	never blocks	backlog

Priority (P0–P3 for defects) = severity × feature priority (§10) lookup table in `qa/severity.yaml`; disputes resolve toward the higher. Rejected defects require a reason class: `not-a-defect`, `as-designed` (ledger ref), `cannot-reproduce` (evidence attached), or `duplicate` (id).

Appendix V — Android/compose glossary for the owner

Term	Plain meaning
APK	the installable app file
AAB	Play-store format (not used here — sideloads need APK)
API level	Android version number (26 = Android 8, 33 = 13, 35 = 15)
Compose	the UI toolkit the app is built with
Room	the on-device database layer over SQLite
FTS	full-text search index inside that database
DataStore	small settings storage
FGS	foreground service — the persistent notification for recording/playback
SAF	the system file-picker permission model for folders
MediaStore	Android's shared media index (music import path)
TalkBack	Android's screen reader
Doze	the system's deep-sleep power state
R8	the code shrinker/optimizer for release builds
Robolectric	JVM tests that fake Android
Espresso/UIAutomator	on-device UI test drivers
Roborazzi	screenshot test tool
adb	the device control bridge the QA harness uses
SHA-256	the file fingerprint used for provenance
ANR	"application not responding" system dialog

Appendix W — Executor quick-start (first working session)

```
1. Clone the repo; open docs/blueprint/myspace-one-development-blueprint.pdf
2. Read: §front, Ch 1, Ch 15, Ch 19  (30 min)
3. Initialize the task graph: docs/plan/task-graph.yaml with G1 slice (§19.6)
4. Bootstrap: `gradle wrapper && gradle assembleFullDebug assembleMycalcDebug`
5. Verify CI fast lane green on an empty-hub skeleton commit
6. Enter the G2 loop; per task: read binding §, write tests, implement, verify, commit with Re
    fs
7. End of session: task graph statuses + session log (Appendix L template)
Cadence rules: never skip local verification; never commit red; evidence paths in every task r
    ow.
```

The build is designed so that *any* competent executor — human or AI — can resume from the task graph + ledger alone. That property (resumability without oral history) is itself a success criterion of the blueprint.

Appendix X — Document control

Field	Value
Title	MySpace One — Application Development Blueprint
Version	1.1
Status	Approved for build execution (G0 artifact)
Date	31 October 2026
Author	Prepared by the AI build agent program for the owner of tarungittarun/AIA-
Audience	Autonomous build agent (primary); product owner; future maintainers
Source files	`docs/blueprint/content/*.md` (this PDF is generated; regenerate, never hand-edit)
Build command	`.venv/bin/python tools/blueprint/build_pdf.py --content docs/blueprint/content --out docs/blueprint/myspace-one-development-blueprint.pdf`
Figures	`tools/blueprint/figures.py` (regenerate before build if diagrams change)
Revision policy	Every feature/spec change lands its doc delta in the same merge (docs-drift CI gate §24.4)

Revision history:

Ver	Date	Change
1.0	31 Oct 2026	Initial complete blueprint: Parts I–XII, 32 chapters + appendices A–X
1.1	31 Oct 2026	Line-by-line correction pass (all § refs resolved, draft-style notes removed, tables normalized); new UI design-language material §6.12–§6.19 incl. Figma workflow, screen/frame registry, redlines and design-QA gate; 20 figures; appendices regrouped (M–P joined E–P); screen inventory superseded by §6.14

End of document.

Appendix Y — Owner's personal acceptance manual (module by module)

How the owner verifies each module by hand in ~15 minutes total. The automated suites say the same things; this appendix translates them into thumb-actions.

Calculator (4 min). Open the app. Type `2+3×4` — expect `14` live. `=` — history appears. Type `123456` — expect `1,23,456`. Copy it, paste into Notes — expect bare `123456`. Hold on part of the number — pill appears; slide to grow/shrink; MAdd. Type `3+4`, select `4`, MUse — expect `3+123456` → `1,23,459`. `5+ =` — expect `10`. `1÷3×3=` — expect exactly `1`. Open ⋮ — six items exactly. Key theme → Rose — keys recolor instantly. Swipe down — history. Restart the app — history and theme survive.

Notes (2 min). New note → checklist "milk, rice, turmeric" → check two → list shows 2/3. Kill app mid-typing → reopen → text intact. Search "milk" → hit with highlight. Delete → Undo comes back. Trash shows it 30 days.

Scanner (2 min). Import an image (or shoot a page). Crop applies. OCR runs — text editable. → Notes creates the note. Without the Essentials pack: the button explains itself and capture/export still work.

Calendar (2 min). New event tomorrow 13:36, alert 15 min, repeat monthly on the 31st — preview shows Feb skipped. Notification appears at 13:21 (test with a 2-minute event). ICS export into Files, delete event, import back.

Files (2 min). Add a SAF folder. Rename/move a file. Analyzer shows the donut. Backup → wipe data → restore with passphrase — everything returns. Wrong passphrase — clean failure.

Music (1.5 min). Import folder → play → 1.5× speed → sleep timer 15 min → headset button next/prev. MiniPlayer persists while checking a note.

Recorder (1.5 min). Record 30 s → flag mid-way → done. Play, tap marker to seek. Transcribe (Speech pack) → segments with timestamps → → Notes.

Diagnostics + About (1 min). Run self-test — six greens. About shows the exact commit and APK SHA-256 printed in the release notes. Model packs panel shows what's installed with SHAs.

Privacy spot-check (1 min). Airplane mode ON. Everything above still works. App info → Permissions — the full app shows only the expected set; MyCalc shows none.

Flavor (1 min). Install MyCalc beside MySpace One — separate icon, calculator-first, zero module mentions, zero permissions.

Appendix Z — Model cards (pack provenance summaries)

Shipped with each pack (full cards in pack LICENSE/cards.md; summaries normative here):

Essentials (OCR + labels + embeddings). Models: DBNet-lite (text detection), CRNN en (recognition), MobileNetV3 (labels), MiniLM-L6 int8 (embeddings). Intended use: printed/hand-printed Latin text in photos of pages, labels for photo organization, semantic search over the user's own notes. Out of scope: handwriting cursive, non-Latin scripts v1, tabular extraction. Eval: §16.8 thresholds (92% char acc, recall@10 0.8). Limitations: low-light/blurred images degrade; confidence badges surface doubt. Data: models are static weights; the app sends them nothing.

Speech (whisper). Model: whisper-tiny int8 (base optional Tier A). Intended use: transcribing the user's own recordings, en-first with autodetect within whisper's language set. Out of scope: real-time captioning v1, speaker diarization. Eval: ≤ 12% WER fixture set. Limitations: heavy accents/noise raise WER; timestamps monotonic but segment boundaries approximate.

Assistant (small LLM). Model: gemma-2B-it 4-bit (alt: Qwen2.5-1.5B-it 4-bit) via MediaPipe. Intended use: summarize/rewrite/checklist-ify the user's selected text; parse one event from a phrase. Out of scope: open-ended chat, factual QA (no knowledge source on device), generation without preview. Eval: rubric ≥ 4/5 on fixture notes; JSON schema discipline 100% after retry. Limitations: ~1.6 GB RAM class; Tier A only; outputs always previewed with undo; provenance recorded.

Appendix AA — Program FAQ

Q: Why 26 minimum when most phones are newer? The owner distributes to real-world mixed hardware; Android 8 covers the tail cheaply (we avoid APIs that would force 30+).

Q: Why Koin over Hilt? Build speed and simpler kapt-free setup on the constrained CI (ADR-04); both would satisfy the architecture.

Q: Why is `5+ = → 10` a release blocker? It is the owner's named parity behavior (Google Calculator rule) and a historical pain point; F-004/TC-ONE-C-004 encode it.

Q: What if the AI packs never ship? The product ships and passes release without them; every AI affordance degrades per §16.7. Packs accelerate later, never gate now.

Q: Why no WorkManager for reminders? Exact alarms need AlarmManager anyway; one scheduling path beats two half-paths (ADR-10).

Q: Can the agent add a module the owner didn't list? No (§1.4). Proposals go to the ledger as decisions for the owner, not into the build.

Q: Why FTS4 not FTS5? API 26's SQLite ships FTS4 unconditionally (ADR-02); porting to FTS5 can be a v2 decision after floor-raising.

Q: How big can my library/notes get? Design targets: 50k tracks, 20k notes, 5k recordings stay within budgets (§23.6); beyond that, performance work is a ledger task, not a surprise.

Q: What stops the agent from "green-washing" a failed run? The report builder only reads manifest-verified artifacts; publisher success is structurally separate from case results (§22.5); the owner reads per-case tables, not job dots.

Q: How do I (owner) request a change? File it against the blueprint: a spec delta PR + traceability update; the agent implements in the normal loop. Changes to constraints (C1–C10) need an explicit owner decision entry.

Appendix BB — Index of figures

Fig	Title	Section
Cover	Honeycomb emblem (brand motif)	cover
1	Combined-app product map	§5.1
2	Navigation graph	§5.2
3	Figma workflow (design↔code loop, access policy)	§6.13
4	Layered architecture	§11.1
5	Calculator basic pad + scientific panel	§7.2
6	History / memory / key-theme surfaces	§7.2.5
7	Notes list + editor	§7.3
8	Scanner viewfinder + OCR card	§7.4
9	Calendar month+agenda + editor	§7.5
10	Files browser + analyzer	§7.6
11	Music library + player	§7.7
12	Recorder capture + saved view	§7.8
13	Diagnostics + model packs	§7.9
14	Room schema ERD	§12.2
15	Phase-gate lifecycle	§15.5
16	AI stack (orchestrator, runtimes, tiers)	§16.4
17	Device capability ladder	§16.8
18	Test pyramid + CI pipeline	§20.1
19	Defect lifecycle	§20.9

Tables are numbered by their section (e.g., "the §17.1.5 table"); the executive anchor tables are: constraints §1.3, promises §1.2, Catalogue-0 §20.6, settings registry §17.7/§17.12, budgets §23.6, risks Ch 30, screen registry §6.14, traceability §27.2.

Appendix CC — Constraint → architecture traceability (why the structure is what it is)

Constraint	Architectural embodiment	Fail-loud mechanism
C1 no internet permission	manifest allow-list; Konsist no-network rule; dex scan	CI gates (SC-01/03)
C2 no sign-in	no auth code path exists; About/identity only local	code review checklist §18.4
C3 no ads	dependency deny-list audit	CI dependency-audit
C4 private data	ADR-09 storage policy; SAF only user-granted	TC-SEC-* suite
C5 packs never download	no download code; import+hash only	TC-PACK-sec-01
C6 API 26 floor	compileSdk 36/minSdk 26; E26C in matrix	emulator matrix
C7 calculator parity	§17.1 normative engine; Catalogue-0	release lane L5 red = stop
C8 ≤ 30 jobs / 3 runners	§24 sharding design; concurrency groups	job-count metric in metrics.json
C9 calculator-only flavor	Gradle flavor + dex scan (F-040)	CI mycalcNoCrossModule
C10 provenance everywhere	BuildConfig SHA; evidence manifests; About screen	S-criteria artifacts
P1 offline-first	airplane-mode CI cases; no network vocabulary in strings	J-05, UX-015
P2 privacy structural	§14 threat model; §14.2 structural bans	SC checks 01–12
P3 one home	design-system module; nav audit	TC-NAV-001, screenshots
P4 fast & gentle	§23.6 budgets as CI asserts	perf lanes
P5 verifiable	Chapters 20–22; §27 traceability	every gate

Appendix DD — Traceability CSV extended excerpt (format example)

```
feature,area,priority,spec,tests,latest_status,commits,notes
F-001,CALC,P0,§17.1,TC-ONE-C-001;TC-LAW-002,pass,ab12ef0,
F-002,CALC,P0,§17.1.4,TC-ONE-C-002;TC-LAW-005,pass,ab12ef0,
F-003,CALC,P0,§17.1.5,TC-ONE-C-003;TC-LAW-004,pass,cd23f41,
F-004,CALC,P0,§17.1.5,TC-ONE-C-004,pass,cd23f41,owner bug class closed
F-008,CALC,P0,§17.1.2,TC-ONE-C-008;TC-LAW-003,pass,ab12ef0,owner bug class closed
F-013,CALC,P0,§7.2.2,TC-ONE-C-B2,blocked,ef45a62,physical-device input required
F-019,CALC,P0,§7.2.2,TC-ONE-C-015,pass,ef45a62,
F-024,CALC,P0,§17.2,TC-ONE-C-013,pass,ff9012a,
F-027,CALC,P0,§6.2,TC-ONE-C-023,pass,112a3b4,
F-032,CALC,P0,§7.2.6,TC-ONE-C-022,pass,112a3b4,
F-036,NCALC,P0,§23.4,TC-SEC-perm-flavor;J-09,pass,22b4c5d,
F-038,NCALC,P0,§14.4,TC-SEC-perm-flavor,pass,22b4c5d,zero-permission verified
F-041,HUB,P0,§5.3,TC-HUB-001,pass,33c5d6e,
F-047,NOTE,P0,§17.3,TC-NOTE-rt;TC-NOTE-save-01,pass,44d6e7f,
F-050,NOTE,P0,§14.3,TC-NOTE-trash-01,pass,44d6e7f,
F-055,SCAN,P0,§7.4,TC-SCAN-cap-01;TC-SCAN-imp-01,pass,55e7f80,
F-057,SCAN,P0,§16.5,TC-SCAN-ocr-01,pass,55e7f80,pack-gated
F-059,SCAN,P0,§16.2,TC-SCAN-abs-01,pass,55e7f80,
F-060,CAL,P0,§7.5,TC-CAL-alarm-01,pass,66f8091,
F-061,CAL,P0,§17.4,TC-CAL-rr-01,pass,66f8091,200-rule corpus
F-062,CAL,P0,§14.4,TC-CAL-alarm-01;02,pass,66f8091,
F-065,FILE,P0,§7.6,TC-FILE-browse-01,pass,77091a2,
F-067,FILE,P0,§14.3,TC-SEC-backup-01..03,pass,77091a2,
F-069,MUSIC,P0,§7.7,TC-MUSIC-imp-01,pass,881a2b3,
F-070,MUSIC,P0,§7.7,TC-MUSIC-play-01,pass,881a2b3,
F-073,REC,P0,§17.5,TC-REC-svc-01,pass,992b3c4,
F-075,REC,P0,§17.5,TC-REC-fmt-01,pass,992b3c4,
F-081,DIAG,P0,§28.3,TC-DIAG-log-01,pass,aa3c4d5,
F-082,DIAG,P0,§16.8,TC-PACK-mgr-01;TC-PACK-sec-01,pass,aa3c4d5,
F-084,SET,P0,§7.10,TC-SET-about-01,pass,bb4d5e6,
UX-002,UX,P0,§8,TC-perf-coldstart,pass,perf-lane,
UX-015,UX,P0,§8,strings-audit,pass,ci-grep,
UX-019,UX,P0,§8,TC-flavor-strings,pass,ci,
(… full matrix generated per gate: reports/traceability.csv …)
```

Appendix EE — "If something fails" (owner-facing, honors the old guide's contract)

The owner's original guides said: note the section code, use Settings → Diagnostics → Share log, attach a screenshot. The combined app keeps that contract, upgraded:

1. **In-app self-test first.** Diagnostics → Run self-test: six checks, one JSON card. Share it — it is the fastest triage artifact.
2. **Share the log (previewed).** Diagnostics → Logs → Share: you see the exact text before anything leaves via the share sheet. Section codes from the guides map to module tags in each log line (`[calc]`, `[scan]`, …).
3. **Screenshot the moment.** The failure state is designed to be self-describing (error card + one action), but your screenshot anchors it.
4. **What gets asked, and why.** Version + commit (About), device model, airplane-or-not, pack status if AI-related. With those four facts plus the JSON, nearly every defect reproduces from this blueprint's fixtures alone.
5. **What never happens.** No remote debugging, no account checks, no silent uploads — by architecture. Support is evidence you hand over, nothing more.

Appendix FF — Extension guide (adding a module in v1.1+)

A worked recipe so future growth (Widgets, Voice search, non-Latin OCR…) follows the same laws:

1. **Spec first.** Add a Chapter-9-style feature table with IDs continuing the catalogue; write acceptance criteria before code; extend `qa/testcases.csv`.
2. **Module scaffold.** `feature/<name>/` + contracts in `core/domain`; register in DI + navigation + hub grid (one row in §5.6) + settings registry (§17.12 keys).
3. **Design-system compliance.** Screens use only tokens/components; empty/error/content states; screenshot goldens; a11y labels.
4. **Data rules.** New tables via numbered migration + fixture test; trash/backup coverage (backup `data.json` gains the table; older apps reject newer backups cleanly — already specified).
5. **AI hooks (optional).** New capability → orchestrator `Capability` + pack tier + fallback rung + eval fixtures; never a direct model call in the feature module.
6. **Quality gates.** Journeys, budgets (§23.6 rows for new screens), architecture tests pass; traceability rows complete; blueprint delta lands in the same merge.
7. **Release.** Feature flag via settings registry until Catalogue-0-class set is green; then promote to P0-class in the next minor.

Appendix GG — Document generation pipeline (how this PDF is built)

```
docs/blueprint/
  content/*.md          # source chapters/appendices (single source of truth)
  figures/              # generated diagrams (tools/blueprint/figures.py)
  myspace-one-development-blueprint.pdf   # generated output
tools/blueprint/
  figures.py            # matplotlib generators → figures/*.png
  build_pdf.py          # markdown → typeset PDF (DejaVu fonts, TOC, bookmarks)
```

Regenerate: `python tools/blueprint/figures.py && python tools/blueprint/build_pdf.py --content docs/blueprint/content --out docs/blueprint/myspace-one-development-blueprint.pdf`. Rules: never hand-edit the PDF; content changes land with their feature changes (docs-drift gate); figures regenerate deterministically (seeded); the PDF asserts ≥ 200 pages at build time so trimming by accident fails loudly.

End of appendices.

Appendix HH — Build files (bootstrap reference)

`settings.gradle.kts`:

```
pluginManagement { repositories { google(); mavenCentral(); gradlePluginPortal() } }
dependencyResolutionManagement {
    repositories { google(); mavenCentral() }
}
rootProject.name = "myspace-one"
include(":app",
        ":feature:calculator", ":feature:notes", ":feature:scanner",
        ":feature:calendar", ":feature:files", ":feature:music",
        ":feature:recorder", ":feature:diagnostics", ":feature:hub",
        ":feature:settings",
        ":core:designsystem", ":core:domain", ":core:data",
        ":core:ai", ":core:common", ":core:testing")
```

Root `build.gradle.kts` (conventions excerpt):

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
    alias(libs.plugins.kotlin.compose) apply false
    alias(libs.plugins.room) apply false
}
subprojects {
    // shared: kotlin jvm target 17, minSdk 26 via convention plugin
}
```

`app/build.gradle.kts` key block (flavors + provenance; see §23.7 for the full DSL):

```
android {
    namespace = "app.space.one"
    compileSdk = 36
    defaultConfig {
        minSdk = 26; targetSdk = 35
        versionCode = 31300; versionName = "3.13.0"
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }
    // flavors per §23.7; signing per §25.2
}
```

Module convention (each `feature/*`):

```
plugins { id("myspace.android.feature") }   // convention plugin: compose + koin + lint
android { namespace = "app.space.one.feature.calculator" }
dependencies {
    implementation(project(":core:designsystem"))
    implementation(project(":core:domain"))
    implementation(project(":core:data"))
    testImplementation(project(":core:testing"))
}
```

`core/domain` stays a pure Kotlin (`java-library` + kotlinx-coroutines-core only) module — its `build.gradle.kts` must not contain `android.*` blocks at all, enforced by a CI check (the architecture test that keeps the engines JVM-testable).

Appendix II — G1 day-by-day plan (first sprint, executor-ready)

Day	Tasks (§19.6)	Definition of done for the day
1	T-BOOT-001 workspace	`gradle help` green; version catalog resolves; wrappers pinned
2	T-BOOT-002 flavors	both flavors assemble empty activities; namespaces correct
3	T-BOOT-003 CI fast lane	PR lane green on GitHub Actions runners ≤ 12 min
4	T-BOOT-004 empty hub	hub renders on E33B; screenshot test records golden
5	T-BOOT-005 provenance	BuildConfig SHA/time visible in a debug About row
6	T-BOOT-006 keystore	generated in secret store; owner backup receipt filed; rehearsal signed test APK
7	T-BOOT-007 screenshot harness	Roborazzi runs in lane; diff artifacts upload on failure
8	T-BOOT-008 G1 report	gate report committed with artifact paths; defects/ledger current

Rules for the sprint: no task starts without its dependency closed; every day ends with a green tree (or a quarantined flake with an owner); the report is written by the evidence, never from memory. This sprint is the template for every later one — plan, execute, verify, record.

Quick reference card

The whole program on two spreads — print this page and the next; they are the daily working surface for both the agent and the owner.

The five promises and ten constraints (from Chapters 1)

Promise	One-line test
P1 Works everywhere, always	airplane mode cold start, Tier C, first run — all green
P2 Private by architecture	no INTERNET permission, no network classes in dex
P3 One home, zero friction	every screen ≤ 2 taps; one design system
P4 Fast and gentle	hub ≤ 1.2 s cold; no jank; no surprise notifications
P5 Built to be verified	every claim has an artifact path

Constraint	Guard
C1 no internet permission	manifest-audit CI gate
C2 no sign-in	checklist + code audit
C3 no ads/tracking SDKs	dependency deny-list
C4 data stays private/SAF	TC-SEC-*
C5 packs side-loaded, hashed	TC-PACK-sec-01
C6 Android 8.0+ (API 26)	E26C in matrix
C7 calculator parity is blocking	Catalogue-0
C8 ≤ 30 jobs, ≤ 3 runners	CI config + metrics
C9 MyCalc flavor from same code	flavor gates
C10 provenance everywhere	S-criteria artifacts

Command cheat sheet

Task	Command
Regenerate figures	`python tools/blueprint/figures.py`
Rebuild this PDF	`python tools/blueprint/build_pdf.py --content docs/blueprint/content --out docs/blueprint/myspace-one-development-blueprint.pdf`
Fast local checks	`./gradlew spotlessCheck detekt testDebugUnitTest`
Full local suite	`./gradlew test robolectricSuite roborazziVerify`
Device journeys	`./gradlew connectedShadedDebugAndroidTest`
Black-box QA (any APK)	`python3 qa/harness.py --profile release --apk <file> --pin-sha <sha>`
Traceability	`python3 tools/traceability.py --check`
Audits	`python3 tools/audits.py manifest dependencies docs-drift`
Verify a bundle	`python3 tools/verify_manifest.py releases/v<ver>/`

Where everything lives

Want	Go to
Product identity & constraints	Part I (Ch 1–3)
Screen behavior	Part II (Ch 7 + wireframes)
Feature list with acceptance	Part III (Ch 9)
Architecture / data / security	Part IV (Ch 11–14)
Agent rules & gates	Part V (Ch 15) + Part VII (Ch 18–19)
AI models, packs, fallbacks	Part V (Ch 16)
Engine grammar, RRULE, services, settings	Part VI (Ch 17)
Tests and harness	Part VIII (Ch 20–22)
Build, CI, release, ops	Part IX (Ch 23–26)
Traceability, risks, glossary, sign-off	Part X (Ch 27–32)
Step-by-step scripts	Appendices Q, R, Y
Task/defect/decision templates	Appendix L

The program in one sentence each

- **Build**: one Kotlin/Compose codebase, two flavors, six layers, one design system.
- **Data**: one Room DB + DataStore + private files; backups are portable encrypted zips.
- **AI**: three side-loaded packs behind one orchestrator; features never touch runtimes.
- **Test**: five layers, evidence always, BLOCKED honest, Catalogue-0 blocks releases.
- **Ship**: reproducible signed APKs with manifests, PDFs of record, and provenance in the About screen.
- **Operate**: Diagnostics is support; the ledger is memory; the blueprint is law.

Closure

This blueprint set out to do one unusual thing: hand an autonomous agent the complete, unambiguous, testable definition of a real product — a private, offline, combined tool that respects its user absolutely — together with the working method that keeps such an agent honest over a long build. Every chapter serves that single goal. The agent that executes it will make thousands of small decisions; the constraint tables, the acceptance criteria, the harness contract and the ledger discipline exist so that every one of those decisions lands in the same place this page does: **provable, reproducible, and owned.**

When the final gate report is signed, the measure of success is simple: the owner installs one APK on an offline phone, and everything — calculation, notes, scans, plans, files, music, recordings, diagnostics, and the quiet intelligence of on-device models — simply works. Nothing more was promised; nothing less will ship.

Phase-gate checklist card

Print or keep open during execution; a gate closes only when every box has an artifact path written next to it.

G0 Inception

- [ ] Blueprint accepted by owner (this PDF, version recorded)
- [ ] Task graph seeded for G1 (`docs/plan/task-graph.yaml`)
- [ ] Decision ledger file created (`docs/plan/decisions.md`)

G1 Bootstrap

- [ ] Both flavors assemble (full + mycalc, debug)
- [ ] Empty hub renders on E33B emulator (screenshot evidence)
- [ ] CI fast lane green (build + lint + unit, ≤ 12 min)
- [ ] Keystore generated; owner backup receipt; restore rehearsal signed test APK
- [ ] Screenshot harness records first golden
- [ ] G1 gate report committed

G2 Foundations

- [ ] Token table + token unit test (Kotlin == tokens.csv)
- [ ] Design-system components with screenshot goldens (light/dark)
- [ ] Room v1 schema + exported-schema tests + DAO tests
- [ ] Settings registry + RestoreKeys
- [ ] Architecture tests active in fast lane (Konsist-style)
- [ ] G2 gate report committed

G3 Calculator

- [ ] Engine: §17.1 normative tables implemented; laws L1–L10 green (10k cases/law)
- [ ] Catalogue-0 CALC cases TC-ONE-C-001…026 green (L4)
- [ ] Flavor parity J-09 green; mycalc dex has zero module classes
- [ ] Perf: keystroke→frame ≤ 16 ms; cold start ≤ 0.8 s (flavor)
- [ ] L5 calculator campaign on dist APK: results JSON + screenshots + manifest
- [ ] G3 gate report committed

G4 Modules

- [ ] Notes / Scanner / Calendar / Files / Music / Recorder / Hub P0 sets green
- [ ] Journeys J-01…J-10 green (AI mocked)
- [ ] Edge registers (§7.14) covered in L1/L2/L4
- [ ] Storage-full drill J-10 green
- [ ] G4 gate report committed

G5 On-device AI

- [ ] Pack manager lifecycle + hash gates (TC-PACK-*)

- [ ] OCR / ASR / embeddings / LLM acceptance per §16.5 on tier profiles
- [ ] Fallback ladder rungs forced and asserted (TC-AI-fallback-*)
- [ ] Eval kit thresholds met for pinned pack versions (JSON evidence)
- [ ] Features green with packs absent (structural optionality)
- [ ] G5 gate report committed

G6 Hardening

- [ ] TC-A11Y-* battery green; TalkBack journeys
- [ ] Perf budgets §23.6 + feel budgets §7.18 on Tier B/C profiles
- [ ] Security checklist SC-01…SC-12 verified
- [ ] String/copy audits (UX-015; microcopy dictionary exact)
- [ ] Battery/Doze + rotation + death-storm suites green
- [ ] G6 gate report committed

G7 Release

- [ ] S1–S8 all true on the exact release artifacts
- [ ] Reproducible double-build SHA match
- [ ] L5 release-candidate profile green (both flavors, airplane-mode profile included)
- [ ] Evidence bundle manifest verified; PDF of record built
- [ ] Docs regenerated (blueprint, traceability, licenses, api-surface)
- [ ] Owner walkthrough §32 accepted; sign-off record filled
- [ ] Release published per §25.5 with RELEASE-NOTES

G8 Operate

- [ ] v1.1 backlog seeded (P2 + ledger wishes)
- [ ] Defect ledger triaged; runbook committed
- [ ] Maintenance calendar scheduled (§29.8)
- [ ] G8 gate report committed